CS106L: Standard C++ Programming — 现代 C++ 学习笔记

> **课程**：Stanford CS106L "Standard C++ Programming"（2026 春季学期） **讲师**：Rachel Fernandez、Preston Seay（`cs106l-spr2526-staff@lists.stanford.edu`） **课程主页**：https://web.stanford.edu/class/cs106l/（存档页：https://web.stanford.edu/class/archive/cs/cs106l/cs106l.1266/） **上课时间**：周二、周四 15:00–16:20，Thornton 110 **课程性质**：1 学分（S/NC），第 8 周结束正式课程，共 7 个短作业、无考试 **本笔记依据**：课程主页公开日程表、全部 17 份公开讲座幻灯片（PDF）、公开的课堂代码仓库（cs106l-lecture-code）与作业仓库（cs106l-assignments）

课程概览（About CS106L）

课程目标与定位

🌽 **CS106L 是一门深入探索现代 C++ 语言的 1 学分课程。** 与 CS106B 等"用 C++ 讲概念"的课程不同，CS106L 关注的是**代码本身**：什么是好的、强大而优雅的 C++ 代码。课程覆盖现代 C++ 中最激动人心的特性，包括现代编程范式（**直至 C++26**），并只使用标准库（STL），不依赖任何 Stanford 私有库——让你真正理解 C++ 是如何被设计出来的，以及为什么。

- 🥦 **先修要求**：正在学习或已学过 CS106B/X（或同等水平），即已掌握函数、对象/类等编程基础。
- 🥕 **课程形式**：7 个非常简短的周作业（每个约 1–2 小时，周五发布、下周五截止，共 3 个免费晚交天数）。作业不追求难度，而是对上周课堂概念的动手练习。无考试、无论文，全部 S/NC 评分。
- 🏢 **答疑（Office Hours）**：对所有人开放，第 3 周开始。周四 16:30–17:20（Thornton 210）、周三 15:00–15:50（160-B37）。

为什么学 C++?

- C++ 是"一切事物的隐形地基"：游戏引擎（Valorant、CS2）、高频交易、自动驾驶、Arduino、GPU 编程、数据库（MySQL、MongoDB）、浏览器（Chrome）、VR（Quest）、底层 ML 框架（PyTorch、TensorFlow）、编译器/虚拟机（LLVM、GCC、JVM）、操作系统（Windows、macOS、Linux）……
- C++ 于 1985 年首次发布，至今仍稳居 TIOBE 指数前三（2026 年 3 月数据）；标准每三年修订一次：`C++98 → C++03 → C++11 → C++14 → C++17 → C++20 → C++23 → C++26`。
- C++ 帮助养成优秀的"编码卫生"：类型检查与类型安全、引用/拷贝/移动语义下的内存效率、`const` 正确性——这些限制正是许多其他语言所放宽的。

C++ 设计哲学（本笔记贯穿始终的准绳）

1. 直接在代码中表达想法与意图（express ideas and intent directly in code）。

2. 尽可能在**编译期**强制安全（enforce safety at compile time whenever possible）。
3. 不浪费时间和空间（do not waste time or space）。
4. 封装杂乱的结构（compartmentalize messy constructs）。
5. 把完全的控制、责任与选择权交给程序员（allow the programmer full control, responsibility, and choice）。

> "Code should be elegant and efficient; I hate to have to choose between those." —— Bjarne Stroustrup

学习方法建议

- 本笔记每讲严格按"**概述 → 核心特性与语法详解 → 代码示例与逐步解说 → 与旧标准对比 → 关键要点 → 常见陷阱 → 关联作业提示**"七段式组织；代码示例均标注所需 C++ 标准（C++11/14/17/20/23/26）且尽可能可直接编译运行。
- 建议：先通读概述与关键要点，再精读语法详解，最后动手编译运行每个代码示例，对照"特性机制解说"理解底层机制。
- 常用参考：https://en.cppreference.com（官方标准参考，课程推荐）；Godbolt Compiler Explorer（https://godbolt.org）用于观察编译产物。

课程日程表（Schedule）

周次	周二（Tuesday）	周四（Thursday）	周五发布作业
Week 1	3/31 **L1. Welcome!**（📖 Slides）	4/2 **L2. Types & Structs**（📖 Slides, 💻 Code）	—
Week 2	4/7 **L3. Initialization & References**（📖 Slides, 💻 Code）	4/9 **L4. Streams**（📖 Slides, 💻 Code）	**A1: SimpleEnroll**
Week 3	4/14 **L5. Containers**（📖 Slides, 💻 Code）	4/16 **L6. Iterators & Pointers**（📖 Slides, 💻 Code）	**A2: Marriage Pact**
Week 4	4/21 **L7. Classes**（📖 Slides, 💻 Code）	4/23 **L8. Optional: Inheritance Practice**（📖 Slides, 💻 Code）	**A3: Make a Class!**
Week 5	4/28 **L9. Class Templates & Const Correctness**（📖 Slides, 💻 Code）	4/30 **L10. Function Templates**（📖 Slides, 💻 Code）	**A4: Ispell**
Week 6	5/5 **L11. Functions & Lambdas**（📖 Slides）	5/7 **L12. Operator Overloading**（📖 Slides, 💻 Code）	**A5: Treebook**
Week 7	5/12 **L13. Special Member Functions**（📖 Slides, 💻 Code）	5/14 **L14. Move Semantics**（📖 Slides, 💻 Code）	**A6: ExploreCourses**

周次	周二（Tuesday）	周四（Thursday）	周五发布作业
Week 8	5/19 **L15. std::optional & Type Safety**（📖 Slides，💻 Code）	5/21 **L16. RAII, Smart Pointers, & Building C++ Projects**（📖 Slides，💻 Code）	**A7: Unique Pointer**
Week 9	5/26 **L17. Optional Lecture: C++ Iceberg**（📖 Slides，💻 Code）	5/28 无课	—
Week 10	无课	无课	—

作业总览（Assignments）

作业	名称	主题 / 考察技能	说明
A1	SimpleEnroll	流（streams）、初始化、引用	用 ExploreCourses 数据判断哪些 CS 课程本学期开设
A2	Marriage Pact	STL 容器、指针	用容器与指针完成"配对"逻辑（含简答题）
A3	Make a Class!	类、头文件/源文件分离	自由设计一个类（`class.h` / `class.cpp` / `sandbox.cpp`）
A4	Ispell	模板、`<algorithm>`、ranges	实现 Unix 拼写检查器核心逻辑（spellcheck）
A5	Treebook	运算符重载、特殊成员函数	实现社交网络 `User` 类的 `operator<<`、`operator==`、SMF 等
A6	ExploreCourses	`std::optional`、monadic 操作	在课程数据库中查找可能不存在的 `Course`，返回 `std::optional`
A7	Unique Pointer	RAII、智能指针、模板、移动语义	从零实现一个简化版 `std::unique_ptr`

> **访问性说明**：作业详细文档、练习视频、答题平台（Paperless、Ed、Canvas 内页）等需要斯坦福账号登录，未公开。本笔记主要依据课程主页公开的日程表、主题描述、公开幻灯片与公开 GitHub 代码仓库构建知识框架。

Lecture 1 (Week 1 – Tuesday): 欢迎与课程概览 (Welcome!)

概述

本讲是 CS106L（Stanford 的《Standard C++ Programming》，Spring 2026，教学语言为现代 C++，覆盖 C++26）的开篇课：讲师自我介绍、课程定位与评分方式，并用大量真实案例回答"为什么要学 C++"。核心内容包含三块：C++ 的历史与设计哲学（Readable、Safety、Efficiency、Abstraction、Programmer Choice 五大支柱）、"同一段 Hello World 在 C++ 中可以有汇编层 / C 层 / 现代 C++ 层三种写法"的语言分层观，以及整个学期的知识地图。

本讲没有新语法，但建立了一个贯穿全课程的心智模型：**C++ 是编译型、静态类型的语言，既追求效率又追求代码的优雅与可读性**。课程以现代标准（C++11 之后直至 C++26）为教学语言，全部作业只使用标准库（STL）——不依赖 Stanford 自制库。最后一部分是课程后勤：讲座时间与签到、8 个每周作业、S/NC 评分规则与联系方式，务必记牢。

核心特性与语法详解

本讲是概念课，这里把"核心特性"理解为支撑后续所有讲座的语言世界观与基本构件。

- **C++ 设计哲学（Bjarne Stroustrup 的五大原则）**
- **定义与目的**：这是理解 C++ 一切设计的钥匙——为什么 C++ 有那么多"看上去很麻烦"的规则。
- **核心内容**：
  1. *Express ideas and intent directly in code*（直接在代码中表达思想与意图）；
  2. *Enforce safety at compile time whenever possible*（尽可能在编译期强制安全）；
  3. *Do not waste time or space*（不浪费时间和空间——零开销原则）；
  4. *Compartmentalize messy constructs*（把丑陋的细节封装隔离，如内存管理）；
  5. *Allow the programmer full control, responsibility, and choice*（把控制权、责任与选择权交给程序员）。
- **设计意图与最佳实践**：Bjarne 的名言 "Code should be elegant and efficient; I hate to have to choose between those" 是课程灵魂——后续每讲（类型、引用、模板、移动语义……）都可以用这五条来解释"为什么语言要这样设计"。
- **C++ 的三种"合法程序"（语言分层观）**
- **定义与目的**：同一个任务，C++ 里可以写出汇编级、C 级、现代 C++ 级三种代码，且全部合法。这体现了 C++ 对 C 的**向后兼容**（backwards compatible）以及"程序员选择权"。
- **核心语法**：现代 C++ 版 `auto str = std::make_unique<std::string>("Hello World!");`；C 版 `printf("%s", "Hello, world!\n");`；汇编版 `asm("...");` 内联汇编。
- **设计意图与最佳实践**：你写的是哪一层，取决于需求（性能关键区 vs 业务逻辑）。CS106L 教的是**现代 C++ 层**——安全、可读、高效三者的平衡点。
- **现代 C++ 的几大语言构件（本讲先"混个脸熟"）**
- **定义与目的**：幻灯片用一行 Hello World 串起了本课程的核心主题。

- **核心语法**：
  - 流与运算符重载：`std::cout << *str << std::endl;`
  - 模板与类型推导：`std::make_unique<std::string>(...)`、`auto`
  - 智能指针：`std::unique_ptr`（RAII，自动管理内存）
  - 命名空间：`std::` 前缀
- **设计意图与最佳实践**：这些构件将在第 3 讲（初始化与引用）、第 4 讲（流）、第 11–12 讲（智能指针）逐一展开，本讲只需认识它们的"长相"。
- **C++ 标准演进史（C++98 → C++26）**
- **定义与目的**：C++ 自 1979 年起步、1985 年首次发布，至今仍是 TIOBE 前三的语言；标准委员会每三年修订一次标准。
- **核心内容**：1979（前身）→ 1983（C++ 诞生）→ 1998（C++98 首次标准化）→ 2003（C++03）→ 2011（C++11，现代 C++ 起点）→ 2014 → 2017 → 2020 → 2023 → **2026（We are here!）**。
- **设计意图与最佳实践**：本课程以 C++11 之后（含 C++26 新特性）的"现代 C++"为教学语言；编译时通常用 `-std=c++20` 或更新标准。
- **C++ 与 C 的历史关系**
- **定义与目的**：1972 年 Dennis Ritchie 创造 C（快、简单、跨平台），但缺少对象/类、泛型/模板能力，写大型程序繁琐；1983 年 Bjarne Stroustrup 在 C 的基础上创造了 C++，目标是**快 + 简单 + 跨平台 + 高层特性**。
- **设计意图与最佳实践**：理解"汇编的缺点 → C 的诞生 → C 的缺点 → C++ 的诞生"这条演进线，就知道 C++ 为什么"既要效率又要抽象"。
- **编译型语言的执行模型（预告）**
- **定义与目的**：C++ 是编译型语言：编译器先把**整个**程序翻译成机器码打包成可执行文件，再运行；因此有"编译期（compile time）"与"运行期（run time）"两个阶段。错误尽可能在编译期被发现（详细对比见 Lecture 2）。
- **设计意图与最佳实践**：越早发现错误成本越低——这是"编译期强制安全"哲学的体现。
- **为什么 C++ 无处不在（"The invisible foundation of everything"）**
- **定义与目的**：幻灯片列举了大量 C++ 的真实应用场景，回答"为什么值得学"。
- **核心内容**：游戏（Valorant、CS:GO/CS2）、高频交易、自动驾驶、Arduino 嵌入式、GPU 编程、数据库（MySQL、MongoDB）、浏览器（Chrome、Safari、Edge）、虚拟现实（Quest）、底层 ML 框架（PyTorch、TensorFlow）、编译器与虚拟机（JVM、LLVM、GCC）、操作系统（Windows、macOS、Linux）。
- **设计意图与最佳实践**：C++ 擅长**处理大量数据、且处理得非常高效、同时保持代码优雅可读**；学完 CS106L 后，CS111（操作系统）、CME213（并行计算）、CS143（编译器）、CS144（网络）、CS248A（图形学）、MUSIC256A 等课程都会直接用到 C++。
- **本课程的知识地图（Topics We'll Cover）**
- **定义与目的**：幻灯片给出了整个学期的路线图，帮助你建立全局观。

- **核心内容**：欢迎 → 类型与结构体 → 初始化与引用 → 流 → 容器 → 迭代器与指针 → 类 → 高级类 → 模板 → 高级模板 → 函数与 lambda → 运算符重载 → 特殊成员函数 → 移动语义 → `std::optional` 与类型安全 → RAII、智能指针 → C++ 项目实践。
- **设计意图与最佳实践**：每讲都有明确的前置依赖（如"移动语义"依赖"特殊成员函数"），按顺序跟进即可；本讲之后的第 2、3 讲（类型与结构体、初始化与引用）是全部内容的语法地基。
- **CS106L 与 CS106B 等课程的定位差异**
- **定义与目的**：CS106B 等课关注抽象、递归、指针等**概念**，只用 C++ 的最小集合并重度依赖 Stanford 自制库；CS106L 关注**代码本身**：只用标准库（STL），理解"好代码长什么样、C++ 为什么这样设计"。
- **设计意图与最佳实践**：CS106L 是 1 学分的"轻松"课程（S/NC 评分），但内容密度高；把注意力从"解决问题"转向"把解决方案写得优雅、正确、高效"。
- **课程后勤要点速览（Course Logistics）**
- **定义与目的**：课堂公布的管理规则，直接影响你的学分，值得抄进笔记。
- **核心内容**：
  - **讲座**：每周二、四 15:00–16:20，Thornton 110；讲座不录像；从第 2 周起每次开讲有 1–2 道签到小测，每人有 **2 次免签到额度**；签到二维码只出现在前 10 分钟幻灯片上；
  - **生病/特殊情况**：身体不适请留在家里并通过邮件/Ed 告知，绝不要求带病上课；
  - **Office Hours**：第 2 周前公布（线下），欢迎来聊天提问；
  - **作业**：8 个每周小作业（每个 1–2 小时），周五发布、下周五截止，每人有 **3 个免费晚交日**；
  - **评分**：S/NC（通过/不通过）——第 2 到第 9 周 14 次讲座中出勤 **12 次** + 完成全部 **8 个作业**，即可拿到 S；
  - **联系方式**：`cs106l-spr2526-staff@lists.stanford.edu`（课程统一邮箱）、Ed 论坛（经 Canvas 加入）、课后或 OH 面谈。
- **设计意图与最佳实践**：出勤与作业是硬指标，建议把"每讲扫码 + 每周五查作业"设为固定日程；有问题优先走 Ed 公开提问（他人也能受益）。
- **工具链与编译（Toolchain）**
- **定义与目的**：C++ 是编译型语言，写代码 → 编译 → 运行三步走；课堂上还用 godbolt.org 做了"编译器到底生成了什么汇编"的现场演示。
- **核心语法**：`g++ -std=c++20 main.cpp -o main` 然后 `./main`。
- **设计意图与最佳实践**：A1 起所有作业都用这个流程；用 `-std=c++20`（本课程教学标准）而非默认的旧标准；想理解"为什么 C++ 高效"，可以去 godbolt.org 看你的代码对应的汇编。

代码示例与逐步解说（核心）

示例 1：现代 C++ Hello World（C++14，改写自课堂幻灯片）

```
#include <iostream>
#include <memory>
#include <string>

int main() {
    // make_unique 返回一个 std::unique_ptr<std::string>
    auto str = std::make_unique<std::string>("Hello World!");
    std::cout << *str << std::endl;   // 解引用智能指针，输出字符串
    return 0;
}
// Prints "Hello World!"
```

代码做什么： – `#include <memory>`、`<string>`：引入智能指针与字符串类型； – `std::make_unique<std::string>("Hello World!")`：在堆上构造一个 `std::string`，返回管理它的 `std::unique_ptr`； – `auto str = ...`：让编译器从右侧表达式的类型推导出 `str` 的类型（`std::unique_ptr<std::string>`）； – `*str` 解引用得到字符串，`<<` 运算符把它送到标准输出，`std::endl` 换行并刷新缓冲区； – 函数结束、`str` 析构时，智能指针自动 `delete` 堆内存——**不需要手写 `delete`**。

特性机制解说： – `std::unique_ptr` 是 RAII（Resource Acquisition Is Initialization）思想的典型代表：内存的释放绑定在对象的析构上，杜绝 `new`/`delete` 配对遗漏导致的内存泄漏； – `auto` 是**编译期**完成的类型推导，`str` 的类型在编译时就固定为 `std::unique_ptr<std::string>`（静态类型，不是 Python 式的动态类型）； – `<<` 之所以能作用于自定义类型，靠的是**运算符重载**——标准库为 `std::ostream` 重载了针对各种类型的 `operator<<`。

示例 2：同一程序的三层写法对比（C++98 兼容 / 现代 C++）

```
// ① C 风格（C++ 向后兼容 C，这是合法的 C++ 程序）
#include <cstdio>
int main() {
    printf("%s\n", "Hello, world!");   // 调用的是 C 函数 printf
    return 0;
}
```

```
// ② 现代 C++ 风格
#include <iostream>
int main() {
    std::cout << "Hello, world!" << std::endl;   // 类型安全的流输出
    return 0;
}
```

代码做什么：两段程序输出完全相同，但实现机制迥异——`printf` 依赖格式字符串与可变参数（类型错误在编译期检查很弱），`std::cout <<` 则通过重载在编译期确定与类型匹配的输出函数。

特性机制解说：这是"C++ 是 C 的超集"的直接证据：`printf`、C 风格头文件、甚至内联汇编在 C++ 中都合法（幻灯片展示了一个用 `asm(...)` 写 Hello World 的极端例子）。这给了程序员**选择权**（设计哲学第 5

条），但也意味着 C++ 需要更强的自律——所以本课程强调类型安全与现代写法。

示例 3：现代 C++ 的"优雅 + 高效"小示例（C++11）

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> names = {"Valorant", "MySQL", "LLVM"};
    // range-based for：C++11 起的新特性，遍历容器
    for (const auto& name : names) {
        std::cout << name << " is built with C++!" << std::endl;
    }
    return 0;
}
```

代码做什么：创建一个字符串向量，用 range-based for 遍历并逐行输出。

特性机制解说： – `std::vector` 是标准库容器，自动管理动态数组内存； – `const auto& name` 表示"以**常量引用**方式读取每个元素"——不复制字符串（高效），同时保证不修改（安全）。这正是 Lecture 3 将要详细讲解的引用与 const 的提前亮相； – 这一行代码同时体现了五大设计哲学中的 Readable（可读）、Safety（const）、Efficiency（引用避免拷贝）。

示例 4：手动内存管理 vs RAII 智能指针（C++98 风格 vs 现代风格）

```
// ① C++98 风格：手动 new/delete，容易泄漏
#include <iostream>
#include <string>
int main() {
    std::string* s = new std::string("Hello");
    std::cout << *s << std::endl;
    delete s;                 // 忘记 delete 或提前 return 都会泄漏
    return 0;
}
```

```
// ② 现代风格（C++11/14）：unique_ptr 自动管理，无需手动 delete
#include <iostream>
#include <memory>
#include <string>
int main() {
    auto s = std::make_unique<std::string>("Hello");
    std::cout << *s << std::endl;   // 离开作用域时自动释放内存
    return 0;
}
```

代码做什么：两段代码都在堆上创建字符串并输出，区别只在内存如何释放。

特性机制解说： – ① 中一旦中间逻辑 `return` 或抛出异常，`delete` 永远执行不到，内存泄漏；程序员必须手工保证每条路径都释放——这正是 C++98 时代最普遍的 bug 来源； – ② 中 `std::unique_ptr` 在**析构时自动释放**所拥有的堆对象（RAII），所有权转移、异常安全都由类型系统保证，代码更短且不可能"忘记释放"； – 这一对比就是设计哲学第 4 条"Compartmentalize messy constructs（封装丑陋细节）"的活教材——把内存管理的脏活交给库，程序员专注业务逻辑。

示例 5：基础输入输出——流的预告（C++11）

```
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cout << "What's your name? ";   // 输出提示
    std::cin >> name;                    // 从标准输入读取一个词
    std::cout << "Welcome to CS106L, " << name << "!" << std::endl;
    return 0;
}
// 输入：Ada  →  输出：Welcome to CS106L, Ada!
```

代码做什么：先打印提示，再从键盘读取字符串，最后输出欢迎语。

特性机制解说： – `std::cin` 是标准输入流，`>>` 运算符从流中提取数据到变量——与 `std::cout <<` 方向相反； – `>>` 按空白分隔读取，只读一个词（想读整行要 `std::getline`，Lecture 4 详讲）； – 流（stream）是 C++ 标准库最重要的抽象之一，第 4 讲整讲展开；A1 读 CSV 文件就靠它。这一行 `std::cin >> name` 就是"输入 → 程序 → 输出"最小闭环，也是你验证工具链的最佳练习。

与旧标准（如C++98）的对比

- **C++98 时代没有 `auto` 类型推导**：`auto` 在 C++98 中是存储类说明符（如 `auto int x = 1;`），毫无用处；现代 `auto` 的推导语义是 C++11 才引入的。
- **C++98 没有智能指针**：必须手动 `new` / `delete`，一旦忘记 `delete` 或异常路径泄漏，就产生内存泄漏；`std::unique_ptr`（C++11）与 `std::make_unique`（C++14）让"裸指针所有权"问题大幅减少。
- **C++98 没有 range-based for、没有 `nullptr`、没有 lambda**：遍历容器要写 `for (size_t i = 0; i < v.size(); ++i)` 或迭代器循环，代码更长、更容易出错。
- **标准演进节奏**：C++98 后十年停滞（2003 只是小修补），直到 2011 年 C++11 带来"现代 C++"大爆炸；此后每三年一个版本（14/17/20/23/26）。本课程站在 C++26 的视角教学，但**核心概念（引用、const、struct）从 C++98 就有**，只是表达方式不断进化。
- **一个 C++98 时代的 Hello World 长这样**：

```
// C++98 典型写法：无 auto、无智能指针、手动管理
#include <iostream>
#include <string>
int main() {
    std::string* str = new std::string("Hello World!");
    std::cout << *str << std::endl;
    delete str;                         // 记得手动释放
    return 0;
}
```

对比本讲示例 1 的现代写法：类型名改为 `auto`、堆对象交给 `std::unique_ptr`、无需 `delete`。功能等价，但现代版更短、更安全、更不易错——这就是"标准每三年进步一次"带来的实际收益。

关键要点

- C++ 的五大设计哲学：**可读、安全、高效、抽象、程序员选择**——这是解释一切语言特性的总纲。
- C++ 是**编译型、静态类型**的语言，编译期会尽可能多地发现错误，且运行效率高（"不浪费时间和空间"）。
- C++ 向后兼容 C，同一任务存在汇编/C/现代 C++ 多层写法；**本课程只教现代 C++ 写法**。
- C++ 标准每三年一版（C++98 → … → C++26），本课程以现代标准（覆盖 C++26）为教学语言。
- 课程产出：掌握 STL、理解"好代码长什么样"，培养类型安全、内存高效、const 正确的编码习惯——这正是 CS107/CS111 等后续课程与工业界的需要。
- C++ 应用遍布"看不见的地基"：操作系统、浏览器、数据库、游戏、自动驾驶、ML 框架、编译器——学它不是为了应试，而是为了能读懂和写出这个世界的基础设施。

常见陷阱与注意事项

- **忘记 `std::` 前缀**：`cout`、`string` 都是 `std` 命名空间里的名字，不写 `std::` 且没有 `using namespace std;` 就无法编译；写 `using namespace std;` 虽省事，但会引入歧义（见 Lecture 2），是公认的坏风格。
- **忘记 `#include` 对应头文件**：用 `std::cout` 要 `#include <iostream>`，用 `std::string` 要 `#include <string>`，用 `std::unique_ptr` 要 `#include <memory>`；头文件缺失的报错信息往往令人困惑。
- **用 C 风格习惯写现代 C++**：如用 `printf` + 裸指针 + 手动 `new/delete`。不是不能编译，而是放弃了类型安全与 RAII 的保护。
- **混淆"合法"与"推荐"**：内联汇编、C 风格写法在 C++ 中合法，但几乎从不是好选择；课程的评判标准是"现代、优雅、正确"。
- **以为 `auto` 是动态类型**：`auto` 推导出的类型在编译期就固定，之后不能再改变（`auto i = 1; i = "hello";` 是编译错误）——细节在 Lecture 2 展开。

- **误用 `std::endl` 的时机**：`std::endl` 会刷新缓冲区（flush），在大量输出循环里用 `'\n'` 更快——"不浪费时间和空间"从这些细节开始。
- **忽略编译警告**：编译器是你的朋友。养成读警告、`g++ -Wall -Wextra` 编译的习惯，很多运行期 bug 在编译期就有提示。

关联作业提示

None（本讲无作业）。课程有 8 个每周小作业（每次 1–2 小时），第一个作业（A1: SimpleEnroll）将在第 2 周周五发布。建议： – 按 A1 说明完成编译器环境搭建（`g++ -std=c++20 main.cpp -o main`），提前用上面的 Hello World 验证工具链； – 记住课程评分规则（S/NC：Week 2–9 期间出勤 12/14 次讲座 + 完成全部 8 个作业），每讲开头的签到二维码只在前 10 分钟有效； – 把本讲"三层写法"的印象留着：A1 的全部代码都要求用现代 C++（struct、引用、流）完成，正是 Lecture 2、3 的内容； – 可以现在就动手：把示例 1 的现代 Hello World 编译运行一遍，再试试示例 3 的 vector 遍历——工具链顺了，A1 上手就会非常快。

Lecture 2 (Week 1 – Thursday): 类型与结构体 (Types & Structs)

概述

本讲正式进入 C++ 语言本身，核心是回答两个问题：**C++ 程序什么时候出错误**（编译期 vs 运行期）与 **如何组织数据**（struct 与 std::pair）。课堂先用 Python 与 C++ 的对比阐明"C++ 是编译型、静态类型语言，错误尽量在编译期暴露"，然后引入 struct 把多个字段捆绑成新类型，用 `std::pair` 与 `using` / `auto` 让代码更简洁。最后以"求解二次方程"的课堂代码 Demo 串起全部知识：`std::pair` 嵌套返回多值、`using` 类型别名、`auto` 类型推导。这些是后续所有讲座的地基。

核心特性与语法详解

- **编译期（Compile Time）vs 运行期（Run Time）**
- **定义与目的**：解释型语言（如 Python）逐行翻译、逐行执行，全程都在运行期；编译型语言（如 C++）先把**整个**源码翻译成机器码打包成可执行文件（编译期），再执行（运行期）。
- **核心语法**：`g++ main.cpp`（编译）→ `./a.out`（运行）。
- **设计意图与最佳实践**：错误发生在哪个阶段决定了代价。Python 的 `print("hello" * "world")` 会先打印 `Running...` 再抛 `TypeError`（运行期才发现）；同样的 `std::cout << hello * world` 在 C++ 中直接编译失败（`error: no match for 'operator*' (operand types are 'std::string' and 'std::string')`）。**编译期错误 = 免费的错误检查**。
- **静态类型（Static Typing）**

- **定义与目的**：C++ 中每个变量都必须声明类型，声明后类型不可变；编译器在生成机器码之前就检查类型。
- **核心语法**：`int a = 3;` / `std::string b = "test";`。
- **设计意图与最佳实践**：对比 Python（动态类型，`d = 106; d = "hello"` 合法），C++ 中 `int d = 106; d = "hello";` 是编译错误。静态类型带来三点好处：**更高效**（运行期无需类型标签与检查）、**更易理解与推理**、**错误检查更早更彻底**（如 `int add_3(int x)` 传入字符串在编译期就被拒绝）。
- **内置类型（Built-in Types）**
- **定义与目的**：类型是变量的"类别"。C++ 自带基础类型：`int`（106）、`double`（71.4）、`std::string`（"Welcome to CS106L!"）、`bool`（true/false）、`size_t`（12，非负，常用于索引与大小）。
- **核心语法**：`double b = 3.2 * 5 - 1;`（注意：`3.2` 是 double，所以整条表达式是 double）。
- **设计意图与最佳实践**：注意 `int c = 5 / 2;` 中**整数除法截断**，结果是 `2` 而不是 `2.5`——这是静态类型下最常见的"意外"之一。
- **函数重载（Function Overloading）**
- **定义与目的**：定义同名但**参数列表不同**的多个函数，编译器根据实参类型选择版本。
- **核心语法**：`double axolotl(int x)` 与 `double axolotl(double x)` 可共存。
- **设计意图与最佳实践**：`axolotl(2)` 选 int 版本返回 `5.0`（`(double)2 + 3`）；`axolotl(2.0)` 选 double 版本返回 `6.0`（`2.0 * 3`）。重载解析发生在**编译期**，是"静态类型"的直接红利。
- **struct（结构体）**
- **定义与目的**：把多个命名变量捆绑成一个新类型。解决"一个函数如何返回多个值"的根本问题——例如学生 ID 需要同时返回 name、sunet、idNumber。
- **核心语法**：`cpp #include <string> struct StanfordID { std::string name; // 字段（field）：有名字有类型 std::string sunet; int idNumber; }; int main() { StanfordID id; // 声明一个结构体变量 id.name = "THE Stanford Tree"; // 用 '.' 访问字段 return 0; }`
- **设计意图与最佳实践**：`issueNewID()` 的返回类型直接写 `StanfordID`，函数体内构造并 `return id;` 即可返回"三个值"。**THE BIG IDEA：struct 把命名变量捆绑成新类型**。
- **列表初始化 / 统一初始化（List Initialization，C++11 起）**
- **定义与目的**：用花括号 `{}` 一次性初始化全部字段，替代逐字段赋值。
- **核心语法**：`cpp #include <string> struct StanfordID { std::string name; std::string sunet; int idNumber; }; int main() { StanfordID tree = { "THE Stanford Tree", "theTREE", 0000002 }; // '=' 可省略 StanfordID lelandjr { "Leland Stanford Jr", "thejunior", 5430282 }; return 0; }`
- **设计意图与最佳实践**：值的顺序必须与**字段声明顺序**一致；`=` 可选。这是 Lecture 3 的重点（uniform initialization 与窄化转换），本讲先用起来。
- **std::pair（标准库的"通用双字段结构体"）**
- **定义与目的**：`std::pair<T1, T2>` 是标准库提供的模板结构体，只有两个字段 `first` 和 `second`，用于"任意两个值打包"。

- **核心语法**：`cpp #include <string> #include <utility> int main() { std::pair<std::string, int> dozen { "Eggs", 12 }; std::string item = dozen.first; // "Eggs" int quantity = dozen.second; // 12 return 0; }`
- **设计意图与最佳实践**：其实现本质就是一个模板：`cpp template <typename T1, typename T2> struct pair { T1 first; T2 second; };` 所以课堂上的 `struct Order { string item; int quantity; };`、`struct Name {...};`、`struct Point {...};` 都可以用 `std::pair` 表达——"Notice anything?" 它们都只是"两个字段"。模板细节（Lecture 8 深入），本讲只需会使用。
- **std —— C++ 标准库与命名空间**
- **定义与目的**：标准库提供内置类型、函数等；使用前必须 `#include` 对应头文件，并加 `std::` 前缀。
- **核心语法**：`#include <string>` → `std::string`；`#include <utility>` → `std::pair`；`#include <iostream>` → `std::cout, std::endl`。
- **设计意图与最佳实践**：`using namespace std;` 虽可省略前缀，但会引入歧义（若自己定义了 `sort`，与 `std::sort` 冲突），是坏风格。`#include` 的机制是**文本替换**：把头文件内容原样粘贴进源文件，之后才能使用其中定义的名字。查文档认准 **cppreference.com**（幻灯片明确提醒避开过时且广告多的 cplusplus.com）。
- **using 类型别名（Type Alias，C++11 起）**
- **定义与目的**：给长类型名起短名字。
- **核心语法**：`using Zeros = std::pair<double, double>;`
- **设计意图与最佳实践**：`using` 就像"类型的变量"。把 `std::pair<bool, std::pair<double, double>> solveQuadratic(...)` 拆成 `using Zeros = ...; using Solution = std::pair<bool, Zeros>;` 后，签名变成 `Solution solveQuadratic(double a, double b, double c);`，可读性大增。
- **auto 类型推导（C++11 起）**
- **定义与目的**：让编译器从初始化表达式推断变量类型。
- **核心语法**：`auto result = solveQuadratic(a, b, c);`
- **设计意图与最佳实践**：`result` 的类型仍是 `std::pair<bool, std::pair<double, double>>`（编译器查 `solveQuadratic` 的返回类型填入），**与手写完全等价**。`auto` **仍然是静态类型**：`auto i = 1; i = "hello!";` 编译失败。幻灯片对比了两种写法：长类型名用 `auto` 更清晰（`auto result = ...`），短类型名手写更清晰（`int i = 1;`）——按需选用。

代码示例与逐步解说（核心）

示例 1：struct 打包多字段并返回（C++11，改写自课堂 StanfordID 例）

```cpp
#include <iostream>
#include <string>

struct StanfordID {
    std::string name;
    std::string sunet;
    int idNumber;
};

StanfordID issueNewID() {
    // 列表初始化：顺序与字段声明一致
    StanfordID id { "THE Stanford Tree", "theTREE", 0000002 };
    return id;
}

int main() {
    StanfordID id = issueNewID();
    std::cout << id.name << " (" << id.sunet << ") #" << id.idNumber << std::endl;
    return 0;
}
// THE Stanford Tree (theTREE) #2
```

代码做什么： – `struct StanfordID` 声明三个字段；`issueNewID()` 用花括号列表初始化一个 ID 并返回； – `main` 中 `issueNewID()` 的返回值被拷贝初始化到 `id`，再用 `.` 访问三个字段并输出。

特性机制解说： – 花括号初始化按字段顺序填入（name → sunet → idNumber），编译器在编译期检查数量与类型是否匹配； – `return id;` 返回的是**整个 struct 的值拷贝**——struct 像内置类型一样可以整体赋值、传参、返回； – `0000002` 是十进制整数 2（前导零只是字面量写法），所以输出 `#2`。

示例 2：课堂代码——求解二次方程（C++11/17，cs106l_data/lecture_code/lecture02/main.cpp）

```cpp
#include <cmath>
#include <iostream>
#include <utility>

using Zeros = std::pair<double, double>;
using Solution = std::pair<bool, Zeros>;

// 求解 ax^2 + bx + c = 0
// 返回：first 表示是否有解；若有，second 为两个根
Solution solveQuadratic(double a, double b, double c) {
    double discrim = b * b - 4 * a * c;
    if (discrim < 0) return { false, { 106, 106 } };   // 无解：first=false

    double root = sqrt(discrim);
    return { true, { (-b - root) / (2 * a), (-b + root) / (2 * a) } };
}

int main() {
    double a, b, c;
    std::cout << "a: "; std::cin >> a;
    std::cout << "b: "; std::cin >> b;
    std::cout << "c: "; std::cin >> c;

    auto result = solveQuadratic(a, b, c);       // auto 推导出 Solution 类型
    if (result.first) {
        auto solutions = result.second;
        std::cout << "Solutions: " << solutions.first << ", "
                  << solutions.second << std::endl;
    } else {
        std::cout << "No solutions" << std::endl;
    }
    return 0;
}
```

代码做什么： – 用户输入系数 a、b、c；`solveQuadratic` 计算判别式 `b²−4ac`，无解时返回 `{false, {106, 106}}`（占位值），有解时用求根公式 `(−b ± √Δ)/(2a)` 返回两个根； – `main` 中用 `auto result` 接收，检查 `result.first` 决定打印根还是 "No solutions"。

特性机制解说： – **嵌套 pair**：`Solution` 是 `std::pair<bool, Zeros>`，`Zeros` 又是 `std::pair<double, double>`——两个返回值被"缝合"成一个返回值； – **花括号返回**：`return { true, { ... } };` 利用 C++11 的列表初始化按序构造 `pair`，比 `std::make_pair` 更简洁； – **`using` 别名**：把三层嵌套的类型名抽象成 `Zeros` / `Solution`，签名一目了然； – **`auto` 推导**：`result`、`solutions` 的类型由编译器静态推导，等价于手写完整类型名； – `sqrt` 来自 `<cmath>`（课堂代码还演示了不加 `std::` 前缀调用 C 风格数学函数——在 `<cmath>` 下应写 `std::sqrt` 更规范）。

示例 3：`auto` 仍是静态类型（C++11）

```
#include <string>

int main() {
    auto i = 1;          // i 被推导为 int
    // i = "hello!";     // ❌ 编译错误：不能把 const char* 赋给 int
    auto s = std::string("test");   // s 被推导为 std::string
    return 0;
}
```

代码做什么：演示 `auto` 的推导结果在编译期固定。

特性机制解说：`auto` 不引入任何动态行为——它只是"让编译器替你写字面类型"。把被注释的行取消注释即可看到编译错误，这印证了"`auto` 是静态类型"这一幻灯片结论。

示例 4：函数重载（C++11，来自课堂 axolotl 例）

```
#include <iostream>

// (1) int 版本：返回 double
double axolotl(int x) {
    return static_cast<double>(x) + 3;   // 类型转换：int → double
}

// (2) double 版本
double axolotl(double x) {
    return x * 3;
}

int main() {
    std::cout << axolotl(2) << std::endl;    // 实参是 int    → 调用 (1)，输出 5
    std::cout << axolotl(2.0) << std::endl;  // 实参是 double → 调用 (2)，输出 6
    return 0;
}
// 5
// 6
```

代码做什么：同名函数 `axolotl` 有两个版本（int / double 参数），两次调用分别命中不同版本。

特性机制解说： – 函数重载 = **同名函数、不同参数列表**（数量或类型不同即可共存）； – 重载解析（overload resolution）在**编译期**依据实参的静态类型完成：`2` 是 int → 精确匹配 (1)，`2.0` 是 double → 精确匹配 (2)； – 幻灯片用这个例子演示了 `(int)x` 式类型转换（截断小数）与 `static_cast` 的作用：`(1)` 中 `(double)2 + 3 == 5.0`；若实参 `2.5` 传入 int 版本则会先被截断为 `2`——类型不匹配时编译器按"隐式转换代价"排序选择，这正是 Lecture 3 窄化转换的伏笔。

示例 5：`std::pair` 与结构化绑定（C++17）

```
#include <iostream>
#include <string>
#include <utility>

std::pair<std::string, int> makeOrder() {
    return { "Eggs", 12 };          // 列表初始化构造 pair
}

int main() {
    auto order = makeOrder();       // order: std::pair<std::string, int>
    std::cout << order.first << " x" << order.second << std::endl;   // 传统访问

    // C++17 结构化绑定：一次性拆开 first / second
    auto [item, quantity] = makeOrder();
    std::cout << item << " x" << quantity << std::endl;
    return 0;
}
// Eggs x12
// Eggs x12
```

代码做什么：用 `std::pair` 打包"商品 + 数量"，再用两种方式取出字段。

特性机制解说： – `return { "Eggs", 12 };` 利用列表初始化按序构造 `pair<std::string, int>`，`"Eggs"` 被隐式转换为 `std::string`； – `auto [item, quantity] = ...` 是 **C++17 结构化绑定（structured bindings）**：把 `pair` 的两个成员**按声明顺序**解包到两个新变量，省去 `.first` / `.second` 的样板代码； – 结构化绑定同样适用于 struct、数组、tuple——本讲先用 pair 建立直觉，后续讲座会反复用到；它再次体现了"用 `auto` 交给编译器、保持代码简洁"的现代风格。

与旧标准（如C++98）的对比

- **`using` vs `typedef`**：C++98 用 `typedef std::pair<double, double> Zeros;`（类型名在中间，嵌套别名晦涩难读）；C++11 的 `using Zeros = ...;` 更直观、支持模板别名，是现代 C++ 的推荐写法。

```
#include <utility>

int main() {
    // C++98 风格：typedef（类型名夹在中间）
    typedef std::pair<double, double> Zeros98;
    typedef std::pair<bool, Zeros98>  Solution98;

    // 现代 C++ 风格：using 别名（C++11 起，= 号让阅读顺序更自然）
    using Zeros    = std::pair<double, double>;
    using Solution = std::pair<bool, Zeros>;

    Solution s { true, { 1.0, 2.0 } };   // 两种写法等价，现代风格更清晰
    return 0;
}
```

- **`auto` 语义完全不同**：C++98 中 `auto` 是存储类说明符（`auto int x;`，表示自动存储期，毫无用处）；C++11 起才表示类型推导。
- **初始化方式**：C++98 只有 `StanfordID id; id.name = ...;` 逐字段赋值或对**聚合类型**使用 `T x = { ... };`；C++11 的列表初始化适用于一切类型、写法统一（`T x { ... };`、`return { ... };`），并附赠窄化检查（详见 Lecture 3）。
- **`std::pair` 本身 C++98 就有**（`<utility>` 中的 `std::make_pair`），但 C++98 没有 `{}` 初始化，必须写 `std::make_pair("Eggs", 12)` 且类型推导能力弱（`"Eggs"` 会被推导为 `const char*` 而非 `std::string`）。
- **结构化绑定是 C++17 才有的**：C++98 解包 pair 只能写 `pair.first` / `pair.second` 或借助 `std::tie`（C++11）——都没有 `auto [a, b]` 直观。
- **range-based for / 现代容器用法**：C++98 遍历容器只能手写索引或迭代器循环，代码冗长（详见后续讲座）。

关键要点

- C++ 是**编译型、静态类型**语言：错误尽量在**编译期**暴露，类型声明后不可变，运行高效。
- **struct 把命名变量捆绑成新类型**，用 `.` 访问字段；列表初始化 `{ ... }` 按字段顺序一次性填值，`=` 可选。
- `std::pair<T1, T2>` 是"通用双字段 struct"（`first` / `second`），可嵌套使用以返回多个值。
- 使用标准库 = `#include` 对应头文件 + `std::` 前缀；避免 `using namespace std;`。
- **`using` 造类型别名、`auto` 推导类型**（C++11 起）是提升可读性的现代工具，且 `auto` 仍是静态类型。
- 函数重载（同名不同参）在编译期按实参类型选择版本——静态类型的又一红利。
- 记住内置类型常识：`size_t` 是非负整数（常用于索引/大小）；`5 / 2` 整数除法结果为 `2`。

常见陷阱与注意事项

- **整数除法截断**：`int c = 5 / 2;` 得到 `2` 而不是 `2.5`；想得到小数至少一个操作数要写成 `double`。
- **忘记 `#include <utility>`**：用 `std::pair` 不包含头文件会编译失败；同理 `std::string` 需要 `<string>`。
- **滥用 `using namespace std;`**：与自己的 `sort` 等函数重名时产生歧义；规范写法是显式 `std::`。
- **列表初始化顺序错乱**：花括号值必须与**字段声明顺序**一致，否则数据张冠李戴（编译器按序填入，不会替你"按名匹配"）。
- **误以为 `auto` 是动态类型**：`auto i = 1; i = "hello";` 编译错误；`auto` 只是编译期推导，不改变静态类型语义。
- **把 struct 当作"只有数据"而忽略初始化**：默认声明 `StanfordID id;` 时，内置类型字段（如 `idNumber`）的值是**未初始化**的，读取是未定义行为——养成用 `{ }` 初始化的习惯（Lecture 3 重点）。

- **重载时类型不匹配导致的"意外"调用**：如 `axolotl(2.5)` 若只有 int 版本可用，会先把 `2.5` 截断成 `2` 再调用——留意隐式转换的方向，必要时显式 `static_cast`。

关联作业提示

None（本讲无直接对应作业）。不过 A1: SimpleEnroll 会大量用到本讲知识，可提前预热： – A1 的 `Course` struct（Title、Number of Units、Quarter 三个字段）正是"struct 打包数据"的实战——用列表初始化构造每个 Course 记录； – `std::pair` 与 `using` 别名在 A1 的工具代码（如 `split` 返回 `std::vector<std::string>`）中会反复出现； – 建议现在就练熟：声明一个 struct → 用 `{}` 初始化 → 用 `.` 访问字段 → 用 `auto` 接收函数返回值，A1 拿到手就能直接上手。

Lecture 3 (Week 2 – Tuesday): 初始化与引用 (Initialization & References)

概述

本讲是"写出正确、高效 C++"的关键一课，两个主题：**初始化**与**引用**。初始化部分讲清 C++ 的四种基本初始化形式（默认、拷贝、直接、列表）以及**列表初始化对窄化转换（narrowing conversion）的编译期拦截**；引用部分讲清引用是"变量的别名"、绑定同一块内存、必须绑定左值且不可重绑定，并由此引出 **const 引用**与**值传递 vs 引用传递**的选择。课堂代码（initialization.cpp、references.cpp、const.cpp、Reactor.cpp）全部围绕"初始化写错会怎样""参数怎么传才高效且安全"展开——这正是 A1: SimpleEnroll 的核心考点。

核心特性与语法详解

- **默认初始化（Default Initialization）**
- **定义与目的**：`T x;` 不提供任何初值。对类类型（如 `std::string`）调用默认构造函数；对**内置类型（int、double...）不初始化**——值是"不确定的"（indeterminate），读取它是未定义行为。
- **核心语法**：`int x;` / `std::string s;`
- **设计意图与最佳实践**：内置类型的默认初始化不写内存（追求零开销），代价是需要程序员自律。**黄金法则：内置类型永远用 `{}` 或显式初值初始化。**
- **拷贝初始化（Copy Initialization）**
- **定义与目的**：`T x = value;`，从已有值"拷贝"出新的对象。
- **核心语法**：`int x = 5;` / `std::string s = "hi";`
- **设计意图与最佳实践**：C 语言风格的写法，直观但语义上走的是拷贝路径；对于类类型可能调用移动/拷贝构造函数（现代编译器通常会优化掉多余的拷贝，即 copy elision）。
- **直接初始化（Direct Initialization）**

- **定义与目的**：`T x(args);`，用圆括号参数直接构造对象，**允许窄化转换**。
- **核心语法**：`int x(5);` / `std::vector<int> v(10);`（构造 10 个元素）
- **设计意图与最佳实践**：圆括号会触发"最令人头痛的解析"（most vexing parse）等历史包袱，且**不做窄化检查**——Reactor.cpp 的 bug 正源于此（见示例 3）。
- **列表 / 统一初始化（List / Uniform Initialization，C++11 起）**
- **定义与目的**：`T x{ ... };` 用花括号初始化，对所有类型语法统一，且**在编译期拒绝窄化转换**。
- **核心语法**：`int x{5};` / `StanfordID id{ "Tree", "theTREE", 2 };` / `T x{};`（值初始化，内置类型清零）
- **设计意图与最佳实践**：C++11 引入它正是为了"统一 + 安全"。**现代 C++ 首选 `{}`**；`T x{}` 会把内置类型零初始化，杜绝"未初始化变量"类 bug。
- **窄化转换（Narrowing Conversion）**
- **定义与目的**：隐式转换中**丢失信息**的一类：如 double→int（截断小数）、long→int（可能溢出）、double→float（精度可能丢失）。
- **核心语法**：`int numOne{12.0};` → ❌ 编译错误 `narrowing conversion of '1.2e+1' from 'double' to 'int'`。
- **设计意图与最佳实践**：列表初始化把窄化从"静默的错误"变成"编译错误"；**若确实需要截断，用显式类型转换**（如 `static_cast<int>(12.8)`），让意图可见。注意例外：`float f{12.0};` 是**合法**的——因为 12.0 能被 float 精确表示，不构成窄化（标准允许"常量表达式且目标类型可精确表示"的情况）。
- **引用（References，C++98 已有）**
- **定义与目的**：`T& ref = obj;` 声明 `ref` 是 `obj` 的**别名**——两者绑定同一块内存，对 `ref` 的读写就是对 `obj` 的读写。
- **核心语法**：`cpp int main() { int x = 5; int& r = x; // r 是 x 的别名（与 x 绑定同一块内存） r = 10; // 通过 r 改写 x：x 也变成 10 return 0; }`
- **设计意图与最佳实践**：三条铁律：① 引用**声明时必须绑定**一个对象（不能"先声明后绑定"）；② **不可重新绑定**（`r = y;` 是把 y 的值赋给 x，不是让 r 改指 y）；③ **非 const 左值引用只能绑定左值**（可寻址的具名对象），不能绑定字面量/临时值（如 `int& r = 5;` ❌）。
- **const 与 const 引用（const 正确性）**
- **定义与目的**：`const T` 表示"只读对象"；`const T&` 表示"只读视图"——既享受引用"不拷贝、直接看原对象"的高效，又保证**不修改**。
- **核心语法**：`const int x = 5;` / `const std::vector<int>& cr = vec;` / `void print(const std::string& s);`
- **设计意图与最佳实践**：const 引用可以绑定**任何东西**：左值、右值、字面量（`const int& r = 42;` 合法，临时对象生命周期被延长）。对 const 对象只能调用 const 成员函数（如 `vec.size()`），调用非 const 成员（如 `push_back`）是编译错误——const 是**编译器替你执行的纪律**。
- **值传递 vs 引用传递**

- **定义与目的**：函数形参决定"数据怎么进来"：按值（拷贝一份，函数内修改不影响调用方）vs 按引用（直接操作原对象，可修改调用方）。
- **核心语法**：`cpp void byValue(int n); // 拷贝 void byRef(int& n); // 可修改原值 void byConstRef(const int& n); // 只读，不拷贝`
- **设计意图与最佳实践**：选型口诀——**需要修改实参 → 传 `T&`；只读且类型较大（string、vector、struct）→ 传 `const T&`（避免昂贵拷贝）；内置小类型（int、double）只读 → 直接按值传**。这就是"不浪费时间和空间"哲学在参数传递上的体现。

代码示例与逐步解说（核心）

示例 1：四种初始化形式总览（C++11）

```
#include <iostream>
#include <string>

struct Point {
    double x;
    double y;
};

int main() {
    // 默认初始化：内置类型未初始化（值不确定，别读它！）
    int a;                          // 不确定值
    std::string s;                  // 空字符串 ""（类类型有默认构造）

    // 拷贝初始化
    int b = 42;
    std::string t = "hi";

    // 直接初始化（圆括号，允许窄化）
    int c(3.9);                     // c == 3（小数被截断，不报错！）

    // 列表初始化（统一初始化，拒绝窄化）
    int d{7};                       // OK
    Point p{1.0, 2.0};              // 按字段顺序初始化
    // int e{3.9};                  // ❌ 取消注释即编译错误：narrowing conversion

    // 值初始化：内置类型清零
    int f{};                        // f == 0
    std::cout << b << " " << c << " " << d << " " << f << std::endl;
    return 0;
}
```

代码做什么：一行行声明不同初始化方式的变量；`c` 用圆括号从 `3.9` 截断得到 `3`（静默），`e` 用花括号直接编译失败。

特性机制解说： – 编译器对 `{}` 逐项做**窄化检查**： `double → int` 属于"浮点转整型"，必然可能丢精度，属于窄化，直接报错；圆括号初始化走旧的隐式转换规则， `3.9` 静默截断为 `3` ——同一个"错误"，两种初始化写法给出截然不同的命运； – `Point p{1.0, 2.0}` 走聚合初始化：按成员声明顺序填入； – `int f{}` 是**值初始化**，等价于 `int f = 0;` ，规避了默认初始化"不确定值"的坑。

示例 2：课堂代码 initialization.cpp（C++11，验证窄化保护）

```
#include <iostream>

int main() {
    // 列表初始化：窄化转换被编译器拦截
    // int numOne{12.0}; // ❌ 取消注释即编译错误：double → int 是窄化
    float numTwo{12.0};  // ✅ 合法：12.0 可被 float 精确表示，不构成窄化

    std::cout << "numTwo is: " << numTwo << std::endl;
    return 0;
}
```

代码做什么：这是课堂真实代码（ `cs106l_data/lecture_code/lecture03/initialization.cpp` ）的改编版：原代码中 `int numOne{12.0};` 编译失败，报错 `error: narrowing conversion of '1.2e+1' from 'double' to 'int'` （这里已注释掉并保留 `float numTwo{12.0};` 展示合法情形）。

特性机制解说： – 窄化判定的标准规则：**浮点 → 整型**永远是窄化（无例外），所以 `int{12.0}` 必错； – **double → float** 是否窄化取决于"源是常量表达式且值在目标类型可表示范围（哪怕不精确）"—— `12.0` 恰好能被 float 精确表示，故合法；若写成 `float f{12.1};` （12.1 无法精确表示）就会报错。这正是"编译器用规则替你把关"的体现，也是本讲反复强调"用 `{}` "的原因。

示例 3：课堂代码 Reactor.cpp——窄化转换的真实代价（C++11）

```cpp
#include <iostream>

class Reactor {
public:
    Reactor(double temperature) : temperature(temperature) {}   // 构造函数，后续讲座详讲
    void checkCool() {
        if (temperature > 100.0) {
            std::cout << "Emergency cooling!" << std::endl;
        } else {
            std::cout << "Temperature is normal. No emergency cooling required" << std::end
        }
    }
private:
    double temperature;
};

int main() {
    // 直接初始化（圆括号）：允许窄化，100.8 被静默截断为 100
    int criticalTemperature(100.8);
    Reactor reactor(criticalTemperature);   // int 100 → double 100.0
    reactor.checkCool();                     // 输出 "Temperature is normal."!
    return 0;
}
```

代码做什么（课堂真实代码，`cs106l_data/lecture_code/lecture03/Reactor.cpp`）：想模拟"温度超过 100°C 触发紧急冷却"，实际运行时输出却是 **"Temperature is normal. No emergency cooling required"**。

特性机制解说： – `int criticalTemperature(100.8);` 是**直接初始化**，圆括号允许窄化：`100.8` 静默截断为 `100`； – 随后 `100` 被隐式转回 `double`（`100.0`），`100.0 > 100.0` 为假 → 冷却逻辑**从未触发**； – 这是一个"看似无害实则致命"的静默 bug——工业 PLC / 反应堆控制系统正是这类代码的高发区。若把第 32 行改成 `int criticalTemperature{100.8};`，编译器会当场报错，从根源上避免灾难。**这就是列表初始化存在的意义**：把"静默的错误"变成"编译期的错误"。

示例 4：课堂代码 references.cpp——值传递 vs 引用传递（C++11）

```
#include <iostream>

// 引用版本：直接修改调用方的变量
void squareN(int& n) {
    n = n * n;   // 直接改写实参（课堂原版用 std::pow(n, 2)，效果相同）
}

int main() {
    int num = 5;
    std::cout << "(1) num is: " << num << std::endl;   // 5
    squareN(num);                                       // 按引用传：修改的就是 num 本身
    std::cout << "(2) num is: " << num << std::endl;   // 25
    return 0;
}
```

代码做什么（改编自课堂代码 `references.cpp`）：`squareN(int& n)` 把参数按**引用**传入，函数内 `n = n * n;` 直接改写了 `main` 中的 `num`，输出 5 → 25。

特性机制解说： – 原课堂代码用 `#if WITH_REF` 宏在"`int& n`（引用）"与"`int n`（值）"两个版本间切换：值版本下 `main` 里的 `num` 保持不变（拷贝被平方），引用版本下 `num` 被真正改写； – 原代码调用 `squareN(5);`（字面量）在引用版本下**无法编译**——报错 `cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'`：非 const 左值引用只能绑定左值（具名、可寻址的对象），`5` 是右值。这是引用最经典的规则，也是新手最常见的报错之一； – 通过引用修改实参的能力，正是 A1 中 `parse_csv` 需要"把填好的 vector 交还给调用方"的机制。

示例 5：课堂代码 const.cpp——const 对象与 const 引用（C++11）

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec{ 1, 2, 3 };                  // 普通 vector
    const std::vector<int> const_vec{ 1, 2, 3 };      // const 对象
    std::vector<int>& ref_vec{ vec };                 // 普通引用：vec 的别名
    const std::vector<int>& const_ref{ vec };         // const 引用：只读视图

    vec.push_back(3);           // ✅ OK：vec 可变
    // const_vec.push_back(3); // ❌ 编译错误：const 对象不能调用非 const 成员函数
    ref_vec.push_back(3);       // ✅ OK：ref_vec 是 vec 的别名，vec 实际变成 {1,2,3,3}
    // const_ref.push_back(3); // ❌ 编译错误：const 引用禁止修改
    return 0;
}
```

代码做什么（课堂真实代码 `const.cpp`）：声明四种变量后尝试 `push_back`，其中两行被编译器拒绝。

特性机制解说： – `const_vec` 是 const 对象：编译器禁止任何修改操作，`push_back`（非 const 成员函数）报错 `passing 'const std::vector<int>' as 'this' argument discards qualifiers`； – `ref_vec` 是 `vec` 的**别名**：对它 `push_back` 等于对 `vec` 操作（同一块内存、同一份数据）； –

`const_ref` 是 **const 引用**：它指向 `vec`（不拷贝），但只读——既高效（无拷贝）又安全（不可改）； – **本质区别**：`const_vec` 拷贝了一份 const 数据；`const_ref` 没有拷贝，只是"隔着一层只读玻璃看 `vec`"。当 `vec` 后续变化时，`const_ref` 看到的是变化后的新值——引用是别名，不是快照。

示例 6：高效传参——`const std::string&`（C++11）

```
#include <iostream>
#include <string>

// 只读大对象：const 引用，不拷贝、不修改
void greet(const std::string& name) {
    std::cout << "Hello, " << name << "!" << std::endl;
}

int main() {
    std::string me = "CS106L";
    greet(me);             // 绑定左值：OK
    greet("Stanford");     // 绑定字面量（右值）：OK，const 引用可以绑定右值
    return 0;
}
```

代码做什么：`greet` 以 `const std::string&` 接收参数，两次调用都成功。

特性机制解说： – 若形参是 `std::string name`（按值），每次调用都要**深拷贝**整个字符串（分配堆内存 + 复制字符）；`const std::string&` 零拷贝、零修改； – `const` 引用是"万能绑定"：左值、右值、字面量通吃（`const int& r = 42;` 合法，临时对象生命周期被延长到引用消亡）； – 对比：非 const 引用 `std::string&` 只能绑定左值，且意味着"函数可能修改实参"——所以**只读参数一律写 `const T&`**，把意图交给编译器监督。

与旧标准（如C++98）的对比

- **C++98 没有列表初始化**：只有 `T x;`、`T x = v;`、`T x(v);` 三种，窄化转换全部静默发生——Reactor.cpp 那种 `int x(100.8)` 截断 bug 在 C++98 里是**合法且无声**的，只能靠程序员肉眼发现。
- **C++11 的 `{}` 统一了初始化语法**：聚合、类、内置类型一视同仁，并内建窄化检查；C++98 的聚合初始化 `T x = { ... };` 仅适用于无构造函数、无私有成员的"聚合"，且不检查窄化。
- **引用本身 C++98 就有**（引用是 C++ 早期特性），但 C++98 时代惯例是"裸指针传参 + const 指针"，引用被广泛接受为现代风格是 C++11 之后的事；`const T&` 传参的"零拷贝 + 只读"理念在 C++98 同样成立，只是表达与工具（如 range-for）远不如现代版顺手。
- **值初始化 `T{}` 与 `nullptr`、range-for 等**：均为 C++11 起的新特性，C++98 中"清零内置变量"只能手写 `int x = 0;`。

关键要点

- **现代 C++ 优先用列表初始化 `{}`**：语法统一，且编译期拦截窄化转换；内置类型用 `T x{}` 保证清零，杜绝未初始化读取。
- **窄化转换（double→int、long→int 等）会静默丢信息**：圆括号/赋值允许它，花括号拒绝它；确需截断时用 `static_cast<int>(...)` 显式表达。
- **引用是变量的别名，绑定同一块内存**：声明时必须绑定、不可重绑定、非 const 引用只能绑定左值；通过引用修改，实参同步变化。
- **`const T&` 是"只读高效视图"**：零拷贝、禁止修改，可绑定左值/右值/字面量——只读参数的标准答案。
- **参数选型口诀**：要修改实参 → `T&`；只读大对象 → `const T&`；只读小内置类型 → 按值。

常见陷阱与注意事项

- **读取未初始化的内置变量**：`int x; std::cout << x;` 是未定义行为（值不确定）；用 `int x{};` 或给初值。
- **窄化静默截断**：`int x = 3.9;` / `int x(3.9);` 编译通过但 `x == 3`；想保留小数就别用 int，想截断就用显式转换。
- **非 const 引用绑定右值**：`int& r = 5;` 或 `void f(int&); f(5);` 编译错误；要么传具名左值，要么改成 `const int&`。
- **悬垂引用（dangling reference）**：引用指向的对象先消亡（如函数返回局部变量的引用）后继续使用——未定义行为。`int& bad() { int x = 1; return x; }` ❌。
- **误以为引用可重绑定**：`int& r = a; r = b;` 是把 b 的值**赋给 a**，不是让 r 改指 b；想"换个对象"应使用指针。
- **忘记 `&` 导致悄悄拷贝**：形参写成 `T name` 而非 `const T&` 时，大对象（string/vector/struct）每次调用都深拷贝，性能退化且行为上"改了也白改"。
- **对 const 对象调用非 const 成员函数**：`const_vec.push_back(...)` 编译错误；只读访问用 `size()`、`operator[]` 读取等 const 成员。

关联作业提示

A1: SimpleEnroll（第 2 周周五发布、一周后截止）——本讲的初始化与引用正是 A1 的核心考点： – **Part 0：补全 `Course` struct**：用列表初始化构造每个 Course 记录（`Course c{ title, units, quarter };`），字段顺序与 struct 声明一致；字符串字段用 `{}` 初始化最稳妥。 – **Part 1：`parse_csv`**：函数需要"把解析出的 Course 填进调用方的 vector"——因此形参必须是**引用**：`void parse_csv(std::vector<Course>& courses, ...)`。若漏写 `&`，函数内 push_back 的只是副本，调用方看到的 vector 仍为空——这是 A1 最经典的失分点。 – **Part 2/3：`write_courses_offered` /**

`write_courses_not_offered`：只读遍历（统计、比较 Quarter 是否为 "null"）用 `const` **引用**传参与遍历（如 `const std::string&`），避免字符串深拷贝；需要从 `all_courses` 删除已开课程时，删除操作必须发生在遍历之后（边遍历边删会失效，见 A1 提示），删除本身依赖"vector 按引用传入"才能影响调用方。 – **通用技巧**：先本地编译 `g++ -std=c++20 main.cpp -o main` 验证，再跑内置 autograder；遇到"函数改了但外面没变"的诡异现象，第一反应检查形参是不是漏了 `&`。

Lecture 4 (Week 2 – Thursday): 流 (Streams)

概述

本讲介绍 C++ 的**流（stream）**体系：一个统一"读数据 / 写数据"的通用 I/O 抽象。你将理解流的类层次（`ios_base` → `basic_ios` → `istream` / `ostream` 及其派生类型），掌握 `std::cout` / `std::cin`、`stringstream`、`ofstream` / `ifstream` 的用法，弄清输出缓冲与 flush 机制，并避开 `getline` 与 `>>` 混用、`cin` 读取失败等经典陷阱。流是 C++ 一切输入输出的地基：A1（SimpleEnroll）的 CSV 解析与文件写出几乎完全建立在流之上，后续课程的几乎所有程序也离不开它。

核心特性与语法详解

1. 什么是流（Stream）

- **定义与目的**：流是 C++ 的通用输入/输出设施——"a general input/output facility for C++"。它把"数据从外部来源（键盘、文件、字符串）进入程序"和"数据从程序流向外部目的地（终端、文件）"抽象成一条**传送带（conveyor belt）**：输出流把数据送出去，输入流把数据接进来。
- **核心语法**：输出 `std::cout << x << '\n';`；输入 `std::cin >> x;`。
- **设计意图与最佳实践**：Bjarne Stroustrup 说过："为程序设计通用的 I/O 设施是出了名的困难。"流的答案是**抽象**——隐藏不必要的细节（电机、线路、缓冲、硬件），只暴露相关的接口：`<<`（插入运算符 insertion operator）与 `>>`（提取运算符 extraction operator）。就像开车不用关心引擎原理、只需踩油门和转方向盘，用流时你也不必关心数据具体怎么被搬进搬出。

2. 流的类层次（ios_base → basic_ios → istream/ostream）

- **定义与目的**：所有流类型共享同一个继承体系，因此**接口一致、类型可互换**。
- **核心语法**（层次结构）：

```
ios_base └─ basic_ios ├─ istream ──→ std::cin, std::ifstream, std::istringstream └─ ostream ──→ std::cout, std::cerr, std::clog, std::ofstream, std::ostringstream
```

- **机制与职责**：
- `ios_base`：一切流的地基，维护两类信息。

    - **状态信息（State Information）**：流的"健康指标"，例如 `failbit`（逻辑错误，如类型不匹配）、`eofbit`（到达流末尾）。
    - **控制信息（Control Information）**：数据如何呈现，例如 255 应打印成 `"255"`、`"FF"`（十六进制）还是 `"377"`（八进制）。
- `basic_ios`：在 `ios_base` 之上保证流工作正常，并管理数据的**来源**（控制台、键盘、文件……）。
- `istream`：输入流基类，主要操作是 `>>`；`ostream`：输出流基类，主要操作是 `<<`。两者交集即 `iostream`，同时具备读写能力。
- **头文件**：`#include <iostream>`（cin 与 cout）、`<istream>`（cin）、`<ostream>`（cout）、`<fstream>`（文件流）、`<sstream>`（字符串流）。
- **设计意图**：同一套 `<<` / `>>` 操作符可以作用于任何流——这正是抽象带来的接口一致性（幻灯片强调："notice the `<<` and `>>`? That's abstraction at work!"）。

3. 提取/插入运算符的"空白分词"规则

- **定义与目的**：`<<` 和 `>>` 以**空白（whitespace）**为界切分数据，这是流最核心的行为规则。
- **核心语法**：`ss >> first >> last >> language;`
- **机制**：`>>` 一直读取到**下一个空白字符**为止。空白包括：空格 `' '`、`'\n'`、`'\t'`、`'\r'`、`'\f'`、`'\v'`。注意：`>>` 会跳过前导空白，但**不会消费末尾的分隔符**——这会引发后续 `getline` 的经典 bug（见陷阱部分）。

4. std::getline

- **定义与目的**：`>>` 只能读到空白为止，无法读取"一整行"（例如带空格的名字或引语）。`getline` 解决"按行读取"的需求。
- **核心语法**：`std::getline(std::istream& is, std::string& str, char delim = '\n');`
- **机制**：从 `is` 一直读取到分隔符 `delim`（默认 `'\n'`），把内容存入 `str`，并且**消费（吞掉）分隔符本身**。幻灯片特意强调："`getline()` consumes the delim character! PAY ATTENTION TO THIS :)"——这是理解一切换行残留问题的钥匙。

5. stringstream（字符串流）

- **定义与目的**：把**字符串当作流**来使用，尤其擅长"混合数据类型的解析与格式化"（mixing data types）。
- **核心语法**：构造 `std::stringstream ss(initial_quote);`；或先 `ss << ...` 填充再 `ss >> ...` 读出。
- **设计意图**：`istringstream` / `ostringstream` / `stringstream` 位于类层次底部，与 `cin` / `cout` 共享同一接口，因此同一套 `<<` / `>>` 代码既能处理终端也能处理字符串。CS106B 中按行读取文件、A1 提供的 `split` 工具函数，背后都是 `stringstream`。

6. 输出流的缓冲与刷新（flush）

- **定义与目的**：输出字符先进入**中间缓冲区（buffer）**，在显式 flush 时才真正写到目的地，从而减少慢速 I/O 的次数、提升性能。
- **核心语法**：`std::cout << std::flush;`（只刷新）与 `std::cout << std::endl;`（换行 + 刷新）。
- **何时刷新**：
- `std::cout << std::flush;`
- `std::cout << std::endl;`
- 程序正常结束时
- 缓冲区满时
- **tied 流交互时**：例如 `cout` 与 `cin` 绑定，`cin` 取输入前 `cout` 会先 flush，保证提示语先显示。
- **`std::endl` vs `'\n'`**：`std::endl` = 换行 + **flush**；`'\n'` 只换行。循环里用 `std::endl` 会导致每次迭代都 flush，严重拖慢程序——课程明确建议 "Use '\n'!"。
- **cerr vs clog**：`cerr` 用于输出错误，**无缓冲**（立即送出）；`clog` 用于非关键事件日志，**有缓冲**。

7. 文件流：std::ofstream / std::ifstream

- **定义与目的**：把数据写到文件 / 从文件读取数据。
- **核心语法**：
- 构造：`std::ofstream ofs("hello.txt");`（构造时即关联文件）
- 打开标志：`std::ios::trunc`（**默认**，截断重建）、`std::ios::app`（追加）、`std::ios::ate`（打开后立即跳到文件末尾）
- 常用成员：`is_open()`、`open()`、`close()`、`fail()`
- **机制与陷阱**：关闭（`close()`）后再 `<<` 写入会**静默失败**（不报错、不写入）；调用 `open()` 可重新关联文件（可换打开模式）。输入/输出文件流在同一来源/目的地上**互补**：`ofstream` 写、`ifstream` 读。

8. std::cin 的行为与失败

- **定义与目的**：标准输入流，**有缓冲**：用户输入先整体进入缓冲区，程序再按空白切词从缓冲区读取。
- **核心语法**：`std::cin >> pi;`
- **机制**：若缓冲区为空，则阻塞等待用户输入；若非空，则读到下一空白为止。当 `cin >> double` 时遇到非数字的单词（如 `"Fernandez"`），提取失败：流置 `failbit`，目标变量保持原值或变为 0，且**之后所有 `>>` 读取都会直接失败**——直到你 `clear()` 恢复状态（本讲先认识现象，A1 不会深究恢复）。

代码示例与逐步解说（核心）

示例 1：stringstream 混合类型解析（课堂代码 recap.cpp 改写）

```
// C++11（课程以 C++20 编译，本示例仅用 C++11 特性）
#include <iostream>
#include <sstream>
#include <string>

int main() {
    // 1. 构造一个装着整段文本的字符串流（等价于 ss << initial_quote）
    std::string initial_quote =
        "Bjarne Stroustrup C makes it easy to shoot yourself in the foot\n";
    std::stringstream ss(initial_quote);

    // 2. 按空白分词，依次取走 "Bjarne" / "Stroustrup" / "C"
    std::string first, last, language;
    ss >> first >> last >> language;

    // 3. 剩余部分用 getline 整行取走（含前导空格）
    std::string extracted_quote;
    std::getline(ss, extracted_quote);

    std::cout << first << " " << last << " said this: '" << language
              << " " << extracted_quote << "'\n";
    return 0;
}
```

- **代码做什么**：`ss >> first >> last >> language` 用空白把引语前三个词切出来；`std::getline(ss, extracted_quote)` 把剩下的 `" makes it easy to shoot yourself in the foot"`（前面带一个空格，因为 `>>` 不消费分隔符）整行装入 `extracted_quote`；最后打印。
- **特性机制解说**：`>>` 是"按空白分词器"——它跳过前导空白、提取到下一个空白为止、**把分隔符留在流里**。而 `getline` 一直读到 `'\n'` 并把 `'\n'` **吃掉**。两者一前一后配合，才能既按词取、又按行取。

示例 2：cout / cin 基本输入输出（课堂练习 Exercise 1 & 2）

```
// C++11
#include <iostream>
#include <string>

int main() {
    // 输出：三个值用同一个 << 接口
    std::cout << "Hello, Streams!\n";
    std::cout << 42 << '\n';
    std::cout << 3.14 << '\n';

    // 输入：先提示、再读取
    std::string name;
    int number;
    std::cout << "Enter your name: ";
    std::cin >> name;               // 读到下一个空白为止（假设名字是单个单词）
    std::cout << "Enter your favourite number: ";
    std::cin >> number;

    std::cout << "Hello " << name << ", your favourite number is "
              << number << "!\n";
    return 0;
}
```

- **代码做什么**：先打印三行（字符串、int、double），再提示用户输入名字和数字并回显。
- **特性机制解说**：`<<` 对**每种内置类型都有重载**（`ostream& operator<<(ostream&, int/double/const char*/...)`），编译器按实参类型自动挑选——这就是"类型安全"：`printf("%d", 3.14)` 是未定义行为，而 `std::cout << 3.14` 永远正确。`cin >> name` 与 `cin >> number` 之间由空白分隔，所以输入 `Alice 7` 或分行输入 `Alice\n7` 效果相同。

示例 3：ofstream / ifstream 文件读写（课堂练习 Exercise 4/6 改写，含追加）

```cpp
// C++11
#include <fstream>
#include <iostream>
#include <string>

int main() {
    // 写：默认 std::ios::trunc —— 文件被截断重建
    std::ofstream out("grades.txt");
    if (!out.is_open()) {
        std::cerr << "Error: could not open file\n";
        return 1;
    }
    out << "Alice" << " " << 92 << "\n";
    out.close();                    // 关闭后不能再写

    // 追加：std::ios::app 保留原有内容
    std::ofstream app("grades.txt", std::ios::app);
    if (app.is_open()) {
        app << "Bob" << " " << 87 << "\n";
    }
    app.close();

    // 读回：while 循环直到文件末尾
    std::ifstream in("grades.txt");
    if (!in.is_open()) {
        std::cerr << "Error: could not open file\n";
        return 1;
    }
    std::string name;
    int grade;
    while (in >> name >> grade) {   // 1. 尝试读取  2. 检查是否成功
        std::cout << "Name: " << name << "  |  Grade: " << grade << "\n";
    }
    return 0;
}
```

- **代码做什么**：写入一条成绩 → 追加第二条 → 逐条读回打印（`Alice | 92`、`Bob | 87`）。
- **特性机制解说**：
- `while (in >> name >> grade)` 是**读取循环的标准形态**：`>>` 返回流本身，流在 `failbit` 置位（读到 EOF 或类型失败）时 `operator bool` 为 `false`，循环自然终止——绝不要写 `while (!in.eof())`。
- `std::ios::trunc` 是默认行为：每次运行文件内容被清空重写；`std::ios::app` 让每次打开都在文件尾追加（课堂 post-practice 的成绩追踪器正是靠它累计 3 名学生）。
- `cerr` 无缓冲、立即输出，适合错误信息。

示例 4：cin 失败与 getline 混用的经典 bug（幻灯片 cinGetline 场景）

```
// C++11
#include <iostream>
#include <string>

int main() {
    double pi, tao;
    std::string name;

    std::cin >> pi;                 // 输入: 3.14
    std::getline(std::cin, name);   // 第一次 getline: 吃掉 >> 残留的 '\n' → name = ""
    std::getline(std::cin, name);   // 第二次 getline: 真正读入 "Rachel Fernandez"
    std::cin >> tao;                // 输入: 6.2

    std::cout << "my name is: " << name
              << " tao is: " << tao
              << " pi is: " << pi << '\n';
    return 0;
}
```

- **代码做什么**：输入 `3.14`、`Rachel Fernandez`、`6.2`，期望打印 `my name is: Rachel Fernandez tao is: 6.2 pi is: 3.14`。
- **特性机制解说**：这是本讲最经典的 bug 演示。`cin >> pi` 读到 `3.14` 后把 `'\n'` **留在缓冲区**；紧接着的 `getline` 立刻读到这个 `'\n'`，返回**空串**并吞掉换行。若不补一次 `getline`，后续 `cin >> tao` 会去读 `"Fernandez"` ——`double` 解析失败，`tao` 变成垃圾值且流进入 `failbit`。解法（幻灯片）：在数字读取后**多调一次 `getline` 吃掉残留换行**。课程结论："Don't use `getline()` and `std::cin >>` together, unless you really really have to!"

示例 5：逐行读文件 + stringstream 解析 CSV（A1 预演）

```cpp
// C++11
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::ifstream in("courses.csv");
    if (!in.is_open()) {
        std::cerr << "Error: could not open file\n";
        return 1;
    }

    std::string line;
    std::getline(in, line);             // 跳过列名（header）行

    while (std::getline(in, line)) {    // 每个记录是一整行
        std::stringstream ss(line);     // 把这一行当流处理
        std::string title, units, quarter;
        std::getline(ss, title, ',');   // 用逗号作分隔符
        std::getline(ss, units, ',');
        std::getline(ss, quarter);
        std::cout << title << " | " << units << " | " << quarter << '\n';
    }
    return 0;
}
```

- **代码做什么**：逐行读取 CSV，跳过第一行列名，把每行按逗号切成三列并打印。
- **特性机制解说**：`getline` 的第三个参数可以换成任意分隔符——这里把默认的 `'\n'` 换成 `','`，实现"按列读取"；外层再按 `'\n'` 按行读取，形成"行内套列"的两级解析。这正是 A1 中 `parse_csv` 的思路：`ifstream` 按行 + `stringstream` / `split` 按逗号分列。

与旧标准（如C++98）的对比

- 流本身是 **C++98 时代就存在**的设施：`<iostream>`、`<fstream>`、`<sstream>` 及其所有核心 API（`<<`、`>>`、`getline`、`flush`、`open/close/is_open`）从 C++98 至今几乎没有变化。本讲内容的"新旧对比"主要发生在**外围工具**上：
- **对比 C 风格 I/O（printf / scanf / FILE*）**：C 的 I/O 类型不安全（格式串 `%d` 与实参类型不匹配是未定义行为）、不可扩展（无法为自定义类型重载）、失败检测靠返回值容易漏。C++ 流对每种内置类型都有重载，自定义类型可在后续课程（L12 运算符重载）中扩展 `<<` / `>>`，失败有 `failbit` / `eofbit` 状态可查。
- **字符串 ↔ 数字转换**：C++98 只能靠 `stringstream`（如 `ss << 42; ss >> str;`）或 C 函数；C++11 起引入 `std::to_string`、`std::stoi` / `std::stod` 等，简单转换不再需要流。
- **`std::endl` 与 `'\n'`**：自 C++98 起就存在"`endl` 附带 flush"这一行为，现代 C++ 社区一致建议循环输出用 `'\n'`（本讲幻灯片也明确要求 "Use '\n'!"）。

- **展望 C++23**：`<print>` 头文件引入 `std::print` / `std::println`，格式化输出更简洁高效；但流仍是通用、可扩展 I/O 的基石，理解缓冲与状态机制对任何 I/O 方式都适用。

关键要点

1. 流是一条"传送带"：输入流用 `>>` 读、输出流用 `<<` 写，同一接口覆盖控制台、文件、字符串——这就是抽象的力量。
2. `>>` 按空白分词、**不消费末尾分隔符**；`getline` 读整行、**消费分隔符**。两者混用前，先想清楚缓冲区里还剩什么。
3. 输出有缓冲：`'\n'` 只换行，`std::endl` 换行 **+ flush**；大量循环输出用 `'\n'`，flush 交给程序结束、缓冲区满与 tied 流。
4. 文件流：构造后先查 `is_open()`；`close()` 后写入静默失败；追加用 `std::ios::app`；读文件用 `while (in >> x)` 而非 `!in.eof()`。
5. `cin` 提取失败会置 `failbit` 并锁死后续读取；遇到 `>>` 与 `getline` 混用时，用"额外一次 getline"消费残留换行。

常见陷阱与注意事项

1. **getline 与 `cin >>` 混用**：`cin >> pi` 之后缓冲区残留 `'\n'`，紧跟的 `getline` 读到空串。修复：多调一次 `getline`（或 `std::cin.ignore()`）吃掉换行。课程原话："Don't use getline() and std::cin together, unless you really really have to!"
2. **对已关闭的流写入**：`ofs.close()` 之后 `ofs << ...` 静默失败、不报错，排查时极易忽略。
3. **把 `std::endl` 当换行符**：循环内每次迭代都 flush，I/O 次数暴涨，程序明显变慢——`'\n'` 才是换行符。
4. **`cin >> double` 读到非数字**：如 `cin >> tao` 遇到 `"Fernandez"`，提取失败、`failbit` 置位、之后所有读取失效（示例 4 的垃圾值场景）。
5. **忘记检查 `is_open()`**：文件不存在或路径错误时直接读写，读侧得到空数据、写侧悄悄丢弃——先检查再操作。

关联作业提示

A1: SimpleEnroll 几乎完全建立在流之上：

- Part 1 **`parse_csv`**：用 `std::ifstream` 打开 `courses.csv`；注意"**每个记录是一整行**"，所以外层用 `std::getline(ifs, line)` 逐行读，跳过第一行列名（header）；再用提供的 `split`（内部是 `stringstream`）或 `getline(ss, field, ',')` 按逗号分列，把 Title / Units / Quarter 填入 `Course` 结构体。

- **Part 2/3 写文件**：用 `std::ofstream` 写 `student_output/courses_offered.csv` 与 `courses_not_offered.csv`；务必先写回列名行，且格式严格为 `<Title>,<Units>,<Quarter>`（**逗号后无空格**），否则 autograder 不通过；写完用 `close()`。
- **流与类型**：`Course` 结构体的字段（`std::string` 与 `int`）正是流的 `<<` / `>>` 能直接处理的类型——回忆幻灯片问题："remember what types streams deal with?"（流处理的都是可 `<<` / `>>` 的类型）。
- **提醒**：不要在遍历 `all_courses` 的同时删除元素（A1 明确警告）——先把要保留的课程收集到新容器，再统一删除。

Lecture 5 (Week 3 – Tuesday): 容器 (Containers)

概述

本讲介绍 C++ 标准库（STL）的**容器（containers）**：`std::vector`、`std::deque`、`std::map`、`std::set`、`std::unordered_map`、`std::unordered_set`。你将学会每种容器的数据结构本质、时间复杂度权衡与适用场景，以及如何统一地遍历它们（for-each 循环，其底层机制是下一讲的主角——迭代器）。容器解决"如何存储一组相关的东西"，是 STL 三大件（容器 / 迭代器 / 算法）的基石，也是 A2（Marriage Pact）的核心工具。幻灯片为图片型（文本极少），本笔记依据课程主题、课堂代码（`temperature.cpp`、`double-agent.cpp`）与 C++ 专业知识整理。

核心特性与语法详解

1. 容器是什么（Container）

- **定义与目的**：容器是"存储一组对象"的数据结构抽象。STL 提供多种容器，各有不同的**底层实现**与**操作代价**，对应不同场景。
- **核心语法**：`std::vector<int> v {1, 2, 3};`（C++11 起支持 `{}` 列表初始化）。
- **设计意图**：不自己手写链表/数组/树——标准库已经实现并优化好了。选择容器的本质是选择**时间复杂度**：同一个操作在不同容器上开销可以差出几个数量级。

2. 序列容器：std::vector 与 std::deque

- **定义与目的**：按"位置"组织元素，元素有先后顺序、可按下标访问。
- **std::vector——动态数组**：元素存储在**一块连续内存**中。
- 随机访问 `v[i]`：**O(1)**
- 尾部插入/删除 `push_back` / `pop_back`：**均摊 O(1)**（容量不足时整体搬家扩容，均摊后仍为常数）
- 头部/中间插入删除 `insert` / `erase`：**O(n)**（后续元素全部平移）
- 适用：绝大多数默认选择——需要随机访问、主要在尾部增删。

- **std::deque——双端队列**：分块连续内存（多段连续缓冲拼接）。
- 头部与尾部插入/删除 `push_front` / `push_back` / `pop_front` / `pop_back`：**均摊 O(1)**
- 随机访问：**O(1)**（分块定位）
- 中间插入删除：**O(n)**
- 适用：需要**在两端都频繁增删**（本讲课后小测："Which type(s) lets you insert at the back and front equally efficiently?" → `std::deque`）。
- **vector 没有 `push_front`**：头部插入只能用 `insert(v.begin(), x)`（O(n)）——这是设计上"不给你慢方法"的体现（下一讲会再次遇到这个哲学）。

3. 有序关联容器：std::map 与 std::set

- **定义与目的**：按键（key）组织数据，内部是**红黑树（平衡二叉搜索树）**，元素按键**有序**存储。
- **std::map**：键 → 值的映射，键**唯一**。查找/插入/删除：**O(log n)**。遍历时按键**升序**输出。
- **std::set**：只有键、没有值的"集合"，键唯一、有序，查找/插入/删除 **O(log n)**。
- **核心语法**：
```
cpp std::map<std::string, int> ages {{"Alice", 20}, {"Bob", 21}};
ages["Carol"] = 19; // 插入或更新 if (ages.find("Alice") != ages.end()) { /* 存在 */ }
std::set<int> s {3, 1, 2}; // 内部按 1,2,3 排序存储
```
- **比较运算符要求**：`map` / `set` 的键类型必须支持 `operator<`（严格弱序），因为红黑树靠比较维持有序性——这正是课后小测第二问："Which type(s) requires a comparison operator on the element type?" → `std::map, std::set`。
- **适用**：需要有序遍历、范围查询、按序输出；能接受 O(log n) 的代价。

4. 哈希关联容器：std::unordered_map 与 std::unordered_set

- **定义与目的**：用**哈希表（hash table）**组织数据，以空间换时间。
- 查找/插入/删除：**均摊 O(1)**（理想情况；最坏 O(n)，取决于哈希质量与负载因子）。
- **不需要比较运算符**，需要的是**哈希函数**（键类型提供 `std::hash<T>`）与相等比较 `operator==`。
- **遍历顺序无意义**：元素按桶（bucket）存放，顺序与插入顺序、大小关系都无关。
- **为什么通常更快**：课后小测第三问："Which is usually faster: unordered_set or set? Why?" → `unordered_set`：哈希 + 较小的负载因子（load factor）让查找期望 O(1)，而 `set` 的红黑树查找严格 O(log n)；元素多时差距明显。
- **适用**：只需要"键存在与否 / 键 → 值"、不需要有序输出时，优先选 unordered 版本。

5. 容器的初始化方式

- **核心语法**（C++11 起，`{}` 统一初始化 + initializer_list）：
```
cpp std::vector<int> v {1, 2, 3}; // 三个元素 1,2,3 std::vector<int> v2(3, 7); // 三个元素都是 7（大小 + 初值）
std::map<std::string, int> m {{"a", 1}, {"b", 2}}; // 嵌套 {} 表示键值对
std::set<std::string> names {"Chris", "Nick", "Sean"};
```

- **注意区分**：`vector<int> v{3}` 是"一个元素 3"；`vector<int> v(3)` 是"三个元素 0"。花括号优先匹配 initializer_list。

6. 遍历容器：for-each 循环

- **定义与目的**：统一遍历所有容器（vector/deque/map/set/unordered_*）的方式。
- **核心语法**：`cpp for (const auto& elem : container) { /* 使用 elem */ } for (const auto& [key, value] : map) { /* C++17 结构化绑定 */ }`
- **机制**：for-each 是语法糖，编译器把它展开成迭代器循环（`auto b = c.begin(); auto e = c.end(); for (auto it = b; it != e; ++it) { auto& elem = *it; ... }`）——这正是下一讲（L6）的核心内容。`map` 的元素类型是 `std::pair<const K, V>`，所以遍历 `map` 时 `elem.first` 是键、`elem.second` 是值。
- **最佳实践**：只读遍历用 `const auto&`（避免拷贝大对象）；需要修改元素用 `auto&`；`unordered_map` 的遍历顺序不保证，别依赖。

7. 常用成员函数速查

操作	vector	deque	map/set	unordered_map/set
随机访问 `[i]` / `at(i)`	O(1)	O(1)	—（无下标）	—（无下标）
查找 `find`	O(n)（线性扫）	O(n)	O(log n)	均摊 O(1)
插入	尾部均摊 O(1)	两端 O(1)	O(log n)	均摊 O(1)
`push_back` / `push_front`	有 / 无	都有	—	—
有序遍历	插入序	插入序	按键升序	无序
元素类型要求	无	无	键支持 `operator<`	键支持 `std::hash` + `==`

代码示例与逐步解说（核心）

示例 1：std::vector 与线性扫描（课堂代码 temperature.cpp 补全）

```cpp
// C++11
#include <iostream>
#include <vector>

// 返回最高温度；没有温度数据时返回 -1
int findPeakHeat(const std::vector<int>& temps) {
    if (temps.empty()) {
        return -1;                      // 空容器保护：不要访问 temps[0]
    }
    int best = temps[0];
    for (const auto& t : temps) {       // 遍历：t 是 const int&
        if (t > best) best = t;
    }
    return best;
}

int main() {
    // 未来 7 天最高气温预报（C++11 列表初始化）
    std::vector<int> weeklyForecast = {62, 63, 65, 68, 69, 66, 65};
    std::cout << "Max temp this week will be " << findPeakHeat(weeklyForecast) << '\n';
    return 0;
}
```

- **代码做什么**：把 7 个温度放进 `vector`，用 for-each 找出最大值并打印。
- **特性机制解说**：`std::vector<int>` 在堆上维护**一块连续内存**，`weeklyForecast` 只持有指向这块内存的指针、大小与容量。`const auto& t` 让 `t` 成为元素的**引用**（不拷贝 int 本身，虽然 int 拷贝便宜，但这是好习惯的起点）；`empty()` 先于 `[0]` 检查，避免对空容器访问未定义行为。若数据量更大，可直接用 `<algorithm>` 的 `std::max_element`（L10 之后会讲），但手写 for-each 更能理解容器语义。

示例 2：std::map + std::set 找"双重身份员工"（课堂代码 double-agent.cpp 补全）

```cpp
// C++17（结构化绑定；若只支持 C++11 可改用 pair.first/.second）
#include <iostream>
#include <map>
#include <set>
#include <string>

// 找出出现在多个部门里的员工
std::set<std::string> findDoubleAgents(
        const std::map<std::string, std::set<std::string>>& departments) {
    std::set<std::string> seen;
    std::set<std::string> doubleAgents;

    for (const auto& [dept, members] : departments) {  // C++17 结构化绑定
        for (const auto& name : members) {             // 遍历每个部门的员工集合
            if (seen.count(name)) {                    // 之前见过 → 双重身份
                doubleAgents.insert(name);
            } else {
                seen.insert(name);
            }
        }
    }
    return doubleAgents;                               // 按名字升序返回
}

int main() {
    std::map<std::string, std::set<std::string>> company = {
        {"Sales",      {"Jim", "Dwight", "Phyllis"}},
        {"Accounting", {"Angela", "Oscar", "Kevin"}},
        {"Pranks",     {"Jim", "Pam"}}
    };

    std::set<std::string> multiTaskers = findDoubleAgents(company);
    std::cout << "Double Agents: ";
    for (const auto& name : multiTaskers) std::cout << name << " ";  // 应打印 Jim
    std::cout << '\n';
    return 0;
}
```

- **代码做什么**：公司是 `map<部门, set<员工>>`；遍历所有部门的所有员工，凡在多个部门出现的名字（Jim 同时在 Sales 和 Pranks）放入结果 `set` 并打印。
- **特性机制解说**：
- `std::map` 的遍历元素是 `std::pair<const std::string, std::set<std::string>>`——C++17 结构化绑定 `[dept, members]` 等价于 C++11 的 `pair.first` / `pair.second`。
- `set` 的 `count(x)` 返回 0 或 1（键唯一），可当"是否存在"用；`insert` 保持有序（红黑树），所以结果自动按字母序输出。
- 嵌套容器 `map<string, set<string>>` 展示了容器可任意组合，这是 STL"组件可拼装"的设计。
- 整个函数是纯"容器 + 遍历"逻辑，没有手写任何内存管理——这正是容器的价值。

示例 3：std::unordered_map 词频统计（补充示例）

```
// C++17（结构化绑定；C++11 可用 pair.first/.second）
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_map>

int main() {
    std::string text = "the quick brown fox jumps over the lazy dog the end";
    std::stringstream ss(text);

    std::unordered_map<std::string, int> freq;
    std::string word;
    while (ss >> word) {        // 按空白分词
        ++freq[word];           // operator[]：不存在则默认构造 0，再自增
    }

    for (const auto& [w, c] : freq) {
        std::cout << w << ": " << c << '\n';
    }
    return 0;
}
```

- **代码做什么**：统计一句话里每个单词出现次数。
- **特性机制解说**：`++freq[word]` 依赖 `operator[]` 的语义——键不存在时**默认构造一个 `int{0}` 并插入**，然后自增。这是 `map` / `unordered_map` 最方便但也最危险的操作（见陷阱 1）。哈希表让每个单词的查找均摊 O(1)；遍历顺序由桶布局决定、与输入顺序无关。若要"按词频从高到低"输出，需要先把条目搬到 vector 再排序——`unordered_map` 本身不提供有序遍历。

示例 4：map 的 operator[] 陷阱（补充示例）

```cpp
// C++11
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> scores;
    scores["Alice"] = 92;       // 插入

    std::cout << scores["Bob"] << '\n';   // 危险：查询时也插入了 {"Bob", 0}！

    if (scores.find("Bob") != scores.end()) {   // C++20 可写作 scores.contains("Bob")
        std::cout << "Bob exists\n";
    } else {
        std::cout << "Bob does not exist\n";
    }

    std::cout << "size = " << scores.size() << '\n';   // 2，而不是 1！
    return 0;
}
```

- **代码做什么**：试图"查询"Bob 的成绩，结果 Bob 被凭空插入 map。
- **特性机制解说**：`m[k]` 的完整语义是"**如果键不存在，就默认构造一个值插进去**，然后返回值的引用"。因此**只做存在性检查绝不能用 `[]`**——应该用 `find`（C++20 起可用 `contains`）。这个陷阱对 `std::map` 与 `std::unordered_map` 同样成立。

示例 5：std::deque 双端操作（补充示例）

```cpp
// C++11
#include <deque>
#include <iostream>
#include <vector>

int main() {
    std::deque<int> d;
    d.push_back(1);             // 尾部插入 O(1)
    d.push_front(0);            // 头部插入 O(1)
    d.push_back(2);
    std::cout << d[0] << ' ' << d[1] << ' ' << d[2] << '\n';  // 0 1 2（随机访问 O(1)）

    std::vector<int> v {1};
    v.push_back(2);             // 尾部均摊 O(1)
    // v.push_front(0);         // ❌ vector 没有 push_front！
    v.insert(v.begin(), 0);     // 头部插入要整体平移：O(n)
    return 0;
}
```

- **代码做什么**：对比 deque 与 vector 在头部插入的代价。

- **特性机制解说**：deque 内部是"分块连续 + 中央索引"的结构，两端都能 O(1) 增删、仍支持 O(1) 下标访问（比 `std::list` 强）。课后小测第一问的答案正是 `std::deque`。而 vector 没有 `push_front`——C++ 的哲学是**不提供注定慢的操作**，逼你根据场景选对容器。

与旧标准（如C++98）的对比

- `std::vector`、`std::deque`、`std::map`、`std::set` **C++98 已有**（源自 1994 年左右的 SGI STL），底层结构与复杂度约定至今未变。真正的变化在**使用体验**：
- **无序容器**：`unordered_map` / `unordered_set` 在 C++11 才进入标准（C++98 时代只能靠 TR1 的 `std::tr1::unordered_map` 或第三方库）。此前要实现 O(1) 查找只能手写哈希表。
- **初始化**：C++98 没有 `{}` 列表初始化，容器只能"先构造再逐个 insert/push_back"；C++11 的 initializer_list 让 `std::map<std::string, int> m {{"a",1},{"b",2}};` 一行完成。
- **遍历**：C++98 写 `for (std::map<std::string, int>::iterator it = m.begin(); it != m.end(); ++it)`，类型冗长易错；C++11 的 `auto` + 范围 for 让遍历变成 `for (const auto& p : m)`。
- **C++17 结构化绑定**：`for (const auto& [k, v] : m)` 取代了 `pair.first/.second` 的手工解包；C++17 的 `map::try_emplace` 解决"先查再插"的重复哈希问题。
- **C++20 `contains()`**：`m.contains(k)` 比 `m.find(k) != m.end()` 更直白；C++20 的 `ranges` 视图进一步简化了容器管道操作（L10 之后涉及）。

关键要点

1. **先选对容器，再写代码**：随机访问 + 尾部增删 → vector；双端频繁增删 → deque；按键有序查找 → map/set（O(log n)）；只要 O(1) 查找、不在乎顺序 → unordered_map/unordered_set。
2. **有序 vs 无序是根本分歧**：`map` / `set` 要求键支持 `operator<` 且遍历有序；`unordered_*` 要求键支持 `std::hash` + `operator==`，遍历无序但通常更快（哈希 + 小负载因子）。
3. **for-each 统一遍历一切容器**：`for (const auto& elem : c)` 对 vector/deque/map/set/unordered_* 都成立（map 元素是 pair）；只读用 `const auto&`。
4. **`operator[]` 会插入默认值**：存在性检查用 `find` / `contains` / `count`，不要用 `[]`。
5. **遍历时不要修改容器结构**（插入/删除元素会使迭代器失效）——收集到新容器后再统一操作。

常见陷阱与注意事项

1. **用 `m[k]` 做存在性检查**：会默默插入默认值，污染数据、改变 `size()`。改用 `find`（C++20 `contains`）、`set` 用 `count`。
2. **对空容器取元素**：`v[0]`、`v.front()`、`v.back()` 在容器为空时是未定义行为——先 `empty()` 检查（示例 1 的 `findPeakHeat` 返回 -1 正是为此）。

3. **`vector<int> v{3}` vs `v(3)`**：前者是"一个元素 3"，后者是"三个元素 0"——花括号优先匹配 initializer_list，语义完全不同。
4. **误以为 unordered 容器有序**：`unordered_map` 的遍历顺序由哈希桶决定，与插入顺序、键大小都无关；需要有序输出时请用 `map`，或把条目搬到 vector 再 `std::sort`。
5. **遍历中插入/删除元素**：`v.erase(it)` 或 `m.insert` 使相关迭代器失效（vector 尤甚，可能全部失效），在循环里改容器会得到未定义行为——先收集、后修改。
6. **键类型不满足要求**：给 `map`/`set` 用没有 `operator<` 的键、给 `unordered_*` 用没有 `std::hash` 的类型，编译器直接报错——这不是 bug 而是设计的善意提醒。

关联作业提示

A2: Marriage Pact 的核心工具就是容器与指针：

- Part 1 **`get_applicants`**：从 `students.txt` 逐行读名字，存进 `std::set`（或 `std::unordered_set`，二选一，需同步修改函数签名）。这正是本讲容器选择的实战：几千个名字、只需判存在/遍历，两者皆可；`short_answer.txt` 的 Q1 要求你书面回答两者权衡（有序 vs O(1) 哈希、内存、哈希函数质量）并举一个**非课堂示例**的合法哈希函数——本讲的复杂度对比表就是你的论据。
- Part 2 **`find_matches`**：遍历 `students` 集合（用 for-each 或迭代器），对每个名字调用你自己写的 `initials()` 辅助函数，与 `kYourName` 的缩写比较；匹配的名字把**指针**放进 `std::queue<const std::string*>`——注意存的是 `std::string*`（指向 set 中字符串的地址）而不是字符串本身，这正是本讲"容器存对象、指针指对象"的衔接。
- **`get_match`**：从 queue 取"真命天子"时注意 queue 为空的情况（打印 `"NO MATCHES FOUND."`）。
- Q2 **简答题**："为什么存指针而不是名字？set 出作用域后指针会怎样？"——答案与容器元素地址的稳定性、指针悬垂（dangling）有关，是 L6 指针内容的直接延伸，务必结合本讲容器内存模型回答。

Lecture 6 (Week 3 – Thursday): 迭代器与指针 (Iterators & Pointers)

概述

本讲回答上一讲遗留的问题："for-each 循环（`for (const auto& elem : container)`）到底是怎么工作的？"答案是**迭代器（iterators）**：一个在容器中"跟踪当前位置、能前进、能取值"的抽象。你将掌握迭代器的四件套接口（`begin`/`end`/`++`/`*`/`==`）、五大迭代器分类（Input / Output / Forward / Bidirectional / Random Access），并理解指针与内存的基本模型——指针是指向内存中任意对象的"地址数字"，迭代器与指针接口同构。这是 STL 泛型算法（L10 的 `std::sort` 等）能作用于所有容器的基础，也是 A2（Marriage Pact）中"存指针到 queue"与指针悬垂问题（Q2）的知识来源。

核心特性与语法详解

1. 为什么需要迭代器

- **定义与目的**：用下标遍历（`for (size_t i = 0; i < v.size(); ++i) v[i]`）只对**连续容器**（vector）有效；`std::set` / `std::map` 没有下标、内部不是数组，无法用索引。我们需要一个**对所有容器统一的遍历抽象**。
- **核心语法**（四件套）：`cpp auto it = c.begin(); // 1. 初始化：指向容器第一个元素 ++it; // 2. 前进：移动到下一个元素 auto& elem = *it; // 3. 解引用：取得当前元素（it == end() 时未定义！） it == c.end() // 4. 比较：判断是否走完了`
- **设计意图与最佳实践**：课堂用"抓娃娃机"作比喻——迭代器是那只"爪子"：能抓玩具（`*it`）、能向前移（`++it`）；容器是那台机器：告诉你从哪里开始（`begin()`）、何时停止（`end()`）。容器与迭代器**协作**，才构成遍历。

2. begin() 与 end() 的语义

- **定义与目的**：`begin()` 返回指向**第一个元素**的迭代器（容器非空时）；`end()` 返回**past-the-end** 迭代器——指向**最后一个元素之后一个位置**，它**从不指向任何元素**，只作为"走完了吗？"的哨兵。
- **核心语法**：`cpp auto b = c.begin(); auto e = c.end(); for (auto it = b; it != e; ++it) { ... }`
- **机制**：
- **空容器**：`begin() == end()`，循环体一次都不执行。这是循环能安全处理空容器的基础。
- 对 `end()` 解引用（`*c.end()`）是**未定义行为**——end 后面没有元素。
- `--e`（先 `auto e = c.end(); --e;`）可得到最后一个元素的迭代器（要求双向迭代器）。

3. for-each 循环的展开（编译器做了什么）

- **定义与目的**：`for (auto elem : s)` 是迭代器循环的**语法糖**。
- **核心语法**（幻灯片给出的等价展开）：`cpp // 你写的： for (auto elem : s) { std::cout << elem; } // 编译器看到的： auto b = s.begin(); auto e = s.end(); for (auto it = b; it != e; ++it) { auto elem = *it; std::cout << elem; }`
- **机制**：因此 for-each 对**任何提供 begin/end 与迭代器四件套的类型**都成立——这正是它能同时遍历 vector、map、set、deque、unordered_* 的原因（上一讲的课后小测即由此引出）。

4. 迭代器的类型：为什么那么长，以及 auto / using

- **定义与目的**：`std::map<int, int>::iterator` 这类类型名极其冗长。
- **核心语法**：`cpp std::map<int, int> m {{1,2},{3,4},{5,6}}; auto it = m.begin(); // C++11 起：让编译器推断 auto elem = *it; // elem 是 std::pair<int, int> // 显式写法（C++98 风格）： std::map<int, int>::iterator it2 = m.begin();`

- **机制**：`<map>` 头文件内部有 `using iterator = /* 某种迭代器类型 */;`（`using` 是类型别名，等价于 C 的 `typedef`）。`*m.begin()` 的类型是 `std::pair<const int, int>`，所以 `auto elem = *it;` 得到的是 pair 的**拷贝**。幻灯片提醒："Iterator types are really long, so we like to use `auto` with iterators."

5. `++it` 还是 `it++`？

- **定义与目的**：前缀与后缀自增语义不同，代价不同。
- **核心语法**（迭代器类的两个重载）：`cpp Iterator& operator++(); // 前缀 ++it：先自增，返回指向同一对象的引用 Iterator operator++(int); // 后缀 it++：自增，但返回旧值的拷贝`
- **机制**：迭代器是**完整的对象**，拷贝它比拷贝 `int` 贵得多。后缀版本为了返回旧值必须做一次拷贝，因此循环里统一写 `++it`。Bjarne 的原话："`++i` is sometimes faster than, and is never slower than, `i++` ... why not just write `++i` instead? You never lose anything, and you sometimes gain something."

6. 迭代器分类（五大类型）

- **定义与目的**：不是所有迭代器生而平等——不同容器支持的操作不同，按"能力"分级；算法（如 `std::sort`）会要求最低级别的迭代器。
- **分类（能力从弱到强）**：
- **Input（输入）**：最基本，只能**读**：`auto elem = *it;`。单遍（single-pass）语义，如流迭代器。
- **Output（输出）**：只能**写**：`*it = elem;`。
- **Forward（前向）**：Input 的加强，支持**多遍遍历**（multi-pass guarantee：`it1 == it2` 蕴含 `++it1 == ++it2`，即同一个迭代器可以反复前进、重走同一段）。**所有 STL 容器迭代器都至少是 Forward**。为什么流不是？——流读一次就没了，无法重放，所以流迭代器只是 Input。
- **Bidirectional（双向）**：Forward 之上支持 `--it` 后退。`std::map`、`std::set` 的迭代器属于此类（红黑树节点只能沿链前后移动）。
- **Random Access（随机访问）**：最强大，支持 `it += n`、`it - n`、`it[n]`、`it1 < it2` 等"任意跳转与比较"。`std::vector`、`std::deque` 的迭代器属于此类（底层连续/近连续内存）。
- **机制与为什么重要**：`std::sort` 需要**随机访问迭代器**（它要反复跳跃划分区间）。`std::sort(vec.begin(), vec.end())` ✅ 可编译；`std::sort(set.begin(), set.end())` ❌ 编译失败——set 的迭代器只是双向的。C++ 的设计哲学："**不提供注定慢的方法**"：在红黑树上做 `it + 5` 是 O(n) 的慢操作，所以 map/set 的迭代器干脆不提供随机访问，让错误在编译期暴露。

7. 指针与内存（Pointers and Memory）

- **定义与目的**：迭代器指向容器元素；**指针指向内存中的任意对象**。理解指针前先理解内存。
- **内存基础**：
- 每个变量都住在内存的某个地址；程序的所有地址构成**地址空间**（64 位系统从 `0x0` 到 `2^64-1`）。

- 内存**按字节寻址**：每个字节一个编号；对象的地址 = 它**最低字节**的地址。`int` 占 32 位 = 4 字节，`int x = 106;` 的地址是其 4 个字节中最低的那个（字节序有大小端之分，演示常用 Big Endian，实际多为 Little Endian）。
- **取地址与解引用**：`cpp int x = 106; int* px = &x; // & = 取地址运算符：得到 x 的地址 std::cout << *px; // * = 解引用：顺着地址取回对象，打印 106`
- **指针就是"一个数字"**：`px` 里存的不过是一个无符号整数（地址值，如 `0x50527c`），打印 `px` 看到的就是这个数。
- **指针可以指向任何对象**：结构体 `StanfordID* p = &id; p->name;`（`->` 等价于 `(*p).`）、`std::vector<int>* p = &v;`、数组元素 `int* arr = &v[0];`。
- **数组指针算术**（vector 是连续内存，所以指针算术成立）：`cpp int* arr = &v[0]; arr += 1; // 前进一个 int（不是 1 字节！按所指类型缩放） ++arr; if (arr == &v[4]) ...`
- **迭代器与指针同构**：`vector<T>::iterator` 的底层类型几乎就是 `T*`（真实实现中不是裸指针，但接口完全一致：初始化、`+=`、`++`、`*`、`==`）。所以对 vector 而言，指针遍历和迭代器遍历写起来一模一样。

代码示例与逐步解说（核心）

示例 1：用迭代器遍历 std::set（幻灯片示例，含 for-each 对照）

```
// C++11
#include <iostream>
#include <set>

int main() {
    std::set<int> s {1, 2, 3, 4};

    // 手写迭代器循环（set 没有下标，只能这样遍历）
    for (auto it = s.begin(); it != s.end(); ++it) {
        std::cout << *it << ' ';
    }
    std::cout << '\n';

    // for-each 是上面这段的语法糖（编译器自动展开）
    for (auto elem : s) {
        std::cout << elem << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

- **代码做什么**：用两种等价方式打印 `1 2 3 4`（set 自动升序）。
- **特性机制解说**：`s.begin()` 是 `std::set<int>::iterator`；`++it` 沿红黑树的中序遍历移动到下一个节点；`*it` 解引用取出节点里的 `int`；`it != s.end()` 用哨兵判断结束。注意迭代器是**双向**的——

`s.begin()` 没有下标、`it += 2` 会编译失败，这正体现了"容器实现决定迭代器能力"。

示例 2：迭代器四件套与空容器（幻灯片接口逐条演示）

```
// C++11
#include <iostream>
#include <vector>

int main() {
    std::vector<char> v {'d', 'a', 'w', 'g', 's'};

    auto it = v.begin();              // 1. 初始化：指向 'd'
    ++it;                             // 2. 前进：指向 'a'
    auto& elem = *it;                 // 3. 解引用：拿到元素的引用
    std::cout << elem << '\n';        // a
    std::cout << (it == v.end() ? "at end\n" : "not at end\n");  // not at end

    ++it; ++it; ++it; ++it;           // 4 次前进：'w' → 'g' → 's' → 哨兵位
    std::cout << (it == v.end() ? "reached end\n" : "not end\n"); // reached end

    std::vector<int> empty;
    std::cout << (empty.begin() == empty.end()
                      ? "empty: begin == end\n" : "?\n");
    return 0;
}
```

- **代码做什么**：完整走一遍 `begin/++/*/==` 四件套，并验证空容器的 `begin() == end()`。
- **特性机制解说**：`end()` 返回的哨兵**不指向元素**——`it` 走过第 5 个元素后与 `end()` 相等，此刻若执行 `*it` 就是未定义行为。空容器时 `begin()==end()`，任何"从头走到尾"的循环体自然执行零次，这就是 for-each 能安全处理空容器的原因。

示例 3：用迭代器做递归二分查找（课堂代码 binary-search.cpp 补全）

```
// C++11
#include <iostream>
#include <iterator>   // std::distance
#include <vector>

// 在 [begin, end) 区间内递归查找 value（区间为半开区间）
bool binarySearch(std::vector<int>::iterator begin,
                  std::vector<int>::iterator end, int value) {
    if (begin >= end) {                    // 1. 基准情形：区间为空
        return false;
    }

    auto mid = begin + (std::distance(begin, end) / 2);   // 2. 中点迭代器

    if (*mid == value) {                   // 3. 命中
        return true;
    }
    if (*mid > value) {
        return binarySearch(begin, mid, value);           // 左半 [begin, mid)
    }
    return binarySearch(mid + 1, end, value);             // 右半 [mid+1, end)
}

int main() {
    std::vector<int> data = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int testValues[] = {23, 2, 91, 40};    // 命中 / 首边界 / 尾边界 / 未命中

    std::cout << "Recursive Iterator Binary Search Test:\n";
    for (int val : testValues) {
        std::cout << "Searching for " << val << ": "
                  << (binarySearch(data.begin(), data.end(), val)
                          ? "SUCCESS" : "FAILURE") << '\n';
    }
    return 0;
}
```

- **代码做什么**：对有序 vector 递归二分查找；`begin` / `end` 是迭代器而非下标，体现"用迭代器表达区间"。
- **特性机制解说**：
- `begin >= end` 依赖**随机访问迭代器**才有的比较运算——若把参数换成 `std::set<int>::iterator` 将无法编译（二分查找本来就需要随机访问）。
- `begin + n` 是"跳过 n 个元素"的随机访问运算；`std::distance(begin, end)` 计算两个迭代器之间的元素个数。
- 区间写作 `[begin, end)`（左闭右开）是 STL 的通用约定：左端点包含、右端点（end）不包含。递归分治时天然用 `[begin, mid)` 与 `[mid+1, end)` 表达左右两半。
- 把区间当"值"传来传去，正是迭代器比下标优雅的地方：同一套代码可推广到 deque 等随机访问容器。

示例 4：指针基础——取地址、解引用、指向任意对象（幻灯片示例）

```
// C++11
#include <iostream>
#include <string>
#include <vector>

struct StanfordID {
    std::string name;
};

int main() {
    int x = 106;
    int* px = &x;                  // & 取地址：px 保存 x 的地址
    std::cout << x << '\n';        // 106
    std::cout << *px << '\n';      // 106（* 解引用）
    std::cout << px << '\n';       // 0x…（地址本身，一个"数字"）

    StanfordID id {"rfern"};
    StanfordID* p = &id;
    auto name = p->name;           // -> 等价于 (*p).name

    std::vector<int> v {1, 2, 3, 4, 5};
    int* arr = &v[0];              // 指向 vector 底层连续数组的首元素
    std::cout << *arr << ' ';      // 1
    arr += 1; std::cout << *arr << ' ';   // 2（指针算术按 int 大小缩放）
    ++arr;    std::cout << *arr << ' ';   // 3
    arr += 2; std::cout << *arr << ' ';   // 5
    if (arr == &v[4]) std::cout << "At last index";
    std::cout << '\n';
    return 0;
}
```

- **代码做什么**：展示 `&` 取地址、`*` 解引用、`->` 成员访问、以及指向 vector 内部数组的指针算术，输出 `106 106 0x… 1 2 3 5 At last index`。
- **特性机制解说**：`px` 里存的只是一个地址数字；`*px` 让编译器"按 `int*` 的类型信息去那地址读取 4 字节"。`arr += 1` 前进的不是 1 字节而是 **1 个 `int`（4 字节）**——指针算术按**所指类型的大小**缩放，这正是 `int*` 与 `char*` 区别的根源。vector 保证元素连续存放，所以 `&v[0]` 之后可以放心做数组式指针算术；若指向 set 的节点则完全无效（节点不连续）。

示例 5：迭代器与指针的接口同构（幻灯片对照）

```
// C++11
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v {1, 2, 3, 4, 5};

    // 用裸指针遍历 vector 的底层数组
    int* p = &v[0];
    for (int* q = p; q != p + 5; ++q) std::cout << *q << ' ';
    std::cout << '\n';

    // 用迭代器遍历——接口一模一样
    for (auto it = v.begin(); it != v.end(); ++it) std::cout << *it << ' ';
    std::cout << '\n';

    // 随机访问操作逐条对照
    auto it = v.begin();
    it += 1; std::cout << *it << ' ';             // 2
    ++it;    std::cout << *it << ' ';             // 3
    it += 2; std::cout << *it << ' ';             // 5
    if (it == v.end() - 1) std::cout << "At last element";
    std::cout << '\n';
    return 0;
}
```

- **代码做什么**：先用指针、再用迭代器，走完全相同的"初始化 → 随机访问 → 前进 → 比较"路线，输出两遍 `1 2 3 4 5` 与 `2 3 5 At last element`。
- **特性机制解说**：幻灯片指出，`vector<T>::iterator` 底层类型**几乎就是** `T*`（"In the real STL implementation, the actual type is not `T*`. But for all intents and purposes, you can think of it this way."）。两者共享同一接口：`*`、`++`、`+=`、`==`、`<`。区别在于：迭代器是**类型安全、容器感知**的抽象（对 map/set 会是另一种实现），且自带 `end()` 哨兵约定；指针则更"原始"，可以指向内存里任何地方——包括不该碰的地方。这就是 A2 中"存指针"需要小心的原因。

与旧标准（如C++98）的对比

- **迭代器是 C++98 就有的 STL 核心设施**：`begin()/end()/++/*/==`、五大分类、`std::distance`/`std::advance` 在 C++98 里都已存在。本讲真正的新东西是**使用体验**：
- **C++98 的冗长遍历**：`for (std::map<std::string, int>::const_iterator it = m.begin(); it != m.end(); ++it)`——类型名长到令人窒息，还容易写错 `const_iterator`。**C++11 的 `auto` + 范围 for** 让遍历变成 `for (const auto& p : m)`，由编译器生成迭代器循环。幻灯片正是用"set 没有下标 → 只能迭代器"来引出这一点。
- **C++11 起**：`auto` 推断迭代器类型；范围 for（`for (auto elem : s)`）正式成为语法糖；`cbegin()/cend()` 提供只读迭代器。

- **C++20 ranges**：`std::ranges::sort`、视图（views）把"迭代器对"升级为"范围"抽象，但仍建立在迭代器之上；课程 L10 之后会展开。
- **指针方面**：裸指针与手动 `new/delete` 是 C++98 时代的内存管理常态，极易泄漏；**C++11 的智能指针**（`std::unique_ptr` 等）把"指针"封装成 RAII 对象（课程 L16 详细讲）。本讲先理解裸指针的机制，后续再学安全封装。
- **对比 C**：指针与指针算术（`arr + i`）直接继承自 C；C++ 的迭代器是 C 指针思想的安全泛化——把"指针能做的操作"抽象成接口，让每个容器都能提供。

关键要点

1. **迭代器四件套**：`auto it = c.begin();` → `++it` → `*it` → `it != c.end()`；`end()` 是"过去末尾"的哨兵，**永远不要解引用 end()**。
2. **for-each 就是迭代器循环**：`for (auto elem : s)` 被编译器展开为 `begin/end/++/*` 循环，所以它对一切提供 begin/end 的容器统一有效；空容器由 `begin()==end()` 天然处理。
3. **迭代器按能力分级**：Input(读) → Output(写) → Forward(多遍) → Bidirectional(可退) → Random Access(可跳)；`std::sort` 等算法要求随机访问，map/set 只给双向——"不给慢方法"。
4. **循环里写 `++it` 而不是 `it++`**：前缀不拷贝旧值，"never slower, sometimes faster"（Bjarne）。
5. **指针 = 内存地址 = 一个数字**：`&` 取地址、`*` 解引用、`->` 访问成员；指针算术按所指类型缩放；`vector<T>::iterator` 与 `T*` 接口同构。

常见陷阱与注意事项

1. **解引用 `end()`**：`*c.end()`、`c.end()[0]` 都是未定义行为；**越界访问**（如对 size 3 的 vector，`it += 3` 后再 `*it`，幻灯片专门演示过）同样 UB——迭代器不像下标有 `at()` 帮你检查。
2. **对空容器解引用 `begin()`**：空容器 `begin()==end()`，`*v.begin()` 是 UB；先判空或依赖循环条件。
3. **滥用 `it++`**：把 `i++` 的坏习惯带到迭代器上，每次迭代白做一次对象拷贝；统一 `++it`。
4. **指针/迭代器算术越界**：`arr += 100` 超出数组边界后再解引用是 UB；`arr == &v[4]` 这类比较也只在同一数组内才有定义。
5. **迭代期间修改容器**：`insert` / `erase` 会使迭代器失效（vector 扩容可能使全部失效），之后再用旧迭代器是 UB——本讲先记住现象，L13（特殊成员函数）与后续课程会讲透容器与迭代器的生命周期。
6. **存指针后容器析构**（A2 Q2 的核心）：把指向 `set` 元素的指针存进 `queue`，一旦 `set` 出作用域，指针就**悬垂（dangling）**——解引用是 UB。指针"指向谁"的责任完全在程序员。

关联作业提示

A2: Marriage Pact 是迭代器与指针的实战舞台：

- **Part 2 `find_matches`**：遍历 `std::set<std::string> students` 找与你同缩写的人。set 没有下标，只能用本讲的迭代器循环或 for-each（`for (const auto& name : students)`）；作业提示也明确建议回看"Thursday's lecture on iterators and pointers"。
- **存指针到 `std::queue`**：匹配成功时把 `&name`（指向 set 中字符串的地址，注意不能存局部变量 `name` 的地址，要存 `*it`/元素的地址）push 进 `std::queue<const std::string*>`。
- **Q2 简答题**："为什么存指针而不存字符串？set 出作用域后指针会怎样？"——答案要用到本讲的指针模型：拷贝整个字符串昂贵、指针廉价；`set` 的节点在插入后**地址稳定**（不像 vector 扩容会搬移），所以 set 存活期间指针有效；一旦 `set` 析构，指针悬垂，解引用即 UB。这正是"指针指向的对象生命周期由程序员负责"的体现。
- **binary-search.cpp 的区间思维**：`[begin, end)` 半开区间约定在 A2 遍历 set、以及未来一切 STL 算法（`std::sort`、`std::find`）中反复出现，务必内化。

Lecture 7 (Week 4 – Tuesday): 类 (Classes)

概述

本讲是 C++ 面向对象编程的起点：C 语言没有对象，无法把"数据"与"操作这些数据的函数"封装在一起，而 C++（最初就叫 "C with Classes"）通过 **类（class）** 这一用户自定义类型实现了封装、继承与多态。学习目标是掌握 `struct` 与 `class` 的区别、`public`/`private` 访问控制、头文件（`.h`）与源文件（`.cpp`）的分离、构造器（默认/带参/成员初始化列表/重载）、`this` 指针、getter/setter、析构函数、类型别名，以及继承的三种方式（public/protected/private）、虚函数与 vtable、菱形继承与虚继承。这些是后续讲"类模板""const 正确性"以及作业 A3 的地基。

核心特性与语法详解

1. struct vs class：默认访问级别

- **定义与目的**：`struct` 和 `class` 几乎完全相同，唯一区别是**默认访问级别**：`struct` 的成员默认 `public`，`class` 的成员默认 `private`。结构体适合"纯数据聚合"（如 `std::pair`），类适合"封装 + 行为"。
- **核心语法**：`cpp struct Point { int x; int y; }; // 成员默认 public class Point { int x; int y; }; // 成员默认 private!`
- **设计意图与最佳实践**：类的设计哲学是**信息隐藏**：数据成员放在 `private` 区，只通过 `public` 接口（成员函数）间接访问，这样可以在 setter 里做合法性校验（例如拒绝负数 id），防止用户把对象弄成无效状态。

2. public / private 访问控制

- **定义与目的**：`public` 成员对所有人可见（用户接口）；`private` 成员只对本类可见（实现细节）；`protected` 介于两者之间——对子类可见、对外部不可见（详见本讲"继承"一节）。
- **核心语法**：`cpp class ClassName { private: // 只有本类的成员函数能访问 public: // 所有人都能访问 };`
- **设计意图与最佳实践**："用户能碰 public，摸不到 private"。公开的应该是**做什么**（接口），隐藏的应该是**怎么做**（实现）。后续若改实现，只要接口不变，调用方代码不用动。

3. 头文件（.h）与源文件（.cpp）分离

- **定义与目的**：`.h` 定义**接口**（类声明、函数原型、类型定义、常量），被多个源文件共享；`.cpp` 实现**接口**（成员函数的函数体），被单独编译成目标文件。
- **核心语法**：`cpp // StanfordID.h —— 接口 class StanfordID { public: StanfordID(std::string name, std::string sunet, int idNumber); std::string getName(); private: std::string name; }; cpp // StanfordID.cpp —— 实现 #include "StanfordID.h" StanfordID::StanfordID(std::string name, std::string sunet, int idNumber) : name{name}, sunet{sunet} {} std::string StanfordID::getName() { return this->name; }`
- **设计意图与最佳实践**：`.cpp` 里定义成员函数时必须用 `类名::` 作为作用域（就像 `std::` 之于标准库），告诉编译器"这个函数属于哪个类"。**注意**：`class StanfordID {...};` 结尾要加分号；`StanfordID::StanfordID(...)` 中 `::` 前面的名字是类、后面的名字是构造器名。

4. 构造函数（Constructor）

- **定义与目的**：构造函数在对象创建时初始化其状态（成员变量）。课堂例子 `StanfordID` 的对象需要 `name`、`sunet`、`idNumber` 三个字段，构造函数负责把它们设好。
- **核心语法**（语法就是类名）：`cpp // 带参构造器（.cpp 中实现） StanfordID::StanfordID(std::string name, std::string sunet, int idNumber) : name{name}, sunet{sunet}, idNumber{idNumber} {}`
- **设计意图与最佳实践**：
- **成员初始化列表**（`name{name}`，注意花括号）：直接初始化成员，是首选方式（避免先默认构造再赋值）。
- **构造器重载**：编译器根据实参个数/类型自动选择调用哪一个（`StanfordID s1;` 调默认构造器，`StanfordID s2{"a","b",1};` 调带参构造器）。
- 构造函数体内还可以做**参数校验**（例如 `if (idNumber > 0) ...`），保证对象一出生就是合法的。

5. this 指针

- **定义与目的**：`this` 是指向"当前对象"的指针，用于在参数名与成员名相同时消除歧义。
- **核心语法**：`cpp void StanfordID::setName(std::string name) { this->name = name; // 左边是成员，右边是参数 }`

- **设计意图与最佳实践**：当参数和成员同名时，`name = name;` 只是"参数赋给自己"，成员根本没被修改——这是经典 bug。写 `this->name` 明确"我要的是这个对象的 name 成员"。

6. Getter / Setter

- **定义与目的**：私有字段不直接暴露，通过 getter（读）和 setter（写）受控访问；setter 内部可以做校验（如 `setID` 拒绝负数）。
- **核心语法**：`cpp std::string StanfordID::getName() { return this->name; } void StanfordID::setID(int idNumber) { if (idNumber >= 0) { this->idNumber = idNumber; } }`
- **设计意图与最佳实践**：把"数据的合法性规则"收进 setter 一处，所有调用方自动受益。现代 C++ 还要求只读 getter 标记为 `const`（见 Lecture 9 的 const 正确性）。

7. 析构函数（Destructor）

- **定义与目的**：对象生命周期结束时（离开作用域）**自动**调用，负责释放动态分配的资源（`delete [] my_array;`）。不写也可以——只要类里没有 `new` 出来的资源，编译器会隐式生成析构函数。
- **核心语法**：`cpp StanfordID::~StanfordID() { // free/deallocate any data here（本例无动态资源，可为空） }`
- **设计意图与最佳实践**：析构函数**不能手动调用**（除非极特殊情况），它在对象离开作用域时自动触发。凡是构造器里 `new` 过的，析构函数里必须对应 `delete`（这引出了后面的"三/五法则"与 RAII 思想）。

8. 类型别名（Type Aliasing）

- **定义与目的**：为类型起一个同义标识符，提高可读性、便于统一修改。
- **核心语法**（C++11 起用 `using`，等价于 C++98 的 `typedef`）：`cpp using String = std::string; // 之后 String 就是 std::string String name;`
- **设计意图与最佳实践**：在类内部 `using String = std::string;` 后，整个类可以统一用短名，将来想换成 `std::u8string` 只改一行。`using` 比 `typedef` 更清晰且支持模板别名。

9. 继承（Inheritance）：public / protected / private

- **定义与目的**：动态多态（不同对象共用同一接口）与可扩展性（通过子类为基类增加特定属性）。例如 `Circle : public Shape`、"Player is an Entity"。
- **核心语法**：`cpp class Circle : public Shape { ... }; // is-a 关系 class B : protected A { ... }; // 接口只对子类可见 class B : private A { ... }; // 默认！接口只对本类可见`
- **设计意图与最佳实践**：三种继承方式决定基类成员的访问级别如何"传递"：

基类成员	`class B: public A`	`class B: protected A`	`class B: private A`（默认）
public	public	protected	private
protected	protected	protected	private

基类成员	`class B: public A`	`class B: protected A`	`class B: private A`（默认）
private	不可访问	不可访问	不可访问

经典判断（课堂 Pop Quiz）：实现 `class MyStack : ______ MyVector` 时应选 **private**——因为用户不该用 vector 的 `insert` 破坏栈的约束，子类也不需要 vector 的接口；而 `Player : public Entity` 必须是 **public**，因为"玩家真的是一个实体"，必须完整暴露实体的公共接口。**注意：class 的继承默认是 private**（`class B : A` 等价于 `class B : private A`），这是最常见的错误之一。

10. 虚函数、纯虚函数与 vtable

- **定义与目的**：`virtual` 开启**动态分派**——通过基类指针/引用调用时，实际执行的是对象真实类型（运行时类型）的版本。`virtual` 的机制是给对象附加一个**vpointer**，指向一张**vtable**（虚函数表），表里记录了每个虚函数该调用哪个实现。
- **核心语法**：`cpp class Shape { public: virtual double area() const = 0; // 纯虚函数：无默认实现 }; class Circle : public Shape { public: double area() const override { return 3.14 * _radius * _radius; } };`
- **设计意图与最佳实践**：纯虚函数（`= 0`）在基类"声明但不在基类实现"，强制子类必须覆盖；含纯虚函数的类是**抽象类**，不能实例化。`override` 关键字让编译器帮你检查"是否真的覆盖了基类的虚函数"，防止拼写错误静默地创建新函数。多态基类还应把**析构函数**声明为 `virtual`（见陷阱部分）。

11. 菱形继承（Diamond Problem）与虚继承

- **定义与目的**：当 `B`、`C` 都继承 `A`，而 `D` 同时继承 `B` 和 `C` 时，`D` 会得到**两份** `A` 的子对象（`D` 里有两个 `hello()`），`obj.hello()` 产生歧义——这就是菱形问题。
- **核心语法**：`cpp class B : virtual public A { ... }; // 虚继承：共享一份 A class C : virtual public A { ... }; class D : public B, public C { ... }; // D 中只有一份 A`
- **设计意图与最佳实践**：虚继承让 `B`、`C` **共享同一份** `A` 子对象，`D obj; obj.hello();` 不再歧义。幻灯片给的定义："Virtual — existing in essence, but not literally"——`virtual` 意味着创建 vtable、意味着把静态类型判断推迟到运行时。菱形继承在实际工程中应尽量避免（接口设计复杂、布局开销大），但理解它能帮助理解多重继承的本质。

代码示例与逐步解说（核心）

示例 1：为什么需要 class——struct 的无力（C++11）

代码：

```
// C++11
#include <iostream>
#include <string>

struct StanfordID {
    std::string name;
    std::string sunet;
    int idNumber;   // 所有字段默认 public
};

int main() {
    StanfordID s;
    s.name = "Preston Seay";
    s.sunet = "pseay";
    s.idNumber = 12345;
    s.idNumber = -12345;  // 💀 用户可以直接破坏数据
    std::cout << s.name << " " << s.idNumber << "\n";
}
```

代码做什么：构造一个 `StanfordID` 结构体并逐字段赋值，然后演示用户可以随意把 `idNumber` 改成负数——结构体对数据毫无保护。

特性机制解说：`struct` 的成员默认是 `public`，编译器对成员访问不做任何检查，`s.idNumber = -12345;` 与 `s.name = ...` 完全等价。这正是"没有访问控制"的体现：数据与操作它的规则没有绑定在一起，任何代码都能绕过约束。类的引入就是为了把 `idNumber >= 0` 这类"规则"与数据本身封装起来。

示例 2：完整的 StanfordID 类——封装、构造器重载、this、getter/setter、析构（C++11）

代码：

```cpp
// C++11
#include <iostream>
#include <string>

class StanfordID {
private:
    std::string name;
    std::string sunet;
    int idNumber;

public:
    // 默认构造器（重载之一）
    StanfordID()
        : name{"John Appleseed"}, sunet{"jappleseed"}, idNumber{1} {}

    // 带参构造器（成员初始化列表 + 参数校验）
    StanfordID(std::string name, std::string sunet, int idNumber)
        : name{name}, sunet{sunet}, idNumber{idNumber} {
        if (idNumber < 0) this->idNumber = 0;   // this-> 区分参数与成员
    }

    // getter
    std::string getName() const { return name; }
    std::string getSunet() const { return sunet; }
    int getID() const { return idNumber; }

    // setter（带校验）
    void setID(int idNumber) {
        if (idNumber >= 0) this->idNumber = idNumber;
    }

    // 析构函数：本类没有 new 资源，可为空
    ~StanfordID() {}
};

int main() {
    StanfordID defaultStudent;                    // 调用默认构造器
    StanfordID s{"Preston Seay", "pseay", 12345}; // 调用带参构造器
    s.setID(-1);        // 被拒绝：负数无效
    s.setID(999);       // 生效
    std::cout << s.getName() << " " << s.getSunet()
              << " " << s.getID() << "\n";
    std::cout << defaultStudent.getName() << "\n";
    // 输出：Preston Seay pseay 999
    //       John Appleseed
}
```

代码做什么：先构造默认学生与带参学生；`setID(-1)` 被校验逻辑拒绝，`setID(999)` 生效；最后打印两人信息。对象 `defaultStudent` 与 `s` 在 `main` 结束离开作用域时自动调用析构函数。

特性机制解说： – **构造器重载**：`StanfordID defaultStudent;` 与 `StanfordID s{...};` 实参个数不同，编译器在重载决议（overload resolution）中选出对应版本。 – **成员初始化列表**：`name{name}` 中，花括号

左侧是成员、右侧是参数。成员在进入函数体**之前**就已初始化（直接构造，而非先默认构造再赋值），这正是它优于 `name = name;` 的原因。 – **this 指针**：`this->idNumber = idNumber;` 中 `this` 是 `StanfordID*`，指向"正在被调用的那个对象"。`s.setID(999)` 时 `this` 指向 `s`，于是改写的是 `s` 的成员。成员函数调用其实等价于 `setID(&s, 999)`——`this` 是隐藏的第一个参数。 – **析构函数**：`~StanfordID() {}` 没有显式调用点，`main` 结束时两个对象自动析构。

示例 3：Shape 抽象基类与多态——纯虚函数、override、动态分派（C++11）

代码：

```
// C++11
#include <iostream>

class Shape {
public:
    virtual double area() const = 0;  // 纯虚函数：抽象类
    virtual ~Shape() = default;       // 多态基类必须虚析构
};

class Circle : public Shape {
private:
    double _radius;
public:
    explicit Circle(double radius) : _radius{radius} {}
    double area() const override { return 3.14 * _radius * _radius; }
};

class Rectangle : public Shape {
private:
    double _width, _height;
public:
    Rectangle(double w, double h) : _width{w}, _height{h} {}
    double area() const override { return _width * _height; }
};

int main() {
    Circle c{2.0};
    Rectangle r{3.0, 4.0};
    const Shape* shapes[] = {&c, &r};   // 基类指针数组
    for (const Shape* s : shapes) {
        std::cout << s->area() << "\n"; // 输出 12.56 与 12
    }
    // Shape s;  // ❌ 抽象类不能实例化
}
```

代码做什么：定义抽象基类 `Shape`（纯虚 `area()`），`Circle` 与 `Rectangle` 继承并覆盖 `area()`；`main` 用基类指针数组统一调用，每次调用都会执行**真实类型**的 `area`。

特性机制解说： – **纯虚函数**：`= 0` 告诉编译器"基类只声明接口，不提供实现"，同时使 `Shape` 成为抽象类，`Shape s;` 无法编译。 – **动态分派**：`s->area()` 的静态类型是 `const Shape*`，但运行时通过对象的

vpointer → vtable 查到应该调 `Circle::area()` 还是 `Rectangle::area()`。`virtual` 关键字给 `Shape` 的对象悄悄附加了一个 vtable 指针，这使对象内存变大、调用多一次间接跳转——这是 C++ 中虚函数**不是默认开启**的原因（"opt in"）。 – **override**：`double area() const override` 若与基类签名不匹配（比如漏写 `const`），编译器直接报错而不是静默新建函数。

示例 4：菱形继承与虚继承的修复（C++11）

代码：

```
// C++11
#include <iostream>

class A {
public:
    void hello() const { std::cout << "hello from A\n"; }
};

class B : virtual public A {};   // 虚继承
class C : virtual public A {};   // 虚继承

class D : public B, public C {}; // D 只共享一份 A

int main() {
    D obj;
    obj.hello();   // ✅ 不再歧义
    obj.B::hello();// 仍可显式选择路径
    obj.C::hello();
}
```

代码做什么：`B`、`C` 通过 `virtual public A` 继承 `A`，`D` 同时继承 `B` 与 `C`，最终 `D` 里只有**一份** `A` 子对象，`obj.hello()` 直接可用。

特性机制解说：如果不加 `virtual`，`D` 会包含两份 `A` 子对象（一份经 `B`、一份经 `C`），`obj.hello()` 就歧义到必须写 `obj.B::hello()`。虚继承让 `B`、`C` 共享同一个 `A` 子对象——编译器会把共享基类子对象放到派生对象末尾，并通过偏移量访问，实现"一份拷贝"。代价是对象布局更复杂、访问稍慢，所以只在确实需要时才用虚继承。

与旧标准（如 C++98）的对比

本讲新特性（C++11+）	C++98 的做法	新特性优势
成员初始化列表用花括号 `: name{name}`	圆括号 `: name(name)`	统一初始化（uniform initialization）防止窄化转换（如 `double` 隐式截断为 `int` 会编译报错）
`using String = std::string;`	`typedef std::string String;`	语法更直观、支持模板别名（`template<class T> using Vec = std::vector<T>;`）

本讲新特性（C++11+）	C++98 的做法	新特性优势
`= delete` / `= default`（如 `~Shape() = default`）	把拷贝构造/拷贝赋值声明为 private 且不实现来"禁用"	意图清晰、报错信息友好；`= default` 保留编译器生成的版本
`override` 关键字	没有；靠命名约定（如 `area()`）	编译器校验覆盖关系，拼写/签名错误立即报错
类内成员默认初始化 `int idNumber{1};`	只能在构造器初始化列表里写	每个构造器自动获得默认值，减少重复与漏初始化
`nullptr`、基于范围的 for、`auto`	`NULL` / `0`、手写循环	类型安全、代码更短（示例 3 中的 `for (const Shape* s : shapes)` 在 C++98 要写迭代器或下标循环）

关键要点

- **`class` 默认 private，`struct` 默认 public**；封装是类的灵魂——数据私有、接口公开、规则收进 setter。
- **构造函数用成员初始化列表**（花括号）初始化成员，比在函数体内赋值更高效、更安全，还能做参数校验。
- **`this` 是隐藏的"当前对象指针"**；参数与成员同名时必须 `this->member = param;`。
- **`class B : A` 默认是 private 继承**；表达 is-a 关系必须显式 `class B : public A`，否则基类公共接口对外不可见。
- **多态基类要写 `virtual` 析构函数，纯虚函数（`= 0`）制造抽象类**；菱形继承用 `virtual public` 虚继承修复歧义。

常见陷阱与注意事项

- **`name = name;` 参数遮蔽成员**：不带 `this->` 时只是"参数赋给参数"，成员从未被修改，且编译器通常不报错——初始化列表或 `this->` 是正解。
- **默认私有继承**：`class MyStack : MyVector` 会让用户无法调用 vector 的公共接口；课堂答案是 `private`（栈不该暴露 vector），但 is-a 场景（`Player : public Entity`）忘写 `public` 就是 bug。
- **菱形继承不加 `virtual`**：`D` 得到两份 `A`，`obj.hello()` 歧义无法编译。
- **多态基类析构函数不是 `virtual`**：`Shape* p = new Circle(...); delete p;` 只调用 `Shape::~Shape()`，`Circle` 的资源泄漏——未定义行为。
- **纯虚函数签名不一致**：子类 `area()` 忘了 `const` 或参数不同，会静默"新建"函数而不覆盖；用 `override` 让编译器抓住它。

关联作业提示

本讲对应 **A3: Make a Class!**（Lecture 7–8 的知识）。作业要求你在 `class.h` / `class.cpp` 中自创一个类，硬性要求包括：①带一个或多个参数的构造器；②默认（无参）构造器——即**构造器重载**；③至少一个 private 字段；④至少一个 **private 成员函数**（相当于"引擎盖下的实现细节"，不暴露给用户）；⑤至少一个 public getter（建议标 `const`）；⑥至少一个 public setter。

运用本讲知识：用 `class.h` 写声明（`private:` 字段 + `public:` 接口）、`class.cpp` 用 `类名::` 作用域写实现；构造器用**成员初始化列表**并在体内做参数校验；setter 里做约束检查（像 `setID` 拒绝负数）；getter 用 `this->` 或直接返回成员；`sandbox.cpp` 里用统一初始化构造实例。加分项是把它写成**类模板**（`template <typename T>`）——那就必须按 Lecture 9 的规则去掉 `class.cpp` 的编译并让 `.h` 在底部 `#include` 实现文件。`short_answer.txt` 的两道题（什么是 const 正确性、你的类是否 const 正确）请参考 Lecture 9 的 const 成员函数部分作答。

Lecture 8 (Week 4 – Thursday): 可选：继承练习 (Optional: Inheritance Practice)

概述

本讲是**可选**的继承复习课，从"类背后到底长什么样"讲起：对比 Python 与 C++ 的对象内存布局，揭示 `this` 作为隐藏参数传递的机制，然后系统梳理继承（is-a 关系）、三种访问修饰符、**对象切片（object slicing）**、基类指针多态、虚函数与 vtable/vpointer、纯虚函数与抽象类，最后以"组合优于继承"收尾。学习目标是把 Lecture 7 的类知识落实到"运行时行为"层面：为什么 `std::vector<Entity>` 存不下子类、为什么必须用 `Entity*` + `virtual` 才能写出正确的游戏循环。配套练习是 `exercise1.h`（BankAccount 类复习）与 `exercise2.h`（`Stack : private std::vector<int>` 私有继承练习）。

核心特性与语法详解

1. 类的内存布局：C++ 只存数据，Python 存"元信息"

- **定义与目的**：理解 `class Point { int x; int y; };` 的对象在内存里到底占什么。C++ 对象只按声明顺序存放数据成员（`int x` 紧跟 `int y`）；类型检查全部发生在**编译期**，运行时不需要类型信息。
- **核心语法**：`cpp Point p{1, 2}; // 内存里只有两个 int：x=1, y=2`
- **设计意图与最佳实践**：对比 Python，同一个 `Point` 对象要存 refcount、type 指针、`__dict__`（指向 `"_x"` / `"_y"` 两个字符串对象和两个 int 对象）…… 一长串指针。这就是 C++ 更省内存、更高效的原因之一——"你为不需要的灵活性付了代价"。**函数本身不存进对象**：所有对象的成员函数代码都放在内存的 Text（代码）区，对象里只有数据。

2. this 的机制：隐藏的第一个参数

- **定义与目的**：成员函数如何知道"自己在为哪个对象工作"？答案是 `this`——编译器把成员调用改写为普通函数调用，把对象地址作为隐藏参数传进去。
- **核心语法**：`cpp int Point::getX() { return this->x; } // 编译器把它看成： int Point_getX(Point* this) { return this->x; } // 调用点： int x = p.getX(); // → int x = Point_getX(&p);`
- **设计意图与最佳实践**：
- `return x;` 与 `return this->x;` **完全等价**（成员名自动通过 `this` 解析）。
- 但 `void Point::setX(int x) { x = x; }` 与 `this->x = x;` **不等价**——参数 `x` 遮蔽了成员 `x`，前者是"自己赋给自己"。
- 在 **const 成员函数**里，`this` 的类型变成 `const Point*`，所以无法修改成员（见 Lecture 9）。

3. 继承：is-a 关系与公共基类

- **定义与目的**：继承让一个类复用另一个类的成员。核心心智模型是 **is-a**：`std::ifstream` 是一个 `std::istream`，是一个 `std::ios`；`Player` 是一个 `Entity`。课堂用游戏对象举例：Player/Projectile/Weapon/Tree/NPC 全都"是"Entity——有位置、有 hitbox、有 update/render。
- **核心语法**：`cpp class Entity { /* 位置、hitbox、update、render */ }; class Player : public Entity { double hitpoints; public: void damage(double hp); }; class Projectile : public Entity { double vx, vy, vz; }; class Actor : public Entity { double hitpoints; public: void damage(double hp); }; class NPC : public Actor {}; // 多层继承：NPC 是 Actor，也是 Entity`
- **设计意图与最佳实践**：把公共部分（位置、hitbox、update/render）上提到基类，消除五个类里的重复代码；再抽出 `Actor`（有血量、会受伤）形成中间层。这样"给所有实体加一个 `overlapsWith(const Entity&)`"只需要在 `Entity` 里写一次——`player.overlapsWith(bullet)` 即可，任何实体都能互查。继承树定义的是一组 is-a 命题："A Weapon is an Entity"、"An NPC is an Actor, and is also an Entity"。

4. 访问修饰符：默认私有继承、protected

- **定义与目的**：`class Player : Entity` 默认是 **private 继承**——`Entity` 的 public 成员（如 `overlapsWith`）在 `Player` 里变成 private，外部调 `player.overlapsWith(bullet)` 会报"inaccessible"。只有 `public` 继承才真正表达 is-a。
- **核心语法**：`cpp class Player : /* private */ Entity { ... }; // 默认：public 成员变成 private class Player : public Entity { ... }; // ✅ is-a：public 仍是 public class Projectile : public Entity { public: void move() { x += vx; y += vy; z += vz; } // 需要 x,y,z 可见 ⟹ protected };`
- **设计意图与最佳实践**：`protected` 成员"对子类可见、对外部不可见"——想让 `Projectile` 直接读写 `x,y,z` 就得在 `Entity` 里把它们标为 `protected`（注意类成员默认 `private`）。三者取舍：`public`

给用户，`protected` 给子类，`private` 只给自己。**只有 public 继承是 is-a**；`private` 继承是"实现复用"（见 `exercise2.h`：`Stack : private std::vector<int>`——栈复用了 vector 的实现，但绝不向用户暴露 vector 的 `insert` 等接口）。

5. 对象切片（Object Slicing）

- **定义与目的**：`std::vector<Entity>` 的每个元素都是 `Entity`；把 `Player` **按值**放进去时，`Player` 多出来的部分（`hitpoints` 等）被丢弃——**对象切片**。切片只发生在**拷贝**时；指针/引用不会切片。
- **核心语法**：`cpp std::vector<Entity> bad{player, tree, bullet}; // ❌ 全部被切成 Entity std::vector<Entity*> good{&player, &tree, &bullet}; // ✅ 指针保留真实类型`
- **设计意图与最佳实践**：切片后容器里全是纯 `Entity`，循环调用 `entity.update()` 永远执行 `Entity::update()`（空实现），"游戏什么都没发生"。改用 `Entity*` 数组后，指针仍指向完整的 `Player` / `Tree` / `Projectile` 对象，才谈得上多态。

6. 虚函数与动态分派：vtable / vpointer

- **定义与目的**：`Entity*` 只告诉我们"编译期类型是 Entity"，而对象**运行时类型**可能是 Player 或 Projectile。虚函数通过 vtable 实现**动态分派**：按对象真实类型决定调用哪个 `update()`。
- **核心语法**：`cpp class Entity { public: virtual void update() {} // virtual ⟹ 动态分派 virtual void render() {} }; class Projectile : public Entity { public: void update() override {} // override 非必需但强烈建议 };`
- **设计意图与最佳实践**：`virtual` 会给每个对象附加一个 **vpointer**，指向该类的 **vtable**——表中记录了"这个类的 `update` 应该调哪个函数"。`p->update()` 的流程是：取 `p` 的 vpointer → 查 vtable → 跳到对应函数。C++ 中虚函数**不是默认开启**的，因为它有成本：对象内存变大（多一个指针）、调用多一次间接寻址；在量化金融等对纳秒敏感的场景甚至会刻意避免虚函数。`override` 让编译器校验覆盖正确性。

7. 纯虚函数与抽象类

- **定义与目的**：当基类"没有合理的默认实现"时（"Shape 的默认体积是多少？"），用纯虚函数把实现责任下放给子类。
- **核心语法**：`cpp class Shape { public: virtual double volume() = 0; }; Shape s; // ❌ 抽象类不能实例化 class Box : public Shape { /* 实现 volume() */ }; Box b; // ✅ 覆盖全部纯虚函数后是具体类`
- **设计意图与最佳实践**：含一个及以上纯虚函数的类是**抽象类**，只能被继承；子类**覆盖所有**纯虚函数后才可实例化。这是"接口设计"的工具：强制所有子类提供统一接口。

8. 组合优于继承（Composition over Inheritance）

- **定义与目的**：继承树过深会变慢、难以推理（课堂展示了"继承失控"的梗图）。现代游戏引擎很少为每种对象类型建子类；**组合**（has-a）往往更灵活。

- **核心语法**：`cpp class Car : public Engine, public SteeringWheel, public Brakes { /* 不对劲 */ }; class Car { Engine engine; SteeringWheel wheel; Brakes brakes; }; // ✅ has-a`
- **设计意图与最佳实践**："A car **is** an engine"是错的，"A car **has** an engine"才对。组合 + 继承可结合使用（`Car` 拥有 `Engine*`，而 `Engine` 下再挂 `CombustionEngine` / `ElectricEngine` 继承树）；想深入可查 **PIMPL 惯用法**。

代码示例与逐步解说（核心）

示例 1：Point 类与 this 的机制（C++11）

代码：

```
// C++11
#include <iostream>

class Point {
private:
    int x, y;
public:
    Point(int x, int y) : x{x}, y{y} {}
    int getX() const { return this->x; }      // this 是 const Point*
    void setX(int x) { this->x = x; }         // 参数遮蔽成员，必须 this->
};

int main() {
    Point p{1, 2};
    p.setX(42);
    std::cout << p.getX() << "\n";            // 42
    std::cout << sizeof(Point) << "\n";       // 通常 8：只有两个 int
}
```

代码做什么：构造 `Point`，`setX(42)` 修改成员，`getX()` 读回；`sizeof(Point)` 说明对象里只有数据、没有函数指针等元信息。

特性机制解说： – 内存中 `p` 就是连续的两个 `int`（`x=1, y=2` 之后变成 `x=42, y=2`）。成员函数代码存放在 Text 区，所有 `Point` 对象共享同一份代码。 – `p.setX(42)` 被编译器改写成 `Point_setX(&p, 42)`：`this` 就是隐藏的第一个参数，类型 `Point*`。`p.getX()` 则是 `Point_getX(&p)`，因为在 const 成员函数中，`this` 的类型是 `const Point*`，所以函数体内不能写 `this->x = ...`。 – 对比 Python：`p.getX()` 等价于 `Point.getX(p)`——Python 的 `self` 是**显式**参数；C++ 的 `this` 是**隐式**的，二者机制同源。区别是 Python 对象还背着 refcount/type/`__dict__` 一整套运行时元信息，C++ 则把这些开销全部移到编译期。

示例 2：is-a 继承与 protected（游戏实体，C++11）

代码：

```cpp
// C++11
#include <iostream>
#include <vector>

class Entity {
protected:
    double x = 0, y = 0, z = 0;   // protected：子类可见、外部不可见
public:
    virtual void update() {}
    virtual void render() {}
};

class Player : public Entity {
private:
    double hitpoints = 100;
public:
    void damage(double hp) { hitpoints -= hp; }
    void update() override {
        x += 1.0;                // ✅ 子类可以读写 protected 的 x
        std::cout << "Player moves\n";
    }
};

class Projectile : public Entity {
public:
    void update() override { std::cout << "Projectile flies\n"; }
};

int main() {
    Player p;
    Projectile b;
    std::vector<Entity*> entities{&p, &b};     // 基类指针，不切片
    for (Entity* e : entities) e->update();    // Player moves / Projectile flies
}
```

代码做什么：`Player` 与 `Projectile` 公开继承 `Entity`；`Player::update` 通过 `protected` 的 `x` 移动自己；`main` 用 `Entity*` 容器统一驱动每个实体的 `update`。

特性机制解说：– **public 继承**：`Player` 完整继承 `Entity` 的公共接口（`update`/`render`），所以 `Entity* e = &p;` 合法——is-a 成立的编译期证据。 – **protected**：`x, y, z` 标 `protected` 后，`Player` 内部可访问，但 `main` 里写 `p.x = 1;` 会编译失败。 – **指针不切片**：`&p` 指向完整的 `Player` 对象（含 `hitpoints` 与 `x`），`vector<Entity*>` 只存地址，对象本体完好。

示例 3：对象切片演示与修复（C++11）

代码：

```cpp
// C++11
#include <iostream>
#include <string>
#include <vector>

class Entity {
public:
    virtual std::string kind() const { return "Entity"; }
};

class Player : public Entity {
public:
    std::string kind() const override { return "Player"; }
};

class Tree : public Entity {
public:
    std::string kind() const override { return "Tree"; }
};

int main() {
    Player p;
    Tree t;

    std::vector<Entity> byValue{p, t};          // ❌ 切片：只拷走 Entity 部分
    for (const Entity& e : byValue)
        std::cout << e.kind() << " ";           // 输出: Entity Entity

    std::vector<Entity*> byPtr{&p, &t};         // ✅ 指针保留完整对象
    for (const Entity* e : byPtr)
        std::cout << e->kind() << " ";          // 输出: Player Tree
    std::cout << "\n";
}
```

代码做什么：同一个 `Player` 与 `Tree`，分别按值存入 `vector<Entity>`、按指针存入 `vector<Entity*>`，观察调用 `kind()` 的结果差异。

特性机制解说： – **切片机制**：`byValue{p, t}` 调用的是 `Entity` 的拷贝构造——编译器把 `Player` 的 `Entity` 基类子对象"切"出来拷贝，`Player` 独有的部分直接丢弃。存进去的元素是货真价实的 `Entity`，其 vpointer 指向 `Entity` 的 vtable，所以 `kind()` 永远是 `Entity::kind()`。 – **指针方案**：`&p` 是 `Player*`，隐式转换为 `Entity*` 只发生**指针类型转换**，对象本体不动；`e->kind()` 通过对象真实 vpointer 分派到 `Player::kind()`。**切片只发生在拷贝时**——传参按值、容器按值存储、返回按值都会切片；传引用/指针则安全。

示例 4：虚函数与 vtable 动态分派（C++11）

代码：

```
// C++11
#include <iostream>
#include <vector>

class Entity {
public:
    virtual void update() {}   // 默认实现：什么都不做
    virtual void render() {}
};

class Player : public Entity {
public:
    void update() override { std::cout << "Player update\n"; }
    void render() override { std::cout << "Player render\n"; }
};

class Projectile : public Entity {
public:
    void update() override { std::cout << "Projectile update\n"; }
    void render() override { std::cout << "Projectile render\n"; }
};

int main() {
    Player p;
    Projectile b;
    std::vector<Entity*> entities{&p, &b};
    for (Entity* ent : entities) {   // 游戏主循环的雏形
        ent->update();
        ent->render();
    }
}
```

代码做什么：模拟课堂的"游戏主循环"——遍历 `Entity*` 容器，每帧对每个实体调用 `update()` 与 `render()`。输出按真实类型分派：

```
Player update
Player render
Projectile update
Projectile render
```

特性机制解说： – 没有 `virtual` 时，`ent->update()` 在编译期就绑定为 `Entity::update()`（空实现）——这就是上一节"还是没工作"的原因。 – 加上 `virtual` 后，每个 `Player` 对象多一个 **vpointer**，指向 `Player` 的 **vtable**：`{update → Player::update, render → Player::render}`；`Projectile` 的 vtable 则是 `{update → Projectile::update, render → Projectile::render}`。`ent->update()` 的机器码是：`取 ent 的 vpointer → 查表偏移 → 间接调用`，这就是动态分派。 – **成本意识**：vpointer 让每个对象变大；查表是额外一次内存访问。这正是 C++ 要求"显式 opt-in"（写 `virtual`）而 Python/Java 默认全虚的原因。**编译期类型**（`Entity*`）与**运行时类型**（`Player`）在这里分道扬镳，虚函数是唯一让两者重新对齐的机制。

示例 5：纯虚函数与抽象类（C++11）

代码：

```
// C++11
#include <iostream>

class Shape {
public:
    virtual double volume() = 0;    // 纯虚："默认体积"没有意义
    virtual ~Shape() = default;
};

class Box : public Shape {
    double w, h, d;
public:
    Box(double w, double h, double d) : w{w}, h{h}, d{d} {}
    double volume() override { return w * h * d; }
};

class Sphere : public Shape {
    double r;
public:
    explicit Sphere(double r) : r{r} {}
    double volume() override { return 4.0 / 3.0 * 3.14 * r * r * r; }
};

int main() {
    Box b{2, 3, 4};
    Sphere s{1.0};
    Shape* shapes[] = {&b, &s};
    for (Shape* sh : shapes) std::cout << sh->volume() << "\n";  // 24 / 4.18667
    // Shape x;  // ❌ 抽象类无法实例化
}
```

代码做什么：`Shape::volume()` 用 `= 0` 声明为纯虚，`Box` 与 `Sphere` 各自实现；通过 `Shape*` 数组多态调用各自的 `volume`。

特性机制解说：– `= 0` 使 `Shape` 成为抽象类：编译器禁止 `Shape x;`（没有完整的 vtable 可用）。`Box` / `Sphere` 覆盖全部纯虚函数后 vtable 填满，成为**具体类**，可以实例化。 – 与普通虚函数的区别在于**默认实现是否存在**：`virtual void update() {}` 表示"默认什么都不做"（可覆盖可不覆盖）；`virtual void volume() = 0` 表示"本类根本没有默认行为，子类必须实现"。这正对应课堂问题："What's the default volume of a Shape?"——没有，所以纯虚。

示例 6：组合优于继承（C++11）

代码：

```cpp
// C++11
#include <iostream>

// 错误直觉：class Car : public Engine, public SteeringWheel, public Brakes
// 正确做法：Car 拥有（has-a）这些部件
class Engine {
public:
    void start() const { std::cout << "engine started\n"; }
};
class SteeringWheel {};
class Brakes {};

class Car {
private:
    Engine engine;          // 组合：成员对象
    SteeringWheel wheel;
    Brakes brakes;
public:
    void start() const { engine.start(); }   // 委托给部件
};

int main() {
    Car c;
    c.start();              // engine started
}
```

代码做什么：用**成员对象**而非多重继承组装 `Car`；`Car::start()` 委托给内部的 `Engine::start()`。

特性机制解说： – "A car is an engine"是语义错误——组合把"拥有关系"编码成成员变量，天然可替换、可测试（换一个 `Engine` 子类即可换动力系统）。 – 组合 + 继承可以并用：`Car` 持有 `Engine*`，`Engine` 之下再有 `CombustionEngine` / `ElectricEngine` 的继承树，兼得灵活性与复用性（PIMPL 惯用法正是这种思想的体现）。

与旧标准（如 C++98）的对比

本讲新特性（C++11+）	C++98 的做法	新特性优势
`override` 关键字	没有；靠函数名一致	覆盖写错签名/拼写时编译器报错，而不是静默新建虚函数
类内默认成员初始化 `double x = 0;`	只能在每个构造器初始化列表写	所有构造器自动获得默认值，减少遗漏
基于范围的 for `for (Entity* e : entities)`	手写 `for (size_t i = 0; ...)` 或迭代器	更短、更不易出错（示例 3/4 均用到）
`= default` / `= delete`（如 `~Shape() = default`）	手写空函数体 / private 声明禁拷贝	明确表达意图，编译器生成更优代码

本讲新特性（C++11+）	C++98 的做法	新特性优势
`nullptr`	`NULL` / `0`	类型安全，不会与整数 `0` 混淆
`std::vector<Entity*> v{&p, &t};` 初始化列表构造	先默认构造再逐个 `push_back`	一步到位、更高效

注意：**虚函数、继承、vtable、纯虚函数等机制本身在 C++98 就存在**——本讲复习的多是"老机制"，C++11 主要改善的是**书写与校验**（`override`、范围 for、`= default`）。动态分派的概念模型（编译期类型 vs 运行时类型）在任何标准下都成立。

关键要点

- **C++ 对象只存数据、按声明顺序布局**；函数在 Text 区，类型检查在编译期——这是 C++ 比 Python 省内存的核心原因（Python 还要背 refcount/type/`__dict__`）。
- **`this` 是隐藏参数**：`p.getX()` → `Point_getX(&p)`；参数遮蔽成员时 `this->x = x;` 是唯一正解；const 成员函数里 `this` 是 `const Point*`。
- **对象切片只发生在拷贝时**：`vector<Entity>` 装不下 `Player`；要多态就用 `vector<Entity*>`（或引用、`unique_ptr`）。
- **`virtual` 开启动态分派**：vpointer → vtable → 按运行时类型调用；有内存与速度成本，所以 C++ 默认不开启（对比 Python 全虚）。
- **纯虚函数 = 抽象类（不可实例化）**；继承表达 is-a（默认却是 private，要显式 `public`）；能用组合就别滥用继承。

常见陷阱与注意事项

- **把派生类按值放进基类容器**：`std::vector<Entity> v{p, t};` 静默切片，运行时调用的是 `Entity::update()`（空实现）——不报错但行为全错，极难排查。
- **`class B : A` 忘记 `public`**：默认 private 继承，`player.overlapsWith(bullet)` 编译报 "inaccessible"；is-a 必须显式 `public`。同理，`protected` 只面向"子类设计者"，标得过多会削弱封装。
- **多态基类析构函数非 `virtual`**：`delete` 基类指针时只析构基类部分，派生类资源泄漏（未定义行为）；把 `~Shape()` 写成 `virtual` 或 `= default`。
- **`override` 写错签名**（漏 `const`、参数不同）：不报错地"新建"了一个函数，动态分派悄悄失效；让 `override` 关键字把关。
- **过度继承**：为每种对象建一个子类会让继承树失控；先想 "has-a 是否更合适"（组合优于继承）。

关联作业提示

本讲继续支撑 **A3: Make a Class!**。几个直接可用的点：

- **const 正确的 getter**：A3 要求 getter 标 `const`（`int getData() const;`）。参考课堂 `exercise1.h` 的 `double getBalance() const;`——只读查询不修改对象，就该进 const 接口（机制上 `this` 是 `const Point*`，见本讲示例 1）。
- **私有继承的直觉**：A3 若你想做"受限容器"类的思路，可参考 `exercise2.h` 的 `Stack : private std::vector<int>`——复用实现但不暴露 vector 的公共接口，这正是 Lecture 7 课堂 `MyStack : private MyVector` 的结论。
- **内存布局意识**：A3 的类建议只存必要的数据成员（简单类型 + `std::string` 等），体会"对象小而高效"的设计；构造函数用成员初始化列表保证对象一出生就合法。
- **面向对象设计**：A3 要求一个 **private 成员函数**——它是"引擎盖下的实现细节"（比如 `sanitize()`），不必暴露给用户；这正是封装思想的落地。
- 若选择加分项（把类写成**类模板**），请按 Lecture 9 的规则：`.h` 底部 `#include` 实现、去掉 `class.cpp` 的编译命令。

Lecture 9 (Week 5 – Tuesday): 类模板与 const 正确性 (Class Templates & Const Correctness)

概述

本讲把"类"提升到"类的工厂"层面：**类模板（class template）**让你写一份逻辑、为任意类型生成对应的类（`std::vector<int>`、`std::vector<std::string>` 背后是同一个 `template <typename T> class vector`），并讲解模板实例化机制、模板与类型的区别、非类型模板参数（`std::array`）以及模板实现的三个经典怪癖（`.h` 底部 include `.cpp`、`Vector<T>::` 语法、`typename` 与 `class` 等价）。后半讲是 **const 正确性（const correctness）**：const 成员函数、const 重载（`const T& at() const` 与 `T& at()`）、`const_cast` 与 `mutable`。学习目标是写出"既能被 const 对象安全使用、又能被普通对象高效修改"的通用容器类。本讲直接支撑作业 A4（Ispell，大量使用模板与 const 引用参数）。

核心特性与语法详解

1. 模板类：动机与定义

- **定义与目的**：没有模板时，为 `int`、`double`、`std::string` 各写一个 `IntVector` / `DoubleVector` / `StringVector` 是灾难性的重复；模板把"逻辑"与"类型"解耦——**逻辑写一次，类型当参数**。

- **核心语法**：`cpp template <typename T> // T 是"类型参数" class Vector { public: T& at(size_t index); void push_back(const T& elem); private: T* elems; // 所有"元素类型"处都用 T };`
- **设计意图与最佳实践**：把类里所有出现"元素类型"的地方（成员类型、参数、返回值）替换为 `T`。课堂讲了一段历史：STL 之前人们用预处理器宏 `#define GENERATE_VECTOR(MY_TYPE)` 生成类——语法笨拙、难以类型检查、忘了调用或调用两次都是灾难；模板让编译器**自己**完成代码生成，且带完整类型检查。`template` 声明不是代码，是"配方"。

2. 模板实例化：按需代码生成

- **定义与目的**：模板本身不产生任何代码；只有当你写下 `Vector<int> v;` 这种**实例化**时，编译器才为 `T = int` 生成一份具体的类代码。
- **核心语法**：`cpp Vector<int> intVec; // 编译器生成 IntVector 版代码 Vector<double> doubleVec; // 生成 DoubleVector 版代码 Vector<std::string> strVec; // 生成 StringVector 版代码 Vector<Vector<int>> vecVec; // 嵌套模板：元素类型本身是 Vector<int>`
- **设计意图与最佳实践**：模板像工厂：输入 `int` 产出 `Vector<int>`，输入 `std::string` 产出 `Vector<std::string>`。实例化是**惰性**的——只有用到的类型才会生成代码，未实例化的模板不产生任何目标代码。

3. 模板 vs 类型：两个完全不同的类型

- **定义与目的**：`template <typename T> class Vector` 是**模板**，不是类型；`Vector<std::string>` 才是**类型**（也叫模板实例化）。
- **核心语法**：`cpp void foo(std::vector<int> v); std::vector<double> v; foo(v); // ❌ 编译错误：没有从 vector<double> 到 vector<int> 的转换`
- **设计意图与最佳实践**：`Vector<int>` 与 `Vector<double>` 是**完全不同的两个类型**（编译期与运行期皆不同），不能互相赋值/转换。对比 Java：`ArrayList<Integer>` 与 `ArrayList<Double>` 在运行时是同一个类型（类型擦除）——C++ 的模板则保留完整类型信息，这也是 C++ 模板能极致优化的原因。

4. 非类型模板参数与 std::array

- **定义与目的**：模板参数不一定是类型，还可以是**编译期常量**（`size_t`、`bool`、`int`……），让"大小"成为类型的一部分。
- **核心语法**：`cpp template <size_t N> class SizeTemplate {}; // N 是编译期值 template <typename T, std::size_t N> struct std::array; // 标准库示例 std::array<std::string, 5> arr; // 恰好 5 个 string`
- **设计意图与最佳实践**：`std::array<T, N>` 的大小**烘焙进类型**，编译器精确知道它占多少字节，因此可以**栈上分配**、完全避免堆分配（对比 `std::vector` 的堆缓冲）。适合固定大小、性能敏感的场景（游戏、嵌入式）。

5. 模板实现的三个怪癖（Quirks）

- **定义与目的**：模板代码生成发生在编译器内部，导致实现组织方式与非模板类完全不同。课堂给了三条"👻 怪癖"：
- **(1) `.cpp` 实现必须原样复制 `template <typename T>`，并且类名要写 `Vector<T>`**：`cpp // Vector.cpp —— 漏掉 template 前缀或写成 Vector::at 都会报错 template <typename T> T& Vector<T>::at(size_t i) { /* 实现... */ } // 不是 Vector::at!`
- **(2) 模板的 `.h` 必须在文件底部 include `.cpp`**：`cpp // Vector.h —— 非模板类是 .cpp include .h; 模板类反过来 template <typename T> class Vector { public: T& at(size_t i); }; #include "Vector.cpp" // 底部 include 实现`
- **(3) `typename` 与 `class` 完全等价**：`template <typename T>` 与 `template <class T>` 写法相同含义（历史原因：C++98 只有 `class`，后来才引入 `typename`）。`template <class K, typename V>` 混用也可以。
- **设计意图与最佳实践**：理解这些怪癖的"为什么"不必过深（涉及编译器/链接器的模板实现方式），但必须遵守——否则就是链接错误 `undefined reference to ...`。也有替代方案（如显式实例化、把实现放 `.tpp` 文件），课堂建议课后提问。

6. const 正确性与 const 成员函数

- **定义与目的**：`void printVec(const Vector<int>& v)` 里 `v` 是 const 引用，却调 `v.size()`、`v.at(i)` 报错——因为 `size()` 没标 `const`，编译器无法保证它不修改 `v`。**const 成员函数**向编译器承诺"本函数不修改 this 所指对象"。
- **核心语法**：`cpp class Vector { public: size_t size() const; // 声明处加 const }; // 实现处也必须加 const（否则是不同函数）： template <class T> size_t Vector<T>::size() const { return logical_size; }`
- **设计意图与最佳实践**：const 方法的本质是**把 `this` 的类型变成 `const Vector<T>*`**——于是函数体内任何写成员的操作都直接编译报错（如 `this->logical_size = 106;` 报 "cannot assign ... within const member function"）。规则：**不修改对象的成员函数一律标 const**，这样 const 对象与普通对象都能用。const 对象只能访问 const 接口。

7. const 重载（const overloading）

- **定义与目的**：`at()` 既要让 const 用户"能读"，又要让普通用户"能写"——一个签名满足不了，于是定义**两个重载**，由对象的 const 性自动选择。
- **核心语法**：`cpp class Vector { public: const T& at(size_t index) const; // const 对象调用：只读 T& at(size_t index); // 普通对象调用：可读写 };`
- **设计意图与最佳实践**：单独用 `T& at(size_t) const` 是错的（const 用户拿到非 const 引用后 `v.at(0) = 42;` 能改掉 const 对象）；单独用 `const T& at(size_t) const` 也错（普通用户无法写元素）。两个版本各司其职；实现几乎相同（都是 `return elems[index];`），但 C++ 用 const 性参与重载决议，自动匹配。

8. const_cast

- **定义与目的**：`const_cast<target_type>(expr)` 用来"剥掉"或"加上" const 性。最经典的合法用途：在 const 重载里**委托**——const 版本调用非 const 版本，避免复制粘贴逻辑。
- **核心语法**：`cpp template <typename T> const T& Vector<T>::findElement(const T& value) const { return const_cast<Vector<T>&>(*this).findElement(value); // 剥掉 const 非 const 版本 }` 拆解：`*this` 是 `const Vector<T>&`；`const_cast<Vector<T>&>` 把它变成非 const 引用；于是 `.findElement(value)` 解析到**非 const 重载**（编译器按静态类型选重载）。
- **设计意图与最佳实践**：课堂原话——"short answer: just about never"（几乎从不）。`const_cast` 是在对编译器说"别担心，我兜底"，如果对象**本来就是 const**，通过 const_cast 修改它属于**未定义行为**。想改数据就别声明 const；它只在"接口必须 const、实现确实不修改"的委托场景有正当用途。

9. mutable 关键字

- **定义与目的**：比 const_cast 更细粒度的"豁免"：把**个别成员**标为 `mutable`，即使对象是 const，这些成员仍可修改（const_cast 是整对象解禁，mutable 是逐成员解禁）。
- **核心语法**：`cpp struct MutableStruct { int dontTouchThis; // const 对象里不可改 mutable double iCanChange; // const 对象里也能改 };`
- **设计意图与最佳实践**：典型用途是**调试/缓存元数据**——课堂例子 `CameraRay` 存 `mutable Color debugColor;`，`renderRay(const CameraRay&)` 里可以给调试光线着色而不改变光线本身的 const 语义。同样要谨慎使用：它绕过了 const 的保护。

代码示例与逐步解说（核心）

示例 1：手写 Vector 模板与实例化（C++11）

代码：

```cpp
// C++11
#include <iostream>
#include <string>

template <typename T>
class Vector {
private:
    T* elems = nullptr;
    size_t logical_size = 0;
    size_t array_size = 0;
    void grow() {
        array_size = array_size == 0 ? 4 : array_size * 2;
        T* bigger = new T[array_size];
        for (size_t i = 0; i < logical_size; ++i) bigger[i] = elems[i];
        delete[] elems;
        elems = bigger;
    }

public:
    void push_back(const T& elem) {
        if (logical_size == array_size) grow();
        elems[logical_size++] = elem;
    }
    T& at(size_t index) { return elems[index]; }
    size_t size() const { return logical_size; }
    ~Vector() { delete[] elems; }
};

int main() {
    Vector<int> intVec;                 // 实例化：T = int
    intVec.push_back(1);
    intVec.push_back(2);

    Vector<std::string> strVec;         // 实例化：T = std::string
    strVec.push_back("hello");
    strVec.push_back("world");

    std::cout << intVec.at(1) << " " << strVec.at(0) << "\n";  // 2 hello
    std::cout << intVec.size() << " " << strVec.size() << "\n"; // 2 2
}
```

代码做什么：用 `template <typename T>` 写一个极简可增长数组：`push_back` 在容量不足时 `grow()` 翻倍扩容；`main` 分别实例化 `Vector<int>` 与 `Vector<std::string>` 并读写。

特性机制解说： – **实例化 = 惰性代码生成**：编译器见到 `Vector<int>` 时把模板里的 `T` 全部替换成 `int` 生成一份类代码，见到 `Vector<std::string>` 再生成一份——两次生成的类**互不相干**；`main` 里从不出现的 `Vector<double>` 永远不会被生成。 – 这份极简实现省略了拷贝构造/赋值（默认浅拷贝会导致双重 `delete[]`），真实容器必须实现"三/五法则"或使用 RAII 类型——本讲只关注模板机制。

示例 2：非类型模板参数与 std::array（C++11）

代码：

```
// C++11
#include <array>
#include <iostream>
#include <string>

template <size_t N>          // N 是编译期常量，不是类型
class FixedBuffer {
private:
    std::array<int, N> data{};   // 大小 N 烘焙进类型
public:
    void fill(int v) { data.fill(v); }
    size_t size() const { return data.size(); }
};

int main() {
    std::array<std::string, 5> arr;   // 恰好 5 个 string，栈上分配
    arr[0] = "CS106L";
    std::cout << arr.size() << "\n";  // 5

    FixedBuffer<8> buf;               // 8 个 int 的栈上缓冲区
    buf.fill(42);
    std::cout << buf.size() << "\n";  // 8
}
```

代码做什么：用非类型模板参数 `size_t N` 定义固定大小缓冲区；演示 `std::array<std::string, 5>` 与 `FixedBuffer<8>`。

特性机制解说：– `FixedBuffer<8>` 与 `FixedBuffer<16>` 是两个不同类型，`sizeof` 分别为 32 与 64 字节——**大小是类型的一部分**，编译器据此在栈上精确分配，零堆分配、零运行时开销；`std::array<T, N>` 的声明本质就是 `template<typename T, std::size_t N> struct array;`，它把 C 数组的紧凑性与 `std::vector` 的接口（`.size()`、`.fill()`、迭代器）合二为一。

示例 3：课堂真实代码 BoundedValue——模板实现的怪癖（C++17，单文件可运行版）

代码：

```
// C++17（课堂 bounded_value.h / bounded_value.cpp 的完成版，此处合并为单文件以便运行）
#include <algorithm>
#include <iostream>

template <typename T>
class BoundedValue {
private:
    T value;
    T minValue;
    T maxValue;
public:
    // 初始化并保证 value 落在 [minValue, maxValue] 内
    BoundedValue(T value, T minValue, T maxValue)
        : value{std::clamp(value, minValue, maxValue)},
          minValue{minValue}, maxValue{maxValue} {}

    T get() const { return value; }         // const：只读查询

    void set(T newValue) {                  // 超出界限则夹到界限上
        value = std::clamp(newValue, minValue, maxValue);
    }

    T getMin() const { return minValue; }
    T getMax() const { return maxValue; }

    void adjust(T delta) { set(value + delta); }   // 加分项
};

int main() {
    BoundedValue<int> health{120, 0, 100};  // 120 被夹到 100
    std::cout << health.get() << "\n";      // 100
    health.set(-10);
    std::cout << health.get() << "\n";      // 0
    health.set(50);
    std::cout << health.get() << "\n";      // 50
    health.adjust(60);
    std::cout << health.get() << "\n";      // 100
}
```

代码做什么：`BoundedValue<T>` 存一个值及其上下界，任何写入都被 `std::clamp` 限制在界内；`main` 用 `int` 实例化并验证四种情况（越上界、越下界、正常、adjust 越界）。

特性机制解说：这是课堂真实代码 `bounded_value.h` + `bounded_value.cpp`（学生完成 TODO）+ `main.cpp` 的拆分方式，正是本讲怪癖的活教材：

```
// bounded_value.h —— 👻 怪癖(2)：.h 底部 include .cpp
template <typename T> class BoundedValue { /* 声明 */ };
#include "bounded_value.cpp"
// bounded_value.cpp —— 👻 怪癖(1)：复制 template 前缀 + BoundedValue<T>::
template <typename T>
BoundedValue<T>::BoundedValue(T value, T minValue, T maxValue) { /* ... */ }
template <typename T>
T BoundedValue<T>::get() const { return value; }
```

- 编译方式：`g++ -std=c++20 main.cpp -o main`——**不要把 bounded_value.cpp 单独编译/链接**（模板实现没有独立目标代码，靠 `.h` 底部的 include 进入每个使用点）。
- 所有 `get` / `getMin` / `getMax` 标了 `const`，因为它们是只读查询——这正是 const 正确性在真实代码中的样子。

示例 4：const 成员函数与 const 接口（C++11）

代码：

```
// C++11
#include <initializer_list>
#include <iostream>
#include <vector>

class IntVec {
private:
    std::vector<int> data;
public:
    IntVec(std::initializer_list<int> il) : data{il} {}
    size_t size() const { return data.size(); }        // const 成员函数
    const int& at(size_t index) const { return data.at(index); }
    int&       at(size_t index) { return data.at(index); }  // const 重载
};

void printVec(const IntVec& v) {        // v 是 const 引用：只能用 const 接口
    for (size_t i = 0; i < v.size(); ++i)
        std::cout << v.at(i) << " ";    // 解析到 const 版本
    std::cout << "\n";
}

int main() {
    IntVec v{1, 2, 3};
    printVec(v);        // 1 2 3
    v.at(0) = 42;       // 非 const 对象 → 非 const 重载，可写
    std::cout << v.at(0) << "\n";   // 42
}
```

代码做什么：模拟课堂 `Vector` 的 const 接口：`printVec` 接收 `const IntVec&` 并成功调用 `size()` 与 `at(i)`；`main` 里普通对象 `v` 的 `at(0)` 返回可写引用。

特性机制解说： – 若 `size()` 不标 `const`，`printVec` 里 `v.size()` 编译失败：编译器拿到 `const IntVec&`，只允许调用 const 接口。**const 成员函数中 `this` 的类型是 `const IntVec*`**——函数体内写 `data.push_back(1)` 会直接编译报错，const 是编译器强制执行的契约，不是注释。 – **const 重载的选择**：`v.at(0)` 中 `v` 非 const，选 `int& at(size_t)`；`printVec` 里 `v` 是 const 引用，选 `const int& at(size_t) const`。同一函数名、同一参数列表，仅凭 const 性区分两个重载。

示例 5：const_cast 委托——消除 const 重载的冗余（C++11）

代码：

```
// C++11
#include <iostream>
#include <stdexcept>
#include <vector>

class IntVec {
private:
    std::vector<int> data;
public:
    IntVec(std::initializer_list<int> il) : data{il} {}
    int& findElement(int value) {           // 非 const 版本：真正实现
        for (int& e : data)
            if (e == value) return e;
        throw std::out_of_range("Element not found");
    }

    const int& findElement(int value) const {   // const 版本：委托
        return const_cast<IntVec&>(*this).findElement(value);
    }
};

int main() {
    IntVec v{10, 20, 30};
    const IntVec& cv = v;
    std::cout << cv.findElement(20) << "\n";  // 20（const 版本，只读）
    v.findElement(10) = 99;                   // 非 const 版本，可写
    std::cout << v.findElement(99) << "\n";   // 99
}
```

代码做什么：`findElement` 的两个 const 重载中，const 版本用一行 `const_cast<IntVec&>(*this).findElement(value)` 委托给非 const 版本，逻辑只写一份。

特性机制解说： – 逐步拆解 `const_cast<IntVec&>(*this).findElement(value)`：`*this` 是 `const IntVec&`；`const_cast<IntVec&>` 剥掉 const 得到非 const 引用；此时 `.findElement` 按**静态类型**解析到非 const 重载，返回 `int&`；最后隐式转换为 `const int&` 返回。**安全性**：这里的对象（`cv` 引用的 `v`）本来就是非 const 的，只是通过 const 引用访问——剥 const 没有风险；若对象真是 const，内部修改就是未定义行为。 – 替代写法是把实现抽成私有 `int& findElementImpl(...)` 让两个公共版本都调用；const_cast 委托只是少写一个函数。课堂结论：**const_cast 几乎从不使用**。

示例 6：mutable——const 对象中的"可改豁免"（C++11）

代码：

```
// C++11
#include <iostream>

struct CameraRay {
    int origin = 0;
    int direction = 0;
    mutable int debugColor = 0;   // 调试元数据：const 下也可写
};

void renderRay(const CameraRay& ray) {
    ray.debugColor = 42;          // ✅ mutable 成员可修改
    // ray.origin = 1;            // ❌ 普通成员不可修改
    std::cout << ray.debugColor << "\n";
}

int main() {
    CameraRay ray;
    renderRay(ray);               // 42
}
```

代码做什么：`renderRay` 接收 const 引用，但能更新 `mutable` 的 `debugColor` 调试字段，普通成员则被编译器禁止修改。

特性机制解说：– `mutable` 只豁免**被标记的成员**，比 const_cast（整对象解禁）更细粒度、更安全。典型场景：缓存、调试着色、统计计数——它们不属于对象的"逻辑状态"，修改它们不影响 const 语义。编译器视角：const 对象里普通成员按 `const T` 处理，`mutable` 成员始终按 `T` 处理。 – 滥用 mutable 会破坏 const 契约的可信度，务必克制。

与旧标准（如 C++98）的对比

本讲特性（C++11+）	C++98 的做法	新特性优势
类模板本身（`template <typename T> class Vector`）	预处理器宏 `#define GENERATE_VECTOR(T)`	宏的文本替换无类型检查、语法笨拙、忘调/重调即出错；模板由编译器做类型安全的代码生成
`std::array<T, N>`（C++11）	C 数组 `T arr[N]` 或 `std::vector`	有 `.size()`/迭代器/越界检查接口，又不堆分配；大小进入类型系统
模板别名 `template<class T> using Vec = std::vector<T>;`（C++11）	无（typedef 不能模板化）	可为模板"起别名"，简化 `Vector<Vector<int>>` 这类长类型

本讲特性（C++11+）	C++98 的做法	新特性优势
const 成员函数、const_cast、mutable	本为 C++98 已有机制	（提醒：这些是"老机制"，本讲的现代性在于 const 重载与委托惯用法的普及）
`std::clamp`（C++17，示例 3 用到）	手写 `max(min(x, hi), lo)`	语义清晰、避免重复嵌套

关键要点

- **模板是编译期"代码工厂"**：`template <typename T> class Vector` 不是类型，`Vector<int>` 才是；每种 T 各生成一份独立代码，且 `Vector<int>` 与 `Vector<double>` 是完全不同的类型。
- **模板实现必须对编译器可见**：实现放 `.h`（底部 `#include "Vector.cpp"`），成员函数定义要写全 `template <typename T>` 与 `Vector<T>::`；否则链接错误。
- **不修改对象的成员函数一律标 `const`**：const 成员函数中 `this` 是 `const Vector<T>*`，编译器强制保证不修改；const 对象只能用 const 接口。
- **const 重载让"读"与"写"各得其所**：`const T& at(size_t) const` 服务 const 用户，`T& at(size_t)` 服务普通用户；const 版本可用 `const_cast` 委托非 const 版本消除重复。
- **const_cast 与 mutable 是逃生门**：几乎从不使用；唯一的常见合法用途是 const 重载委托与调试元数据。

常见陷阱与注意事项

- **模板实现放 `.cpp` 且 `.h` 不 include 它、或写成 `Vector::at` 而漏掉 `Vector<T>::` 与 `template <typename T>`**：前者编译期正常、链接期报 `undefined reference`；后者当场报"Vector 不是类型 / 不知道 T"——模板实现必须随使用点可见。
- **声明与实现一处忘了 `const`**：`size_t size() const;` 声明了、`size_t Vector<T>::size()` 实现没写 `const`——二者是不同函数，报 "no matching function"。
- **在 const 方法里返回非 const 引用（`T& at(size_t) const`）**：const 用户 `v.at(0) = 42;` 就能改写 const 对象，const 契约被击穿；应返回 `const T&` 或用 const 重载。
- **对真正 const 的对象用 const_cast 修改**：未定义行为（可能静默崩溃）；const_cast 只应在对象本体非 const 时使用。
- **误以为模板实例之间可转换**：`std::vector<int>` 与 `std::vector<double>` 互不兼容；把 `Vector<double>` 传给接收 `Vector<int>` 的函数会编译失败。

关联作业提示

本讲对应 **A4: Ispell**（拼写检查器，基于 STL 算法与 ranges 库），模板与 const 知识贯穿全程：

- **模板函数**：作业提供的 `find_all<Iterator, UnaryPred>` 就是本讲"模板按需实例化"的实战——它会用 `std::string::iterator` 实例化，也会用别的迭代器实例化，一份逻辑通用所有类型。
- **类型别名**：`using Corpus = std::set<Token>;`、`using Dictionary = std::unordered_set<std::string>;` 正是 Lecture 7 的 `using String = std::string;` 的推广（本讲补充了模板别名）。
- **const 正确性**：`tokenize` 与 `spellcheck` 的签名是 `Corpus tokenize(std::string& input)` 与 `std::set<Misspelling> spellcheck(const Corpus& source, const Dictionary& dictionary)`——`spellcheck` 对两个参数都用 **const 引用**，因为只读不写；lambda 捕获 `source` **必须按引用**（`[&]` 或 `[&source]`），否则拷贝会破坏 Token 的迭代器语义（作业特别警告过）。这也呼应"const 接口"思想：只读函数不该拿到可变权限。
- **模板构造函数**：`Token` 有一个模板构造函数 `template <typename It> Token(std::string& source, It begin, It end);`——它在 `std::transform` 的 lambda 里以任意迭代器类型实例化，是"模板逻辑写一次、类型当参数"的直接应用。
- 若你之前按 A3 加分项把类写成了类模板，本讲的"`.h` 底部 include 实现、编译命令去掉 `.cpp`"规则就是你要遵守的那套。

Lecture 10 (Week 5 – Thursday): 函数模板 (Function Templates)

概述

上一讲我们学会了"模板类"——为任意类型生成类代码的蓝图；本讲把同一思想延伸到函数：**模板函数（function template）** 让 `min`、`find` 这样的算法对任何类型自动生成对应版本，编译器像工厂一样按需"打印"出具体函数。课程依次介绍模板函数的显式/隐式实例化、基于模板的泛型 `find`（它正是 STL `<algorithm>` 中所有算法的雏形）、C++20 **Concepts**（在实例化之前约束模板参数、改善报错信息）、**可变参数模板**（variadic templates，用参数包支持任意数量实参），以及**模板元编程**（编译期递归实例化计算阶乘/斐波那契）与 `constexpr`/`consteval`。学完本讲，你就能读懂 STL 算法的签名，也为下一讲 lambda 与 ranges 打下基础。

核心特性与语法详解

1. 模板函数（Template Functions）

- **定义与目的**：用 `template <typename T>` 前缀声明一个"函数蓝图"，`T` 是类型占位符。调用时编译器用具体类型替换 `T`，生成真正的函数。目的：消除为每种类型重复编写相同逻辑的重载代码，实现**代码生成自动化**。

- **核心语法**：`template <typename T> T min(T a, T b);`——声明模板函数；`template <typename T> T min(const T& a, const T& b);`——按 const 引用传参，避免复制。
- **设计意图与最佳实践**：模板本身**不是函数**，`min<std::string>` 才是一个函数（称为一次*实例化*，instantiation）。模板像工厂：喂进类型，产出函数。实践中模板声明与定义都要放在头文件中（否则链接期找不到实例化代码）；对大对象（如 `std::string`）优先用 `const T&` 参数。

2. 显式实例化与隐式实例化

- **定义与目的**：两种"告诉编译器 T 是什么"的方式。显式实例化由程序员指定类型，隐式实例化由编译器从实参推导。
- **核心语法**：
- 显式：`min<int>(106, 107);`——与模板类 `vector<int>` 的写法一致。
- 隐式：`min(106, 107);`——编译器推导出 `T = int`，等价于 `min<int>(106, 107)`，就像 `auto number = 106;` 让编译器推断类型。
- **设计意图与最佳实践**：日常优先隐式实例化（更简洁）；当推导失败或结果有歧义时（字符串字面量、混合类型实参），显式实例化是可靠的退路。

3. 泛型算法思想：以 find 为例

- **定义与目的**：把 `find` 写成模板，参数类型为"迭代器"而不是具体容器，从而一套实现通用于 `vector`、`set`、`unordered_map` 等所有容器。
- **核心语法**：`template <typename Iterator, typename TElem> Iterator find(Iterator begin, Iterator end, TElem value);`
- **设计意图与最佳实践**：`<algorithm>` 里每个算法都是这样的模板函数。只依赖迭代器协议（`*it`、`++it`、`it != end`、`*it == value`），不关心容器内部结构。这也是为什么 `std::find` 是 `find(first, last, value)` 而不是 `find(container, value)`——传迭代器可以只搜索容器的一部分（子区间）。

4. Concepts（C++20）

- **定义与目的**：给模板参数加**约束**的命名集合。没有约束时，`min<StanfordID>` 会先被实例化、然后在函数体内报出"invalid operands to binary expression"这种令人困惑的错误（错误只出现在实例化之后）；有了 concept，约束不满足时**根本不实例化**，直接给出清晰信息，同时提升 IDE 支持。
- **核心语法**：`cpp template <typename T> concept Comparable = requires(const T a, const T b) { { a < b } -> std::convertible_to<bool>; // 约束：a < b 必须合法且结果可转 bool }; template <Comparable T> T min(const T& a, const T& b); // 简写 template <typename T> requires Comparable<T> T min(const T& a, const T& b); // 等价写法`
- **设计意图与最佳实践**：C++ 一直在补 Java（`T extends Comparable<T>`）、C#（`where T : IComparable`）早就有的"泛型约束"能力。标准库自带大量 concept：`std::convertible_to`、`std::input_iterator`、`std::ranges::range` 等。注意 STL 目前尚未全面使用 concepts。

5. 可变参数模板（Variadic Templates）

- **定义与目的**：让函数接受**任意数量、任意类型**的实参（如 `min(2.4, 7.5, 5.3, 1.2, 3.4)` 或 Python 风格 `format`）。原理是"模板 + 递归"：编译器在实例化时自动生成所需数量的重载。
- **核心语法**：`cpp template <Comparable T> T min(const T& v) { return v; } // 基例：终止递归 template <Comparable T, Comparable... Args> // Args 是类型参数包 T min(const T& v, const Args&... args) { // args 是函数参数包 auto m = min(args...); // 包展开：替换为实际参数 return v < m ? v : m; }`
- **设计意图与最佳实践**：参数包可以匹配 0 个或多个类型；`args...` 在调用点被展开成逗号分隔的实际参数列表。必须有基例（base case）终止递归。各参数类型不必相同（如 `format("{} {}", "Rhaenyra", 7)`），但递归写法中返回类型通常由第一个参数决定，需要混合类型时要显式实例化。

6. 模板元编程（TMP）与 constexpr/consteval

- **定义与目的**：利用"模板在编译期实例化"这一事实，在编译期完成计算，把结果**烘焙进可执行文件**，运行时零开销。TMP 是图灵完备的，Boost.MPL 等库用"类型组成的 vector"做策略式设计（policy-based design）。
- **核心语法**：递归模板结构 + 模板特化基例；C++20 起用 `constexpr`（"请尽量在编译期运行"）与 `consteval`（"必须在编译期运行"）函数代替晦涩的模板结构。
- **设计意图与最佳实践**：先考虑 `constexpr` / `consteval`（可读、是 C++20 对 TMP 的"制度化"），只有需要操作类型本身时才写传统 TMP。

代码示例与逐步解说（核心）

示例 1：min 的三种形态与显式/隐式实例化（C++17）

代码

```cpp
#include <iostream>
#include <string>

// 形态一：按值传参，简单但会复制实参
template <typename T>
T min_basic(T a, T b) { return a < b ? a : b; }

// 形态二：按 const 引用传参，避免复制，最常用
template <typename T>
T min_ref(const T& a, const T& b) { return a < b ? a : b; }

// 形态三：允许两个实参类型不同，返回类型交给编译器推导
template <typename U, typename V>
auto min_flex(const U& a, const V& b) { return a < b ? a : b; }

int main() {
  // 显式实例化：手动指定类型
  int m1 = min_basic<int>(106, 107);
  double m2 = min_basic<double>(42.5, 3.14);

  // 隐式实例化：编译器从实参推导 T
  int m3 = min_ref(106, 107);                    // 等价于 min_ref<int>(106, 107)

  // 字符串字面量是 const char*，必须显式实例化才能得到字符串比较
  std::string s = min_ref<std::string>("Arwen", "Aragorn");

  // 混合类型：U = int, V = double，返回类型由三元运算符推导为 double
  auto m4 = min_flex(106, 107.5);

  std::cout << m1 << " " << m2 << " " << m3 << " "
            << s << " " << m4 << "\n";           // 106 3.14 106 Aragorn 106
}
```

代码做什么：定义了 `min` 的三种模板形态，分别演示显式实例化（`min_basic<int>`）、隐式实例化（`min_ref(106, 107)`）、多类型参数（`min_flex(106, 107.5)`），最后打印结果。

特性机制解说：`min_ref(106, 107)` 中，编译器把实参类型代入形参 `const T&` 反向推导出 `T = int`，与 `auto` 的推导思路完全一致。显式实例化则直接替换：`min_ref<std::string>("Arwen", "Aragorn")` 中 `T = std::string`，两个 `const char*` 字面量在绑定 `const std::string&` 时隐式转换为 `std::string`。`min_flex` 的 `auto` 返回类型在函数体确定后才推导——`a < b ? a : b` 中 `int` 与 `double` 的三元表达式公共类型是 `double`，故返回 `double`。注意形态一按值传参会复制整个对象，形态二只复制引用——对 `std::string` 这类大对象，性能差异明显（课堂代码 `main.cpp` 里 `min_basic` / `min_ref` / `min_flex` 正是这三个版本）。

示例 2：泛型 find——一套代码服务所有容器（C++17）

代码

```cpp
#include <iostream>
#include <string>
#include <unordered_set>
#include <vector>

// 泛型 find：对任何"迭代器对 + 值"都成立
template <typename It, typename T>
It my_find(It begin, It end, const T& value) {
  for (auto it = begin; it != end; ++it)
    if (*it == value) return it;
  return end;                          // 找不到返回 end
}

int main() {
  std::vector<int> v { 1, 2, 3, 4, 5 };
  auto it1 = my_find(v.begin(), v.end(), 3);
  if (it1 != v.end()) *it1 = 107;   // v = {1, 2, 107, 4, 5}

  std::unordered_set<std::string> us { "hello", "welcome", "cs106l!" };
  auto it2 = my_find(us.begin(), us.end(), "welcome");

  std::cout << v[2] << "\n";                     // 107
  std::cout << (it2 == us.end() ? "not found" : "found") << "\n";  // found
}
```

代码做什么：手写一个不依赖任何具体容器的 `find`，先在一个 `vector<int>` 中找到 3 并改写为 107，再在 `unordered_set<string>` 中查找 "welcome"，两处调用共用同一份模板实现。

特性机制解说：第一次调用推导 `It = std::vector<int>::iterator`、`T = int`；第二次推导 `It = std::unordered_set<std::string>::iterator`、`T = std::string`。模板体只依赖迭代器协议（`*it`、`++it`、`!=`、`==`），所以只要类型满足这四点即可实例化——这正是 STL 算法的设计哲学：**用迭代器把算法与容器解耦**。课堂代码里该函数叫 `find` 并放在全局命名空间，调用时必须写 `::find(...)`，因为实参（`std::vector<int>::iterator`）位于 `std` 命名空间，触发 **ADL（实参依赖查找）** 后编译器更倾向选择 `std::find`——这是取名时值得注意的细节（此处改名 `my_find` 规避）。另外，传迭代器对而非容器，让我们能只搜索子区间，例如 `find(v.begin() + 1, v.end() - 1, 106)`。

示例 3：用 Concepts 约束模板（C++20）

代码

```
#include <concepts>
#include <iostream>
#include <sstream>
#include <string>
#include <type_traits>

// 定义 concept：T 的 < 运算必须合法且结果可转换为 bool
template <typename T>
concept Comparable = requires(const T a, const T b) {
  { a < b } -> std::convertible_to<bool>;
};

// 简写语法：模板参数表里直接放 concept 名
template <Comparable T>
T min_c(const T& a, const T& b) { return a < b ? a : b; }

int main() {
  std::cout << min_c(10, 20) << "\n";                          // 20
  std::cout << min_c<std::string>("a", "b") << "\n";           // a

  // 编译期检查类型是否满足 concept
  static_assert(Comparable<int>);
  static_assert(!Comparable<std::stringstream>);  // stringstream 没有 operator<

  // 若取消注释，会得到"约束未满足"的清晰报错，而不是函数体内的难懂错误：
  // min_c(std::stringstream(), std::stringstream());
}
```

代码做什么：定义 `Comparable` concept 并用它约束 `min_c`，随后用 `static_assert` 在编译期验证 `int` 满足、`std::stringstream` 不满足该 concept。

特性机制解说：`requires(const T a, const T b) { ... }` 是一个 **requires 表达式**——花括号内每一条"约束"（constraint）都必须能通过编译；`{ a < b } -> std::convertible_to<bool>` 额外要求表达式的结果类型满足 `std::convertible_to<bool>`（`convertible_to` 本身也是一个 concept）。约束检查发生在**实例化之前**：`min_c(std::stringstream(), ...)` 会因为不满足 `Comparable` 而直接拒绝重载解析，报错信息点明"约束未满足"，远好于过去先实例化 `min<StanfordID>`、再在函数体 `return a < b ? a : b;` 处报 `invalid operands`（幻灯片展示了两种报错的对比）。课堂代码还用 `if constexpr (Comparable<T>)` 在编译期分支输出"是否满足"，展示了 concept 可作为编译期布尔值使用。

示例 4：可变参数 min——编译器替我们写重载（C++20）

代码

```
#include <concepts>
#include <iostream>
#include <string>

// 定义 Comparable concept（见示例 3）：要求 T 支持 a < b 且结果可转 bool
template <typename T>
concept Comparable = requires(const T a, const T b) {
    { a < b } -> std::convertible_to<bool>;
};

template <Comparable T>
T min_v(const T& v) { return v; }        // 基例：单个元素就是最小值

template <Comparable T, Comparable... Args>
T min_v(const T& v, const Args&... rest) {
  auto m = min_v<T>(rest...);            // 包展开 + 显式实例化，保持类型一致
  return v < m ? v : m;
}

int main() {
  std::cout << min_v(2, 7, 5, 1) << "\n";                       // 1
  std::cout << min_v<std::string>("cool", "variadic", "template!") << "\n";
  std::cout << min_v(10, 2.5, 3.0f) << "\n";      // 2：第一个参数决定返回类型！
  std::cout << min_v<double>(10, 2.5, 3.0f) << "\n";  // 2.5：显式实例化得到正确结果
}
```

代码做什么：用"基例 + 递归实例化"实现任意个数的 `min`，并演示混合类型时显式实例化（`min_v<double>`）的作用。

特性机制解说：调用 `min_v(2, 7, 5, 1)` 时，编译器选择递归模板并推导 `T = int`、`Args = [int, int, int]`，即实例化 `min<int, int, int, int>`；其函数体里的 `min_v<T>(rest...)` 被**包展开**为 `min_v<int>(a0, a1, a2)`——注意展开发生在编译期，`rest...` 被替换成逗号分隔的实参列表。这一调用又实例化 `min<int, int, int>`，依次递推：`min<int, int>` → `min<int>`，直到匹配基例 `min_v(const T& v)`（单参数、非变参、更特化，编译器总是选择最特化的模板）。于是一次 `min_v(2, 7, 5, 1)` 让编译器自动生成了 4 个重载，正是幻灯片"Templates + recursion = code generation"的体现。类型方面：递归调用显式写 `min_v<T>` 强制每次都用同一类型 `T`，所以 `min_v(10, 2.5, 3.0f)` 把一切都按 `int` 比较（结果为 2）；想要真正按 `double` 比较必须 `min_v<double>(...)`。可变参数模板的另一个经典应用是 `format` 式的异构参数函数（幻灯片实现了 Python 风格 f-string 打印器，`Args` 中每个类型可以不同）。

示例 5：编译期计算——TMP 与 constexpr/consteval（C++20）

代码

```cpp
#include <iostream>

// 传统模板元编程：递归实例化 + 模板特化基例
template <size_t N>
struct Factorial {
  enum { value = N * Factorial<N - 1>::value };
};
template <>
struct Factorial<0> {           // 全特化：N = 0 的基例
  enum { value = 1 };
};

// constexpr：编译器"尽量"在编译期求值（C++14 起函数体可递归）
constexpr size_t factorial_cx(size_t n) {
  if (n == 0) return 1;
  return n * factorial_cx(n - 1);
}

// consteval：强制在编译期求值（C++20）
consteval size_t factorial_ce(size_t n) {
  if (n == 0) return 1;
  return n * factorial_ce(n - 1);
}

int main() {
  std::cout << Factorial<7>::value << "\n";   // 5040，编译期算好
  constexpr auto a = factorial_cx(7);         // 编译期求值
  auto b = factorial_cx(7);                   // 运行期求值也可以
  constexpr auto c = factorial_ce(7);         // 必须是编译期
  std::cout << a << " " << b << " " << c << "\n";
}
```

代码做什么：三种方式在编译期计算 7 的阶乘 5040：模板元编程（`Factorial<7>`）、`constexpr` 函数、`consteval` 函数。

特性机制解说：`Factorial<7>::value` 触发实例化链 `Factorial<7>` → `Factorial<6>` → … → `Factorial<0>`（特化基例给出 `value = 1`），随后逐层回填：1、2、6、24、120、720、5040。整条链发生在编译期，幻灯片展示的汇编显示结果直接以常量 `mov esi, 5040` 烘焙进可执行文件，运行时零开销。`constexpr` 是"尽力而为"——只要上下文需要常量表达式（如 `constexpr auto a`）就在编译期算；`consteval` 是"强制执行"——任何运行期调用都是编译错误。同类例子还有斐波那契（需要 `Fibonacci<0>` 与 `Fibonacci<1>` 两个特化基例：`value = Fibonacci<N-1>::value + Fibonacci<N-2>::value`）。一句话总结两者定位：`constexpr` 是"亲爱的编译器，请尽量在编译期跑我 😘"，`consteval` 是"你必须给我在编译期跑 😡"。传统 TMP 语法晦涩（`enum`、`BOOST_PP_*` 宏满天飞），所以幻灯片建议：能写 `constexpr` / `consteval` 就别写 TMP。

与旧标准（如C++98）的对比

- **函数模板本身**：C++98 就有，`template <typename T> T min(T a, T b);` 在 C++98 下同样成立——本讲的实例化机制是经典特性，不是新东西。
- **Concepts**：C++98/11/14/17 完全无对应物。过去只能用 SFINAE（`std::enable_if`）等"黑魔法"实现约束，报错信息晦涩难懂；C++20 的 `concept` / `requires` 是第一个一等公民的约束语法。C++98 里 `std::set<StanfordID>` 报出的错误会绵延数百行，如今一句话点明"约束未满足"。
- **可变参数模板**：C++11 引入。C++98 只能靠 C 风格 `...`（如 `printf`，类型不安全）或手写 N 个重载（幻灯片里写到第 7 个重载时直接放弃）。C++11 起 `Args...` 参数包 + 递归实例化把这件事类型安全地自动化。
- **模板元编程**：恰恰是 C++98 时代的产物（模板特化 + `enum` 技巧），但语法痛苦；`constexpr` 从 C++11 引入（早期函数体限制多、几乎只能写单条 return），C++14 放开循环/递归，`consteval` 是 C++20 新增。可以说现代 C++ 用 `constexpr` / `consteval` "收编"了 TMP 的常用场景。

关键要点

- **模板 ≠ 函数**：模板是生成函数的工厂，`min<T>` 的实例化才是函数；编译器按需自动生成代码。
- **默认用隐式实例化，模糊时显式实例化**：`min(106, 107)` 与 `min<int>(106, 107)` 等价；字符串字面量、混合类型等推导出歧义时，显式实例化是你的救生圈。
- **Concepts 在实例化之前检查约束**：`template <Comparable T>` 让报错信息与 IDE 体验大幅改善，是 C++20 写泛型代码的默认姿势。
- **可变参数模板 = 基例 + 递归实例化 + 包展开**：`args...` 在编译期展开为实参列表，编译器自动生成所需数量的重载。
- **需要编译期计算时**：优先 `constexpr`（尽量）或 `consteval`（强制），传统 TMP（模板特化 + `enum`）留给操作类型的场景。

常见陷阱与注意事项

- **字符串字面量隐式实例化为 `const char*`**：`min("Preston", "Rachel")` 推导出 `T = const char*`，`a < b` 变成**指针比较**（比地址而非字典序）——Bjarne 式摇头现场。改法：显式 `min<std::string>(...)`。
- **混合类型实参无法推导**：`min_ref(106, 3.14)` 中 `T` 既可能是 `int` 又可能是 `double`，编译失败。改法：双类型参数 + `auto` 返回（`min_flex`），或显式实例化 `min<double>(106, 3.14)`。
- **忘记递归基例**：可变参数模板没有单参数基例时，递归实例化永不终止，编译期爆炸（报错递归深度超限）。

- **模板定义放进 `.cpp` 文件**：模板只有被使用时才实例化，定义在别的翻译单元里会"链接期找不到符号"。模板（声明+定义）应放在头文件中。
- **与 STL 同名冲突（ADL）**：自己写 `find` 且参数是 `std` 容器迭代器时，ADL 会把调用解析到 `std::find`；要么改名，要么写 `::find(...)` 强制走全局命名空间。
- **不满足概念的类型也会被"实例化后才发现错误"**：没有 concepts 时 `min<StanfordID>` 会先实例化、再在 `a < b` 处报错——这正是下一讲/再下一讲运算符重载要解决的问题。

关联作业提示

本讲与 **A4: Ispell**（拼写检查器）直接相关。A4 要求你完全不用 for/while 循环，只靠 STL 算法与 ranges 完成 `tokenize` 和 `spellcheck`，而这一切都建立在模板之上：

- 讲义提供的 `find_all` 本身就是一个模板函数：`template <typename Iterator, typename UnaryPred> std::vector<Iterator> find_all(Iterator begin, Iterator end, UnaryPred pred);` ——本讲"泛型 find"的思想让你立刻理解它的签名与行为（返回所有满足谓词的迭代器，含 `begin` / `end` 边界）。
- `tokenize` 中用到的 `std::transform`、`std::erase_if` 都是 `<algorithm>` 里的模板函数，理解"实例化"帮助你读懂它们为何能对任意容器/迭代器生效。
- `spellcheck` 中 ranges 版本的 `std::ranges::views::filter/transform` 是受约束算法（内部用 concepts 声明 `input_range`），本讲 concepts 知识帮助你读懂其报错。
- A4 的 `Corpus = std::set<Token>` 要求 `Token` 具备 `operator<`（由讲义提供）——等学到 Lecture 12 运算符重载后，你会明白这背后的设计动机。

Lecture 11 (Week 6 – Tuesday): 函数与 Lambda (Functions & Lambdas)

概述

本讲解决一个根本问题：**如何把"行为"当作值来传递**。课程从布尔值函数——谓词（predicate）——出发，先看函数指针（`bool(*)(char)`）这种朴素方案，再用 lambda 表达式与捕获子句（capture clause）解决"函数需要携带状态"的痛点，并揭示 lambda 的真相：它会被编译器脱糖（desugar）成一个匿名的 functor 类。随后系统讲解 `<algorithm>` 的标准算法（`find_if` / `count_if` / `sort` / `transform` / `copy_if` / `unique_copy`），以课堂 tokenizer 为例演示"把问题识别成标准算法"的方法论；最后引入 C++20 的 **ranges 与 views**——惰性、可组合的算法流水线，与 Python 生成器逐行类比。本讲内容直接覆盖 A4 的前半部分（课堂会在课上带写第一段代码）。

核心特性与语法详解

1. 谓词（Predicate）

- **定义与目的**：谓词是**返回 `bool` 的函数**。它把"判断"抽象成一个可传递的值，让算法问出任意问题（"是不是元音？""是否可被 5 整除？"），而不是只做等值比较。
- **核心语法**：`cpp bool isVowel(char c) { ... } // 一元谓词：接收 1 个元素 bool isDivisible(int n, int d) { ... } // 二元谓词：接收 2 个元素`
- **设计意图与最佳实践**：`std::find` 只能找"等于某个值"的元素；把谓词作为参数（如 `std::find_if`），同一个算法就能回答任意问题。这是"用用户定义的行为泛化算法"的关键一步。

2. 函数指针

- **定义与目的**：函数在 C++ 中也有地址，可以存进指针变量再传给算法。`find_if(begin, end, isVowel)` 中模板参数 `Pred` 被推导为 `bool(*)(char)`。
- **核心语法**：`bool (*fp)(char) = isVowel;` —— `fp` 是指向"接收 `char`、返回 `bool` 函数"的指针。
- **设计意图与最佳实践**：函数指针**无法携带状态**——想找"小于 N 的数"而 `N` 要运行时才知道？函数指针只能指向"小于 5""小于 6""小于 7"这种写死的函数，无法把 `N` 带进去。这是它"泛化能力差"的根源，引出 lambda。

3. Lambda 与捕获子句

- **定义与目的**：lambda 是**从外层作用域捕获状态**的函数，本质是就地定义的匿名 functor。它解决了函数指针"无状态"的缺陷：`[n](int x) { return x < n; }` 把运行时读到的 `n` 装进函数里。
- **核心语法**：`cpp auto lessThanN = [n](int x) { return x < n; }; // 捕获子句 [n] + 参数 + 函数体 [x] // 按值捕获 x（复制一份） [x&] // 按引用捕获 x [x, y] // 按值捕获 x、y [&] // 全部按引用捕获 [&, x] // 除 x 按值外，其余按引用 [=] // 全部按值捕获`
- **设计意图与最佳实践**：捕获子句里的变量在 lambda 体内可见（普通函数体内只有参数可见）。`auto` 参数（泛型 lambda，C++14）是模板的简写：`[](auto x) { ... }` 等价于 `template <typename T> ...`，编译器在调用点隐式实例化。不需要捕获时可以不写捕获子句，lambda 就退化成普通匿名函数，还能隐式转换为函数指针。

4. Functor（函数对象）

- **定义与目的**：functor 是**任何定义了 `operator()` 的对象**——"行为像函数的对象"。因为它是对象，所以可以拥有成员变量（状态）。
- **核心语法**：`cpp struct my_functor { int operator()(int a) const { return a * value; } // 函数调用运算符 int value; // 状态！ }; my_functor f; f.value = 5; f(10); // 50`
- **设计意图与最佳实践**：STL 里 `std::greater<T>`、`std::hash<T>` 都是 functor。lambda 的底层实现就是一个编译器生成的匿名 functor 类（详见示例 3）。`std::function` 则是一个"包罗万象"的类型擦除

容器：任何函数指针、lambda、functor 都可以转换进去（代价是稍慢），日常更推荐 `auto` + 模板，不显式操心类型。

5. STL 算法（`<algorithm>`）

- **定义与目的**：`<algorithm>` 是**模板函数的集合**，全部基于迭代器操作，覆盖排序、查找、计数、变换、复制、去重等。STL 四大件：容器（存什么）、迭代器（怎么遍历）、算法（怎么通用地处理）、functor（怎么表示行为）。
- **核心语法**：`cpp std::count_if(first, last, p); // [first, last) 中满足 p 的元素个数 std::sort(first, last, comp); // 按 comp 排序 std::max_element(first, last, comp);// 按 comp 找最大元素 std::copy_if(r1, r2, o, p); // 把满足 p 的元素复制到输出 o std::transform(r1, r2, o, op); // 对每个元素应用 op，写入 o std::unique_copy(i1, i2, o, p); // 去掉连续重复，写入 o`
- **设计意图与最佳实践**：方法论（课堂 tokenizer 三步骤）：①先手工演算一个例子；②提炼逻辑；③判断它是否对应某个标准算法——是，就用标准库，换取正确性与可读性。

6. Ranges 与 Views（C++20/23）

- **定义与目的**：**range** 是任何"有 `begin` 和 `end` "的东西（容器、自定义类型）；**view** 是**惰性地适配另一个 range** 的 range——不复制元素，边遍历边按需计算。ranges 是 STL v2：新的范围算法（`std::ranges::find(v, 'c')` 直接传容器）用 concepts 约束，报错更好；views 让我们用 `|` 管道把算法组合成流水线。
- **核心语法**：`cpp auto it = std::ranges::find(v, 'c'); // 传整个容器 auto view = letters | std::ranges::views::filter(isVowel) | std::ranges::views::transform(toupper); std::vector<char> out = std::ranges::to<std::vector<char>>(view); // 物化 (C++23)`
- **设计意图与最佳实践**：**范围算法是急切的**（`std::ranges::sort(v)` 立刻排序），**views 是惰性的**（构建流水线一行都不算，直到被消费/物化才逐元素求值）。views 就像 Python 生成器：`(l for l in letters if isVowel(l))` 再 `list(view)` 物化。注意 `std::ranges::to` 是 C++23 特性，C++20 下用容器的迭代器构造函数物化（如 `std::vector<char> out(view.begin(), view.end());`）。

代码示例与逐步解说（核心）

示例 1：谓词 + 函数指针——find_if 与"无状态"困境（C++17）

代码

```cpp
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>
#include <vector>

// 一元谓词：判断字符是否为元音
bool isVowel(char c) {
  c = std::toupper(c);
  return c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}

// 另一个"行为"：判断整数能否被 5 整除
bool isGood(int x) { return x % 5 == 0; }

int main() {
  std::string flower = "rose";
  auto it = std::find_if(flower.begin(), flower.end(), isVowel);
  if (it != flower.end()) *it = 'i';          // 找到 'o' 改成 'i' → "rise"

  std::vector<int> nums { 3, 7, 10, 12 };
  auto it2 = std::find_if(nums.begin(), nums.end(), isGood);   // 指向 10

  std::cout << flower << " " << *it2 << "\n";   // rise 10

  // 函数指针的真实类型：Pred = bool(*)(char)
  bool (*fp)(char) = isVowel;

  // 困境：想找"小于 N"的数，而 N 运行时才知道——
  // 函数指针指向写死的函数，无法携带 N 这个状态！
  // find_if(begin, end, ???)   // 没有现成函数可用
}
```

代码做什么：把 `isVowel`、`isGood` 两个谓词直接传给 `std::find_if`，分别找出字符串中的第一个元音和数组里第一个能被 5 整除的数，并展示函数指针的类型写法。

特性机制解说：`std::find_if` 的签名是 `template <typename InputIt, typename UnaryPred> InputIt find_if(InputIt first, InputIt last, UnaryPred p);`——`UnaryPred` 由实参推导：第一次调用 `Pred = bool(*)(char)`，第二次 `Pred = bool(*)(int)`。算法内部对每个元素执行 `p(*it)`，因此**用户定义的行为被注入到通用算法中**。注意 `find_if` 与 `find` 的差别：`find` 问"等于 value 吗？"，`find_if` 问"满足谓词吗？"——后者能回答任意问题。函数指针语法 `bool (*fp)(char)` 要拆开读：`fp` 先与 `*` 结合是指针，`bool(...)(char)` 说明它指向"接收 char、返回 bool"的函数。

示例 2：Lambda 捕获与捕获子句大全（C++17/20）

代码

```cpp
#include <algorithm>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

int main() {
  std::vector<int> v { 3, 1, 4, 1, 5, 9, 2, 6 };

  int n = 5;
  // [n]：按值捕获 n（复制一份）；x 是参数
  auto lessThanN = [n](int x) { return x < n; };
  auto it = std::find_if(v.begin(), v.end(), lessThanN);
  std::cout << "第一个 < n 的元素: " << *it << "\n";          // 3

  int count = std::count_if(v.begin(), v.end(), [n](int x) { return x < n; });
  std::cout << "小于 n 的元素个数: " << count << "\n";        // 5

  // 捕获子句一览
  int a = 1, b = 2;
  auto f1 = [a, b](int x) { return x + a + b; };  // 按值捕获 a、b
  auto f2 = [&](int x) { return x + a + b; };     // 全部按引用捕获
  auto f3 = [&, a](int x) { return x + a + b; };  // 全部按引用，但 a 按值
  auto f4 = [=, &b](int x) { return x + a + b; }; // 全部按值，但 b 按引用

  // 无捕获 lambda 可转换为函数指针
  int (*fp)(int) = [](int x) { return x * 2; };
  std::cout << fp(21) << "\n";                    // 42

  // auto 参数 = 泛型 lambda（C++14）：等价于模板
  auto generic = [](auto x, auto y) { return x + y; };
  std::cout << generic(1, 2) << " "
            << generic(std::string("a"), std::string("b")) << "\n";  // 3 ab

  // 排序：默认升序 vs 自定义比较
  std::sort(v.begin(), v.end(), [](int x, int y) { return x > y; });  // 降序
  std::cout << v.front() << "\n";                 // 9

  // std::function：函数指针/lambda 的统一容器（稍慢）
  std::function<bool(int)> pred = lessThanN;
  std::cout << pred(3) << "\n";                   // 1 (true)
}
```

代码做什么：用 lambda 完成"小于 N"的查找与计数（N 运行时才知道——函数指针做不到），遍历捕获子句的各种写法，并演示泛型 lambda、函数指针转换、自定义排序与 `std::function`。

特性机制解说：lambda 语法 `[捕获](参数) { 函数体 }` 中，捕获子句决定外层变量如何进入函数体：`[n]` 复制一份（快照），`[&n]` 持有引用（能看到后续修改）。函数体里只有参数和捕获变量在作用域内。`[](auto x)` 的 `auto` 参数让 lambda 变成**泛型 lambda**——编译器把它翻译成一个 `template <typename T>` 的 `operator()`，与普通模板函数同理（示例里同一 lambda 既用于 `int` 又用于 `std::string`）。无捕获 lambda 因为没有状态，可以直接退化为函数指针。`std::sort` 的第三参数是二元谓词（比较器），默认

`operator<` 升序，传 `x > y` 即降序。`std::function<bool(int)>` 是类型擦除的"函数容器"：lambda、函数指针、functor 都能放进去，代价是间接调用稍慢，日常用 `auto` 即可。

示例 3：Lambda 的真相——脱糖为匿名 functor 类（C++17）

代码

```
#include <algorithm>
#include <iostream>
#include <vector>

// 编译器眼中的 lambda：[n](int x){ return x < n; }
// 会被展开成这样一个匿名的 functor 类（名字是编译器编的，如 lambda_6_18）：
class lambda_6_18 {
public:
  bool operator()(int x) const { return x < n; }   // 函数调用运算符
  lambda_6_18(int _n) : n{_n} {}                    // 捕获变量经构造参数传入
private:
  int n;                                            // 捕获的变量变成成员字段
};

int main() {
  int n = 10;
  std::vector<int> v { 3, 1, 4, 1, 5, 9, 2, 6 };

  // 写法 A：lambda 语法糖
  auto lessThanN = [n](int x) { return x < n; };
  auto itA = std::find_if(v.begin(), v.end(), lessThanN);

  // 写法 B：手写 functor 类——与 A 完全等价
  lambda_6_18 lessThanN2(n);
  auto itB = std::find_if(v.begin(), v.end(), lessThanN2);

  std::cout << *itA << " " << *itB << "\n";        // 3 3
}
```

代码做什么：同一份逻辑分别用 lambda 与手写 functor 类实现，二者在 `find_if` 中行为完全一致，验证"lambda 就是 functor 的语法糖"。

特性机制解说：lambda 的脱糖规则（课堂幻灯片原版）：①捕获的变量变成类的**成员字段**（按值捕获 → 值成员，按引用捕获 → 引用成员）；②函数体变成 `operator()(参数) const`；③捕获变量经**构造函数**从外层传入。于是 `[n](int x) { return x < n; }` 生成 `lambda_6_18 { n }`，`find_if` 拿到的是这个对象。这也解释了为什么 lambda 有独特的不可拷贝/不可命名类型——`auto` 是唯一方便的写法（"我不知道类型，但编译器知道"）。functor 因为本质是对象，天然拥有状态——课堂 `my_functor` 例子用成员 `value` 充当乘法因子。这与范围 for 循环脱糖成迭代器循环是同一类"语法糖"现象；想亲眼看到展开过程可以试试 cppinsights.io。顺带一提：`std::hash<T>` 等 STL functor 也是这么工作的（模板特化 + `operator()`）。

示例 4：传统 STL 算法管线——Soundex 经典版（C++17，课堂代码改写）

代码

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <iterator>
#include <map>
#include <string>

// 把字母映射成 Soundex 编码数字（课堂 soundex.cpp 的编码表）
static char soundexEncode(char c) {
  static const std::map<char, char> encoding = {
    {'A','0'},{'E','0'},{'I','0'},{'O','0'},{'U','0'},{'H','0'},{'W','0'},{'Y','0'},
    {'B','1'},{'F','1'},{'P','1'},{'V','1'},
    {'C','2'},{'G','2'},{'J','2'},{'K','2'},{'Q','2'},{'S','2'},{'X','2'},{'Z','2'},
    {'D','3'},{'T','3'},{'L','4'},{'M','5'},{'N','5'},{'R','6'}
  };
  return encoding.at(std::toupper(c));
}

static bool notZero(char c) { return c != '0'; }

std::string soundex(const std::string& s) {
  // 1) 只保留字母
  std::string letters;
  std::copy_if(s.begin(), s.end(), std::back_inserter(letters), ::isalpha);

  char first_letter = letters[0];                          // 记住首字母

  // 2) 每个字母 → 编码数字
  std::transform(letters.begin(), letters.end(), letters.begin(), soundexEncode);

  // 3) 去掉连续重复的编码
  std::string unique;
  std::unique_copy(letters.begin(), letters.end(), std::back_inserter(unique));

  unique[0] = std::toupper(first_letter);                  // 首字母放回

  // 4) 去掉 0 编码（元音等）
  std::string no_zeros;
  std::copy_if(unique.begin(), unique.end(), std::back_inserter(no_zeros), notZero);

  // 5) 补零到至少 4 位
  no_zeros += "0000";
  return no_zeros.substr(0, 4);
}

int main() {
  std::cout << soundex("Robert") << " " << soundex("Rupert") << "\n";  // R163 R163
}
```

代码做什么：完整实现经典 Soundex 算法（发音相似的名字得到相同编码）：过滤字母 → 编码 → 去连续重复 → 还原首字母 → 去零 → 补零截断。`Robert` 与 `Rupert` 都得到 `R163`。

特性机制解说：这是"识别标准算法"方法论的典范，每一行都是管道的一级：① `copy_if` + `::isalpha`（谓词）把非字母滤掉，`std::back_inserter(letters)` 是输出迭代器，每写一个元素就 `push_back` 一个，免去手动扩容；② `transform` 把每个字母经 `soundexEncode` 变成数字字符，输入输出都是 `letters`（原地变换）；③ `unique_copy` 只消除**连续**重复（如 "LL"→"L"），这正是 Soundex 的"合并相邻同音"规则；④再一次 `copy_if` 用 `notZero` 谓词去掉 '0'（元音/H/W/Y 编码）；⑤字符串拼接 `"0000"` 后 `substr(0, 4)` 保证定长 4。注意这套传统写法**每一步都急切地物化一个完整容器**（`letters`、`unique`、`no_zeros` 三个中间 `std::string`）——与下一节的 view 惰性管线形成鲜明对比。`std::transform` 还有二元重载：课堂 tokenizer 用 `std::transform(spaces.begin(), spaces.end()-1, spaces.begin()+1, std::back_inserter(tokens), binary_op)` 把"相邻两个空白迭代器"配对成 `Token`（这正是 A4 的做法）。

示例 5：Ranges 基础——惰性视图流水线（C++20/23）

代码

```cpp
#include <algorithm>
#include <cctype>
#include <iostream>
#include <ranges>
#include <string>
#include <vector>

bool isVowel(char c) {
  c = std::toupper(c);
  return c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}

int main() {
  std::vector<char> letters { 'a', 'b', 'c', 'd', 'e' };

  // 范围算法：直接传容器（容器是 range——有 begin/end）
  auto it = std::ranges::find(letters, 'c');
  std::cout << *it << "\n";                                   // c

  // 视图流水线：惰性、可组合（C++20 起可用；std::ranges::to 需 C++23）
  auto view = letters
    | std::ranges::views::filter(isVowel)                     // 只留元音
    | std::ranges::views::transform([](char c) { return std::toupper(c); });

  // 到这一行为止，什么也没算！view 只是一份"配方"
  // 注：std::ranges::to 是 C++23 特性（需 GCC 13+ / Clang 17+）；本机 GCC 12 请用下方 C++20 写法
  std::vector<char> upperVowel = std::ranges::to<std::vector<char>>(view);  // C++23
  for (char c : upperVowel) std::cout << c;                   // AE
  std::cout << "\n";

  // C++20 兼容的物化方式：用迭代器构造容器
  auto view2 = letters | std::ranges::views::filter(isVowel);
  std::vector<char> v2(view2.begin(), view2.end());
  std::cout << v2.size() << "\n";                             // 2
}
```

代码做什么：先用 `std::ranges::find` 传整个容器查找，再用 `|` 管道把 `filter`（只留元音）与 `transform`（转大写）组合成视图，最后物化为 `vector<char>`（'A'、'E'）。

特性机制解说：ranges 的核心概念：**range = 有 begin/end 的东西**（`std::ranges::range` concept 即检查 `ranges::begin(t)` / `ranges::end(t)` 是否合法）；范围算法（`std::ranges::find`、`sort` 等）是 `<algorithm>` 的"重皮肤"版本，用 concepts 约束（`template <ranges::input_range R, ...>`），报错信息比迭代器版更友好。**view 是惰性适配器**：`filter` 不复制元素、不提前过滤，只在被遍历到某个元素时才判断；`transform` 只在元素被取出时才调用函数。所以示例里 `view` 构造完时**零计算**，直到 `std::ranges::to`（C++23）或容器构造函数（C++20）开始消费它，才逐元素"拉取"数据。这就是流水线组合（`|`）的意义：`letters | filter | transform | to` 读起来像声明式的数据处理配方。对比 Python 生成器：

```
view = (l for l in letters if isVowel(l))     # 惰性
view = (l.upper() for l in view)              # 惰性
upperVowel = list(view)                       # 物化
```

`filter` 的两种调用形态等价：`std::ranges::views::filter(letters, isVowel)` 与 `letters | std::ranges::views::filter(isVowel)`（后者是"范围适配器闭包"，可链式拼接）。**注意急切与惰性的区分**：`std::ranges::sort(v)` 是急切的（立刻真的排序），views 才是惰性的。

示例 6：Ranges 进阶——Soundex 的 ranges 版（C++23，课堂代码改写）

代码

```cpp
#include <algorithm>
#include <cctype>
#include <iostream>
#include <map>
#include <ranges>
#include <string>

// Soundex 编码表：字母 -> 数字字符（与示例 4 的 soundexEncode 相同）
static char soundexEncode(char c) {
  static const std::map<char, char> encoding = {
    {'A','0'},{'E','0'},{'I','0'},{'O','0'},{'U','0'},{'H','0'},{'W','0'},{'Y','0'},
    {'B','1'},{'F','1'},{'P','1'},{'V','1'},
    {'C','2'},{'G','2'},{'J','2'},{'K','2'},{'Q','2'},{'S','2'},{'X','2'},{'Z','2'},
    {'D','3'},{'T','3'},
    {'L','4'},
    {'M','5'},{'N','5'},
    {'R','6'}
  };
  return encoding.at(std::toupper(c));
}
static bool notZero(char c) { return c != '0'; }

std::string soundexRanges(const std::string& s) {
  namespace rv = std::ranges::views;

  // 第一个字母：范围版 find_if 直接作用于字符串
  auto first = *std::ranges::find_if(s, ::isalpha);

  // 惰性管线：过滤字母 → 编码
  auto v = s | rv::filter(::isalpha) | rv::transform(soundexEncode);

  // 物化 + 去连续重复（范围版 unique_copy）
  std::string encoded;
  std::ranges::unique_copy(v, std::back_inserter(encoded));
  encoded[0] = std::toupper(first);          // 首字母放回

  return encoded
       | rv::filter(notZero)                 // 去掉 '0'
       | rv::take(4)                         // 只取前 4 个字符
       | std::ranges::to<std::string>();     // 物化成 string（C++23）
}

int main() {
  std::cout << soundexRanges("Robert") << "\n";   // R163
}
```

代码做什么：用 ranges 重写 Soundex：`find_if` 取首字母，`filter | transform` 组成编码管线，`unique_copy` 去连续重复，最后 `filter(notZero) | take(4) | to<string>()` 产出定长 4 的编码。

特性机制解说：这个版本与示例 4 逻辑完全等价，但结构上是**嵌套的惰性视图**：`s | rv::filter(::isalpha) | rv::transform(soundexEncode)` 构建了一个两层的适配器链——遍历 `v` 时，

`transform` 向 `filter` 要元素，`filter` 再向 `s` 要字符并判断是否为字母，整条链**只对实际取出的字符执行编码函数**。`std::ranges::unique_copy(v, std::back_inserter(encoded))` 是范围版算法（第一个参数直接是 range，无需 `begin/end`）。最后的 `encoded | rv::filter(notZero) | rv::take(4) | std::ranges::to<std::string>()` 展示了组合的魅力：`take(4)` 在取出第 4 个满足条件的字符后立即停止，天然实现"截断到 4 位"（配合物化）。课堂注释里还提到 C++26 的 `rv::concat("0000")` 用于补零。评价两极：ranges 让代码更声明式、更可读、报错更好；但它非常新（C++20/23/26 逐步完善）、编译器支持参差、某些场景比手写循环慢（可参考 *The Terrible Problem of Incrementing a Smart Iterator*）。A4 的 `spellcheck` 正是要求用 `filter` / `transform` 视图 + 物化完成。

与旧标准（如C++98）的对比

- **Lambda**：C++11 引入。C++98 没有 lambda，只能手写具名 functor 类（定义 `operator()`、把状态存成员、手写构造函数）——`std::sort` 的自定义比较要专门写一个结构体。lambda 把这些样板全部语法糖化，让"就地定义行为"成为可能；捕获语法 `[x]`、`[&]` 也是 C++11 起才有（C++14 增加泛型 lambda 与 init-capture，C++20 增加模板参数 lambda）。
- **函数指针**：C++98 就有（继承自 C），如今仍可用，但"无状态"的硬伤使其在泛型编程中让位于 lambda/functor。
- **`std::function`**：C++11 引入（Boost.Function 的标准化）。C++98 里想统一存放不同类型可调用对象几乎不可能。
- **Ranges 与 views**：C++98 完全没有对应物——只能写 `std::find(v.begin(), v.end(), x)` 这样显式迭代器对 + 中间容器逐个物化的流水线。`std::ranges::*` 算法与 `views` 是 C++20 新增，`std::ranges::to` 是 C++23；它们把"组合算法"从命令式变成声明式。
- **Functors**：C++98 就有（STL 最初的设计支柱），`std::greater<T>` 等至今未变——lambda 只是 functor 的语法糖，底层机制一直是它。

关键要点

- **谓词 = 返回 bool 的函数**：把"判断"当值传递，算法才能回答任意问题（`find_if` 而非 `find`）。
- **lambda 是携带状态的函数**：捕获子句 `[n]` / `[&]` 决定状态按值快照还是按引用共享；lambda 本质是编译器生成的匿名 functor 类。
- **能用标准算法就用标准算法**：先手工演算 → 提炼逻辑 → 匹配 `transform` / `copy_if` / `unique_copy` 等，换取正确性与可读性（tokenizer 三步骤）。
- **范围算法急切、views 惰性**：`std::ranges::sort` 立即执行；`filter | transform` 只是配方，物化（`to<>` 或容器构造）时才逐元素计算。
- **`auto` + 模板优先，`std::function` 兜底**：函数/lambda 的类型交给编译器推导；`std::function` 能做类型擦除但稍慢。

常见陷阱与注意事项

- **Lambda 引用捕获悬垂**：`[&]` 捕获的变量在 lambda 存活期内必须依然有效。若 lambda 逃逸出变量作用域（如存进容器返回出去），引用会悬垂。A4 中捕获 `source` 必须**按引用**（`Token` 需要引用源字符串），但注意 `source` 的生命周期要覆盖 lambda 的使用。
- **默认捕获 `[=]` 的语义**：按值捕获的是**当前值**的快照，之后外层变量再变，lambda 内看不到；需要看到最新值就 `[&]` 或按引用捕获单个变量。
- **函数指针无法携带状态**：别试图用函数指针表达"小于运行时读到的 N"——这正是 lambda 存在的意义。
- **视图悬垂**：view 只是对底层 range 的"窗口"，底层 range 被销毁后遍历 view 是未定义行为（如对临时对象取 view 再长期保存）。`std::ranges::to` 物化要趁底层 range 还活着。
- **`std::isspace` 的歧义**：直接写 `isspace` 可能匹配到 `<locale>` 里的模板重载导致类型推导失败；课堂/讲义建议写 `std::isspace`（必要时用全局 `::isspace` 区分）。
- **ranges 是新特性**：`std::ranges::to`（C++23）在某些编译器/标准库下不可用——A4 明确要求"只用 C++20 特性"，物化请用容器迭代器构造函数，别用 `std::ranges::to`。

关联作业提示

本讲与 **A4: Ispell** 直接相关（课堂上会带写前半部分）。A4 要求：`tokenize` 用传统 STL 算法、`spellcheck` 用 ranges 库，全程**不允许 for/while 循环**：

- **`tokenize`**（对应示例 4 的方法论）：①用讲义提供的 `find_all`（内部就是 `find_if` 的循环）收集所有空白字符迭代器（含 `begin`/`end` 边界，谓词用 `std::isspace`）；②用二元 `std::transform` 把"相邻两个迭代器"配对，lambda 形如 `[&source](auto it1, auto it2) { return Token(source, it1, it2); }`——**必须按引用捕获 `source`**（讲义特别警告，历史上很多同学栽在这里）；③用 `std::erase_if` 删除空 Token（谓词 `[](const auto& t) { return t.content.empty(); }`）。
- **`spellcheck`**（对应示例 5/6）：`source | rv::filter(拼写错误的 Token) | rv::transform(Token → Misspelling)` 三级流水线；生成建议时还要在 lambda 内**嵌套** `dictionary | rv::filter(levenshtein(...) == 1)` 视图，并用 `std::set<std::string> suggestions(view.begin(), view.end())` 物化——不要用 `std::ranges::to`（C++23，超纲）。
- 复习重点：捕获子句语法（`[&source]` 为什么必须按引用）、`filter`/`transform` 两种调用形态（`ranges::views::filter(r, pred)` 与 `r | views::filter(pred)` 等价）、`namespace rv = std::ranges::views;` 别名技巧、以及"视图惰性、物化才计算"的心智模型。

Lecture 12 (Week 6 – Thursday): 运算符重载 (Operator Overloading)

概述

本讲解决一个根本问题：**如何让自定义类型获得与内置类型一样的运算符语法**。开场的课堂回顾（functor、算法、ranges/views，以 Soundex 的经典版与 ranges 版对照演示）把前两讲串起来，随即抛出一句贯穿全课的名言："*Operators allow you to convey meaning about types that functions don't*"（运算符能传达函数传达不了的类型含义）。课程以 `StanfordID` 为例：`std::map<K,V>` 要求 `K` 有 `operator<`（查找依赖它），`min<StanfordID>` 也需要 `<`——于是我们学习成员/非成员两种重载方式、`friend` 关键字、`operator==` / `!=` 的 rule of contrariety、`operator<<` 流插入，以及最重要的设计哲学 **Principle of Least Astonishment（最少惊讶原则）**。本讲直接服务于 **A5: Treebook**（为 `User` 类实现 `operator<<`、`operator+=`、`operator<`）。

核心特性与语法详解

1. 为什么需要运算符重载（动机）

- **定义与目的**：运算符是"对值/对象/类型执行操作并产生新值或效果"的符号。对自定义类型重载运算符，就是给 `+`、`<`、`<<` 等符号赋予我们定义的行为，让类型表达出"数值般/可比较/可打印"的含义。
- **核心语法**：`return_type operator<symbol>(parameter_list);`
- **设计意图与最佳实践**：`money.add(otherMoney)` 读起来像随机函数调用，而 `money + otherMoney` 一眼就传达"钱可以相加"的数值语义——这就是"运算符传达类型含义"。`std::map<K,V>` 与 `std::set<K>` 都依赖 `K` 的 `operator<` 做有序存储与查找，`std::min` 同样依赖 `<`。重载运算符是解锁这些库功能的钥匙。

2. 哪些运算符可以重载

- **定义与目的**：C++ 允许重载**绝大多数**运算符（算术、比较、位运算、赋值、下标、调用、流插入等）。
- **核心语法**：`bool operator<(const T& other) const;`、`T operator+(const T& rhs) const;`、`T& operator+=(const T& rhs);`、`T& operator[](size_t i);`、`bool operator()(int x) const;` 等。
- **设计意图与最佳实践**：**不能重载**的运算符只有少数几个，必须记住：作用域解析 `::`、三目 `?:`、成员访问 `.`、成员指针访问 `.*`、`sizeof()`、`typeid()`、各种 `cast`。原因：这些运算符的语义与对象内存布局/类型系统深度绑定，重载会破坏语言基础。

3. 成员重载 vs 非成员重载

- **定义与目的**：运算符可以在类内部声明（成员重载），也可以写成类外的自由函数（非成员重载）。
- **核心语法**：`cpp // 成员重载：左操作数是 *this，只需一个参数 bool StanfordID::operator<(const StanfordID& other) const { ... } // 非成员重载：左右操作数都作为参数传入 bool operator<(const StanfordID& lhs, const StanfordID& rhs) { ... }`

- **设计意图与最佳实践**：STL 更偏爱非成员重载，也更符合惯用 C++，理由有二：①允许左操作数是**非类类型**（如 `5 + myInt` 的 `5` 不是类，无法调用成员函数）；②可以对**自己不拥有的类**重载（如 `std::string` 与自定义类型比较）。注意：**同时**定义成员与非成员版本的同签名运算符是未定义行为/歧义（编译器不知道该用哪个）。成员重载的优点是可以直接访问 `this->` 与私有成员。

4. friend 关键字

- **定义与目的**：`friend` 允许非成员函数（或另一个类）访问某个类的**私有成员**。非成员重载没有 `this`，默认碰不到 `private` 字段，friend 正好补上这个缺口。
- **核心语法**：在目标类的头文件里声明 `friend bool operator<(const StanfordID& lhs, const StanfordID& rhs);`，然后在类外定义该函数。
- **设计意图与最佳实践**：friend 声明放在类内（通常 `public` 区或 `private` 区皆可，位置不影响语义）。若实现只依赖公有接口（getter），就不需要 friend（幻灯片明确："friend 并非必需，如果我们没碰私有成员"）。friend 是"最小授权"的例外——能用公有接口就别开后门。

5. operator== 与 !=：Rule of Contrariety

- **定义与目的**：相等性判断是自定义类型最常用的语义之一。**Rule of contrariety（对立规则）**：实现了 `==` 就用它定义 `!=`，反之亦然——两个运算符必须互为否定，绝不能各自独立实现导致语义漂移。
- **核心语法**：`cpp bool StanfordID::operator==(const StanfordID& other) const { return name == other.name && sunet == other.sunet && idNumber == other.idNumber; } bool StanfordID::operator!=(const StanfordID& other) const { return !(*this == other); // 一句话搞定，保证语义一致 }`
- **设计意图与最佳实践**：`!=` 永远是 `!(*this == other)`。C++20 起甚至可以让编译器自动补齐（默认比较，见"与旧标准对比"）。

6. operator<< 流插入

- **定义与目的**：让 `std::cout << myObject;` 成立。签名**固定**：`std::ostream& operator<<(std::ostream& out, const T& obj);`——第一个参数是输出流，返回流本身以支持链式 `cout << a << b`。
- **核心语法**：`std::ostream& operator<<(std::ostream& out, const StanfordID& sid) { out << sid.name << " " << sid.sunet; return out; }`
- **设计意图与最佳实践**：实现细节（分隔符、字段顺序、要不要标签）取决于**你打算怎么用这个输出**——调试打印 vs 用户界面 vs 序列化，格式大不相同（幻灯片展示了两种风格）。通常作为非成员函数 + friend（需要访问私有字段），或只走公有 getter。

7. Principle of Least Astonishment（PoLA）

- **定义与目的**：运算符的目的是传达类型含义，因此**语义必须显而易见**：`+` 就是相加、`<` 就是排序/比较，功能上应与对应运算"合理相似"。

- **核心语法**：设计时的检查清单（不是语法）。
- **设计意图与最佳实践**：绝不要定义 `operator+` 做集合减法（幻灯片原话）。如果某操作的含义不明显，**就别用运算符，写个具名函数**（如 `merge(...)`）。此外：只在需要时重载（不用流就别写 `<<`）；重载了 `==` 就顺手补 `!=`；重载 `<` 时保证**严格弱序**（`std::set` / `std::map` / `std::sort` 都依赖它）。

代码示例与逐步解说（核心）

示例 1：成员 operator< —— 让 min 可用（C++17）

代码

```
#include <iostream>
#include <string>
#include <utility>

class StanfordID {
public:
  StanfordID(std::string name, std::string sunet, int idNumber)
      : name_(std::move(name)), sunet_(std::move(sunet)), idNumber_(idNumber) {}

  // 成员运算符重载：左操作数是 *this，rhs 是右操作数
  bool operator<(const StanfordID& other) const {
    return idNumber_ < other.idNumber_;    // 按学号比较
  }

  int getIdNumber() const { return idNumber_; }

private:
  std::string name_;
  std::string sunet_;
  int idNumber_;
};

// Lecture 10 的模板 min：内部只用 a < b
template <typename T>
T min(const T& a, const T& b) { return a < b ? a : b; }

int main() {
  StanfordID preston{ "Preston", "pseay", 106 };
  StanfordID rachel{ "Rachel", "rfern", 107 };

  auto m = min(preston, rachel);           // 之前编译错误，现在可以了！
  std::cout << m.getIdNumber() << "\n";    // 106
}
```

代码做什么：给 `StanfordID` 实现成员 `operator<`（按 `idNumber` 比较），于是 Lecture 10 的模板 `min` 实例化后能正常编译运行，返回学号较小者。

特性机制解说：没有 `operator<` 时，`min<StanfordID>` 会被实例化成 `StanfordID min(const StanfordID& a, const StanfordID& b) { return a < b ? a : b; }`，编译器在函数体内的 `a < b` 处报 `invalid operands to binary expression ('const StanfordID' and 'const StanfordID')`——因为模板实例化发生在编译期，错误"迟至实例化之后"才暴露。重载 `operator<` 后，`a < b` 被解析为对 `operator<` 的调用。成员重载的机制：`a < b` 等价于 `a.operator<(b)`，左操作数绑定到 `this`，右操作数绑定到参数 `other`；声明为 `const` 成员函数表示比较不会修改对象（`this` 是 `const`）。这也是为什么 `std::map<K,V>` 要求 `K` 有 `operator<`——红黑树的所有查找/插入都建立在 `<` 之上。

示例 2：非成员 operator< + friend —— 与 std::set 协作（C++17）

代码

```
#include <iostream>
#include <set>
#include <string>
#include <utility>

class StudentID {
public:
  StudentID(std::string name, int id) : name_(std::move(name)), id_(id) {}

  int getId() const { return id_; }

  // 在类内声明友元：允许这个非成员函数访问私有成员
  friend bool operator<(const StudentID& lhs, const StudentID& rhs);

private:
  std::string name_;
  int id_;
};

// 非成员重载：左右操作数都作为参数
bool operator<(const StudentID& lhs, const StudentID& rhs) {
  return lhs.id_ < rhs.id_;        // 借助 friend 直接访问私有 id_
}

int main() {
  std::set<StudentID> students;    // std::set 要求元素类型有 operator<
  students.insert(StudentID{ "Rachel", 107 });
  students.insert(StudentID{ "Preston", 106 });
  students.insert(StudentID{ "Anna", 106 });   // 与 Preston 学号相同 → 视为"相等"

  std::cout << "size = " << students.size() << "\n";   // 2（Anna 没进去）
  for (const auto& s : students)
    std::cout << s.getId() << "\n";                    // 106, 107
}
```

代码做什么：改用非成员 `operator<`（`lhs`、`rhs` 双参数）+ `friend` 访问私有 `id_`，把 `StudentID` 放进 `std::set`；学号相同的两个对象被 set 视为等价，第二个被丢弃。

特性机制解说：非成员重载没有 `this`，`lhs < rhs` 直接调用自由函数 `operator<(lhs, rhs)`，两个操作数地位对等——这让"左操作数是非类类型"（如 `3 < myObj`）成为可能，也是 STL 偏爱它的原因。但自由函数无法访问 `private` 成员，所以在类内用 `friend bool operator<(...)` 声明"破例授权"；若实现只走公有 getter（`lhs.getId() < rhs.getId()`），friend 就不是必需的。`std::set` 内部用 `!(a < b) && !(b < a)` 判定等价性（严格弱序）：`Preston`(106) 与 `Anna`(106) 互相都不小于对方，被视为同一元素，`Anna` 未被插入。**绝不要同时定义成员版和非成员版的同一签名运算符**——`a < b` 会同时匹配 `a.operator<(b)` 与 `operator<(a, b)`，造成歧义/未定义行为（幻灯片："ambiguity baddddddd"）。

示例 3：operator== 与 Rule of Contrariety（C++17/20）

代码

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class User {
public:
  User(std::string name, int age) : name_(std::move(name)), age_(age) {}

  bool operator==(const User& other) const {
    return name_ == other.name_ && age_ == other.age_;
  }

  // Rule of contrariety: != 永远定义为 == 的取反
  bool operator!=(const User& other) const {
    return !(*this == other);
  }

  std::string name() const { return name_; }
  int age() const { return age_; }

private:
  std::string name_;
  int age_;
};

int main() {
  User a{ "Alice", 21 };
  User b{ "Alice", 21 };
  User c{ "Alice", 20 };
  std::cout << (a == b) << " " << (a != c) << "\n";   // 1 1

  std::vector<User> users{ { "Bob", 19 }, { "Alice", 21 }, { "Carol", 20 } };
  std::sort(users.begin(), users.end(),
            [](const User& x, const User& y) { return x.age() < y.age(); });
  for (const auto& u : users) std::cout << u.name() << " ";   // Bob Carol Alice
  std::cout << "\n";
}
```

代码做什么：实现 `operator==`（名字与年龄都相同才算相等），并用 `!(*this == other)` 一句话实现 `operator!=`；随后用 lambda 比较器按年龄排序用户。

特性机制解说：`==` 的语义由你定义——对 `User` 而言"完全相同的两个人"就是两个字段都相等。rule of contrariety 的核心是**保证 `!=` 恒等于 `!==`**：如果分别独立实现，很容易出现 `a == b` 为真但 `a != b` 也为真的逻辑 bug。`(*this == other)` 里 `*this` 是左操作数（成员重载），递归调用自身重载，外层 `!` 取反——一句话、零重复。C++20 进一步支持 `operator==` 的**参数反转重写**（`a == b` 找不到时尝试 `b == a`）以及**默认比较**（`friend bool operator==(const User&, const User&) = default;` 逐个成员比较，见下节）。排序这里用的是 lambda 比较器而非重载 `<`——两种做法各有适用场景：比较器是一次性的、局部的；`operator<` 是类型固有的全局语义（`std::set` / `std::map` 需要后者）。

示例 4：operator<< 流插入（C++17）

代码

```
#include <iostream>
#include <ostream>
#include <string>
#include <utility>
#include <vector>

class User {
public:
  User(std::string name, std::vector<std::string> friends)
      : name_(std::move(name)), friends_(std::move(friends)) {}

  // 友元声明：非成员 operator<< 需要访问私有成员
  friend std::ostream& operator<<(std::ostream& out, const User& user);

private:
  std::string name_;
  std::vector<std::string> friends_;
};

std::ostream& operator<<(std::ostream& out, const User& user) {
  out << "User(name=" << user.name_ << ", friends=[";
  for (size_t i = 0; i < user.friends_.size(); ++i) {
    if (i > 0) out << ", ";
    out << user.friends_[i];
  }
  out << "])";
  return out;    // 必须返回流本身，才能链式 cout << a << b
}

int main() {
  User alice{ "Alice", { "Bob", "Charlie" } };
  std::cout << alice << "\n";
  // User(name=Alice, friends=[Bob, Charlie])
}
```

代码做什么：以 friend 非成员函数实现 `operator<<`，让 `std::cout << alice` 打印出 `User(name=Alice, friends=[Bob, Charlie])`（正是 A5 要求的输出格式）。

特性机制解说：`<<` 的重载形态很特殊——左操作数 `std::ostream` 是我们**不拥有的类**（无法给它加成员函数），所以必须用**非成员重载**（这正是"非成员重载可以对不拥有的类操作"的典型例子）；而实现要读 `name_`、`friends_` 私有字段，所以配 `friend`。签名固定为 `std::ostream& operator<<(std::ostream& out, const T& obj)`：返回 `out` 本身是为了支持 `std::cout << alice << "\n"` 的**链式调用**——`<<` 是左结合二元运算符，`(cout << alice) << "\n"`，前一个表达式的结果必须是流才能继续。格式（`friends=[Bob, Charlie]` 的逗号拼接）属于"使用方式决定实现"的范畴：这里选的是人类可读的调试/展示格式；若要做序列化可能换成无空格紧凑格式。若不需要打印，就别重载 `<<`（PoLA 的"只在需要时重载"）。

示例 5：综合练习——Pizza Order 类（C++17，课堂练习题）

代码

```cpp
#include <iostream>
#include <ostream>
#include <string>
#include <utility>

class PizzaOrder {
public:
  PizzaOrder(std::string customer, std::string topping, int slices)
      : customer_(std::move(customer)), topping_(std::move(topping)), slices_(slices) {}

  // +=：给订单增加披萨片数（返回自身引用，与内置 += 一致）
  PizzaOrder& operator+=(int extra) {
    slices_ += extra;
    return *this;
  }

  // ==：三要素完全相同
  bool operator==(const PizzaOrder& other) const {
    return slices_ == other.slices_ && customer_ == other.customer_
        && topping_ == other.topping_;
  }

  // <：按片数比较
  bool operator<(const PizzaOrder& other) const {
    return slices_ < other.slices_;
  }

  // >：借 < 实现（对称写法，保持一致性）
  bool operator>(const PizzaOrder& other) const {
    return other < *this;
  }

  std::string customer() const { return customer_; }
  std::string topping() const { return topping_; }
  int slices() const { return slices_; }

private:
  std::string customer_;
  std::string topping_;
  int slices_;
};

// 非成员 operator<<：只走公有 getter，不需要 friend
std::ostream& operator<<(std::ostream& out, const PizzaOrder& p) {
  return out << p.customer() << ": " << p.slices() << " slices, " << p.topping();
}

int main() {
  PizzaOrder mine{ "Alice", "pepperoni", 4 };
  mine += 2;                                          // 现在 6 片
  std::cout << mine << "\n";                          // Alice: 6 slices, pepperoni

  PizzaOrder yours{ "Bob", "mushroom", 6 };
```

```
    std::cout << (mine == yours ? "same" : "different") << "\n";  // different
    std::cout << (yours < mine ? "yours < mine" : "yours >= mine") << "\n";  // yours >= mine
}
```

代码做什么：把课堂练习 Pizza Order 类补全：`+=` 加片数、`==` 三要素全等、`<` / `>` 按片数比较、`<<` 打印订单，覆盖本讲大部分重载形态。

特性机制解说：这一例浓缩了本讲要点。① `operator+=` 返回 `PizzaOrder&`（自身引用）——与内置 `+=` 语义一致（`a += b` 的结果就是 `a`），这也是 A5 中 `operator+=` 的签名模板；② `==` 用 `&&` 组合所有字段，是"全等"的惯用实现；③ `>` 用 `other < *this` 实现而非另写逻辑——既符合 rule of contrariety 的姊妹精神（比较运算符互为镜像），也保证 `>` 与 `<` 永不矛盾；④ `operator<<` 只调用公有 getter，因此**不需要 friend**——幻灯片明确认可这种写法（"此时 friend 不是必需的，因为我们没碰私有成员"）。最后对照课堂设计准则检查：`+=` 增加片数、`<` 比较片数，全部符合直觉（PoLA）——如果把 `<` 定义成"比较披萨直径"，读者就会一脸问号。

与旧标准（如C++98）的对比

- **运算符重载本身**：是 C++98 就有的经典特性（C++ 继承自 C 的运算符体系 + 类机制），本讲的语法在 C++98 下完全成立。所以这一节的重点不是"新特性替代旧写法"，而是**C++20 对比较运算符的现代化**。
- **C++20 三路比较（spaceship）`<=>`**：写 `auto operator<=>(const User&) const = default;` 即可一键生成 `==`、`!=`、`<`、`<=`、`>`、`>=` 全部六个比较运算符（按成员字典序比较），把 rule of contrariety 手工劳动自动化：`cpp #include <compare> struct Point { int x, y; auto operator<=>(const Point&) const = default; // C++20 }; // 自动获得 == != < <= > >=, Point{1,2} < Point{1,3} 为 true`
- **C++20 比较重写**：`a == b` 找不到匹配时，编译器会尝试把参数反转成 `b == a`（用另一个参数的 `operator==`），对称比较不再需要写两遍；`<` 也有类似的 `<=>` 重写规则。
- **与其他语言的对比**：Java 完全不支持运算符重载（只能 `compareTo`），C#/Python 支持但语法不同（Python 用 `__eq__`、`__lt__` 等特殊方法）。C++ 选择"符号重载 + 与内置类型语法统一"的路线，也因此背负 PoLA 的设计责任。
- **与 lambda/ranges 的关系**：`std::sort`、`std::map` 等现代 STL 用法之所以能优雅工作，正是靠本讲的运算符语义（`<`、`==`、`<<`）——functors（Lecture 11）的 `operator()` 本质上也是运算符重载的一种。

关键要点

- **运算符是"类型含义"的载体**：`min`、`std::set` / `std::map`、`std::cout` 都靠 `operator<` / `==` / `<<` 工作；重载运算符 = 解锁库功能。
- **非成员重载优先**：STL 更偏爱它（左操作数可为非类类型、可对不拥有的类重载）；需要私有成员时用 `friend`。别同时定义成员与非成员同签名版本（歧义）。

- **Rule of contrariety**：`!=` 永远写 `!(*this == other)`；比较运算符成对实现且互为镜像（`>` 用 `other < *this`）。
- **PoLA：语义必须显而易见**：`+` 不能做减法；含义不明显就写具名函数；只在需要时重载。
- **`operator<<` 签名固定**：`std::ostream& operator<<(std::ostream&, const T&)`，必须返回流以支持链式输出。

常见陷阱与注意事项

- **成员与非成员同签名并存**：`bool operator<(const StanfordID&) const;`（成员）与 `bool operator<(const StanfordID&, const StanfordID&);`（非成员）同时存在时，`a < b` 匹配两个候选，产生歧义/未定义行为——二选一。
- **PoLA 违背**：给 `operator+` 塞入减法/拼接等"顺手"语义。别人读代码时会把 `a + b` 理解为加法——语义违反直觉就是 bug 之源（幻灯片原话："你不想定义 operator+ 做集合减法"）。
- **`operator<` 破坏严格弱序**：`std::set` / `std::map` / `std::sort` 都假设 `<` 满足严格弱序（非自反、传递、等价性一致）。若 `<` 只比较部分字段（如只看年龄不看姓名），不同对象可能互相"等价"，元素会被静默丢弃（示例 2 的 Anna）。
- **`==` / `!=` 语义漂移**：分别独立实现 `==` 与 `!=`，忘了取反或漏字段，导致 `a == b` 与 `a != b` 同时为真——务必用 rule of contrariety 一句话定义。
- **`operator<<` 忘记返回流 / 忘加 friend**：不返回 `out` 则链式输出编译失败；非成员实现要访问私有字段却忘了 `friend`，编译器报"private 无法访问"。
- **重载了 `+=` 却忘了返回 `*this`**：复合赋值运算符按惯例返回自身引用（`User&`），便于 `a = b += c` 式链式写法；返回 `void` 会破坏惯例（虽然能编译）。

关联作业提示

本讲直接服务于 **A5: Treebook**（社交网络 `User` 类），三个部分分别对应本讲知识点：

- **Part 1（Viewing Profiles）**：实现 `operator<<`——**必须声明为 `User` 类的 friend 函数**（`user.h` 中 `friend std::ostream& operator<<(std::ostream& out, const User& user);`），并在 `user.cpp` 定义。因为要遍历 `_friends` 私有字段，friend 必不可少（对应示例 4；A5 明确要求输出格式 `User(name=Alice, friends=[Bob, Charlie])`，**不要打印换行符**）。
- **Part 2（Unfriendly Behaviour）**：实现特殊成员函数（析构、拷贝构造、拷贝赋值，删除移动构造/移动赋值）——这是对 Lecture 12 之前"special member functions"内容的巩固，注意深拷贝 `_friends` 指针数组（分配新内存 + 逐个复制 + 更新 `_size` / `_capacity` / `_name`）。
- **Part 3（Always Be Friending）**：实现两个**成员函数**运算符——`User& operator+=(User& rhs)`（把对方加进自己的好友列表，**必须对称**：`alice += charlie` 后 Charlie 的好友列表也要有 Alice，返回 `*this` 引用）与 `bool operator<(const User& rhs) const`（按名字字典序比较，让 `User` 能放进 `std::set`——正是本讲示例 2 的机制：`std::set` 依赖 `operator<` 做有序存储）。

顺带一提：你在 **A4: Ispell** 里其实已经"用过"运算符重载了——`Corpus = std::set<Token>` 要求 `Token` 具备 `operator<`（讲义提供的 `Token` 已实现）。学完本讲再看 A4，你会理解这份代码为什么存在。复习重点：成员 vs 非成员的选择依据（A5 明确要求成员函数）、friend 的声明位置与必要性、`operator+=` 返回自身引用的惯例、以及"为 `std::set` 提供严格弱序的 `<`"。

Lecture 13 (Week 7 – Tuesday): 特殊成员函数 (Special Member Functions)

概述

本讲介绍 C++ 类的六大**特殊成员函数（Special Member Functions, SMFs）**——默认构造函数、析构函数、拷贝构造函数、拷贝赋值运算符、移动构造函数、移动赋值运算符。这些函数编译器会自动生成，但一旦类管理堆内存（如指针成员），默认的"逐成员拷贝"就会造成浅拷贝、双重释放等严重 bug，因此我们需要理解它们何时被调用、何时必须手写。本讲还涵盖成员初始化列表（const/引用成员的唯一初始化途径）、`= delete` 删除函数，以及 Rule of Zero / Three / Five 三条黄金法则。

核心特性与语法详解

1. 六大特殊成员函数（SMFs）总览

- **定义与目的**：特殊成员函数负责类的"生命周期"——对象的创建、复制、移动与销毁。每个类都"天然拥有"这 6 个函数：只要你在使用它们（且没有显式定义过），编译器就会自动生成默认版本。C++ 允许你显式覆盖其中任意一个，从而精确控制类的行为。
- **核心语法**：`cpp T(); // 默认构造函数 ~T(); // 析构函数 T(const T& other); // 拷贝构造函数 T& operator=(const T& other); // 拷贝赋值运算符 T(T&& other); // 移动构造函数（本讲先了解，下讲详解） T& operator=(T&& other); // 移动赋值运算符`
- **设计意图与最佳实践**：编译器免费生成的版本对大多数"自管理"的类已经足够（见 Rule of Zero）；只有当默认行为不正确时（典型场景：类持有指向堆内存的裸指针）才需要手写。手写时遵循 Rule of Three/Five。

2. 拷贝构造 vs 拷贝赋值：触发时机

- **定义与目的**：两者都做"拷贝"，但语义完全不同，是最容易混淆的一对。
- **核心语法**：`cpp Widget widgetTwo = widgetOne; // 拷贝构造：新建对象时初始化 Widget a, b; a = b; // 拷贝赋值：对象已存在，替换内容`
- **设计意图与最佳实践**：判断口诀——**等号右侧对象是否"刚被创建"**。如果是新对象的初始化（`T b = a;`、按值传参、按值返回），走拷贝构造；如果两边都已存在（`a = b;`），走拷贝赋值。特别注意 `T b = a;` 是拷贝构造**不是**赋值！另外，`T b(a);` 也是拷贝构造。

3. 成员初始化列表（Member Initializer List）

- **定义与目的**：在构造函数体**执行之前**直接以期望的值构造成员变量。若在函数体内赋值，成员会先被默认初始化、再被赋值，等于做了两遍工作（低效）；更严重的是，**const 成员和引用成员只能通过初始化列表初始化**，因为它们在诞生那一刻就必须有值、之后不可改。
- **核心语法**：`cpp template <typename T> Vector<T>::Vector() : _size(0), _capacity(4), _data(new T[_capacity]) {}`
- **设计意图与最佳实践**：所有构造函数（包括带参的）都应优先使用初始化列表；成员按**声明顺序**（而非列表书写顺序）初始化，因此列表书写顺序最好与声明顺序一致，避免混淆。

4. 浅拷贝 vs 深拷贝

- **定义与目的**：默认的拷贝构造/拷贝赋值对每个成员做"逐成员拷贝（member-wise copy）"。当成员是指针时，逐成员拷贝只复制**指针值本身**，两个对象将指向**同一块**堆内存——这就是**浅拷贝**。任何一方修改或销毁数据都会影响另一方，析构时还会**双重释放**。**深拷贝**则分配一块新的内存并把数据逐个复制过去，得到完全独立的一份。
- **核心语法**：`cpp // 浅拷贝（编译器默认，危险！） Vector(const Vector& other) : _size(other._size), _capacity(other._capacity), _data(other._data) {} // 深拷贝（手写，正确） Vector(const Vector& other) : _size(other._size), _capacity(other._capacity), _data(new T[other._capacity]) { for (size_t i = 0; i < _size; ++i) _data[i] = other._data[i]; }`
- **设计意图与最佳实践**：只要类拥有"独享"的堆资源，就必须手写深拷贝版本的拷贝构造与拷贝赋值，同时手写析构释放资源（三者成组出现，即 Rule of Three）。

5. `= delete` 删除函数

- **定义与目的**：显式地**移除**某个特殊成员函数（或普通函数）的功能。典型用途：禁止拷贝（如 `std::unique_ptr` 就是删除拷贝、只保留移动）、禁止默认构造、禁止某些重载参与重载决议。
- **核心语法**：`cpp class PasswordManager { public: PasswordManager(const PasswordManager&) = delete; // 禁止拷贝构造 PasswordManager& operator=(const PasswordManager&) = delete; // 禁止拷贝赋值 };`
- **设计意图与最佳实践**：任何尝试拷贝的代码都会在**编译期**报错，把错误从运行时提前到编译期。对比 C++98 把拷贝构造放进 `private` 且不实现的"hack"，`= delete` 意图清晰、报错信息友好。

6. Rule of Zero / Three / Five

- **定义与目的**：回答"我到底要手写几个特殊成员函数？"的经验法则。
- **核心语法**（三条规则）：
- **Rule of Zero**：如果默认生成的 SMF 就能正确工作（成员都是自管理的类型，如 `std::string`、`std::vector`），就一个都别写。
- **Rule of Three**：如果你需要自定义析构函数（通常意味着手动管理资源），那么也必须自定义拷贝构造和拷贝赋值。只写析构、不写拷贝，浅拷贝/双重释放就会乘虚而入。

- **Rule of Five**：如果定义了拷贝构造/拷贝赋值/析构（Rule of Three 三者之一），通常还应定义移动构造和移动赋值，否则代码会退回到昂贵的拷贝路径（性能问题，非正确性问题）。
- **设计意图与最佳实践**：从"默认能用就不写"（Zero）出发；一旦涉及手工资源管理，则按 Three 补齐拷贝、按 Five 再补移动。

代码示例与逐步解说（核心）

示例 1：成员初始化列表与 const/引用成员（C++11）

```
// C++11
#include <iostream>

class MyClass {
    const int _constant;   // const 成员：只能初始化，不能赋值
    int& _reference;       // 引用成员：诞生时必须绑定对象
public:
    // 只有成员初始化列表能初始化 const 和引用成员
    MyClass(int value, int& ref) : _constant(value), _reference(ref) {}

    void print() const {
        std::cout << "constant = " << _constant
                  << ", reference = " << _reference << '\n';
    }
};

int main() {
    int x = 42;
    MyClass obj(7, x);
    obj.print();          // constant = 7, reference = 42
    x = 100;              // 修改外部变量 x
    obj.print();          // constant = 7, reference = 100（引用跟着变）
}
```

- **代码做什么**：`MyClass` 有两个"非可赋值"成员——`const int` 和 `int&`。构造函数用初始化列表 `_constant(value), _reference(ref)` 在成员诞生时直接赋值。main 中构造 `obj` 并打印两次，第二次打印前外部变量 `x` 被改成 100，于是 `_reference` 显示 100，而 `_constant` 保持 7。
- **特性机制解说**：`const` 成员一旦被赋值就不能再改，引用成员必须在初始化时绑定目标——两者都"不可重新赋值"。如果在构造函数体内写 `_constant = value;`，此时成员已经被默认初始化过了（const 成员的默认初始化甚至不合法），必然编译失败。这正是初始化列表存在的意义：**在成员构造那一刻就给出初值**，一步到位，避免"先默认构造、再赋值"的双倍开销。

示例 2：Vector 的深拷贝（改写自幻灯片与课堂 Vector）（C++11）

```cpp
// C++11
#include <iostream>

template <typename T>
class Vector {
public:
    // 默认构造：初始化列表一步到位
    Vector() : _size(0), _capacity(4), _data(new T[_capacity]) {}

    // 拷贝构造：深拷贝
    Vector(const Vector& other)
        : _size(other._size), _capacity(other._capacity),
          _data(new T[other._capacity]) {
        for (size_t i = 0; i < _size; ++i) {
            _data[i] = other._data[i];
        }
    }

    // 拷贝赋值：先释放自己的旧内存，再深拷贝
    Vector& operator=(const Vector& other) {
        if (this == &other) return *this;      // 自赋值保护
        delete[] _data;
        _size = other._size;
        _capacity = other._capacity;
        _data = new T[_capacity];
        for (size_t i = 0; i < _size; ++i) {
            _data[i] = other._data[i];
        }
        return *this;
    }

    // 析构：释放堆数组
    ~Vector() { delete[] _data; }

    void push_back(const T& value) {
        if (_size == _capacity) {
            _capacity *= 2;
            T* bigger = new T[_capacity];
            for (size_t i = 0; i < _size; ++i) bigger[i] = _data[i];
            delete[] _data;
            _data = bigger;
        }
        _data[_size++] = value;
    }

    size_t size() const { return _size; }
    T& operator[](size_t i) { return _data[i]; }

private:
    size_t _size;
    size_t _capacity;
    T* _data;
};
```

```
int main() {
    Vector<int> v;
    v.push_back(10);
    v.push_back(20);

    Vector<int> w = v;      // 拷贝构造：深拷贝
    w[0] = 99;              // 只修改 w，不影响 v

    std::cout << v[0] << ' ' << v[1] << '\n';   // 10 20
    std::cout << w[0] << ' ' << w[1] << '\n';   // 99 20

    Vector<int> u;
    u = v;                  // 拷贝赋值
    std::cout << u[0] << ' ' << u[1] << '\n';   // 10 20
}
```

- **代码做什么**：`v` 里存了两个元素；`Vector<int> w = v;` 触发**拷贝构造**，`w` 获得独立的内存并逐个复制元素；修改 `w[0]` 不影响 `v`。`u = v;` 触发**拷贝赋值**：先 `delete[]` 掉 `u` 原来的数组，再分配新内存并复制。
- **特性机制解说**：拷贝构造的 `_data(new T[other._capacity])` 是关键——**为副本分配全新的内存**，然后循环逐元素复制，这就是深拷贝，代价是 O(n)。而默认的逐成员拷贝会把 `_data` 原样复制，导致 `v` 和 `w` 共享同一数组：修改互相影响，且两个析构函数会 `delete[]` 同一块内存两次——双重释放（double free），属于未定义行为。拷贝赋值还必须先 `delete[]` 自己的旧数据，否则会泄漏旧内存；自赋值检查 `if (this == &other)` 防止 `u = u` 时"先释放再复制"把数据弄丢。

示例 3：Pirate 课堂练习完成版——深拷贝全家桶（C++11）

```cpp
// C++11（课堂 pirate.cpp 练习的完成版）
#include <iostream>
#include <string>

class Treasure {
public:
    std::string name;
    int goldValue;
    Treasure(std::string name, int goldValue)
        : name(name), goldValue(goldValue) {}
};

class Pirate {
private:
    Treasure* treasure;   // 指向堆上的 Treasure

public:
    // 1. 默认构造：分配一块新的 Treasure
    Pirate() : treasure(new Treasure("Rusty Spoon", 1)) {}

    // 2. 带参构造
    Pirate(std::string itemName, int value)
        : treasure(new Treasure(itemName, value)) {}

    // 3. 拷贝构造：深拷贝！
    Pirate(const Pirate& other)
        : treasure(new Treasure(other.treasure->name,
                                other.treasure->goldValue)) {}

    // 4. 拷贝赋值：自赋值检查 → 释放旧的 → 深拷贝
    Pirate& operator=(const Pirate& other) {
        if (this == &other) return *this;
        delete treasure;
        treasure = new Treasure(other.treasure->name,
                                other.treasure->goldValue);
        return *this;
    }

    // 5. 析构：释放堆内存
    ~Pirate() { delete treasure; }

    void renameTreasure(std::string newName) { treasure->name = newName; }
    void upgradeTreasure(int extraGold) { treasure->goldValue += extraGold; }
    void print() const {
        std::cout << treasure->name << " worth "
                  << treasure->goldValue << " gold\n";
    }
};

int main() {
    Pirate b("Golden Crown", 500);
    Pirate c = b;                 // 拷贝构造：深拷贝
    c.renameTreasure("Fake Crown");
```

```
    c.upgradeTreasure(-400);
    b.print();                    // Golden Crown worth 500 gold（b 不受影响）
    c.print();                    // Fake Crown worth 100 gold

    Pirate d("Broken Bottle", 2);
    d = b;                        // 拷贝赋值：先清理再深拷贝
    d.renameTreasure("Stolen Crown");
    d.upgradeTreasure(300);
    b.print();                    // Golden Crown worth 500 gold
    d.print();                    // Stolen Crown worth 800 gold

    b = b;                        // 自赋值：安全，什么都不发生
}
```

- **代码做什么**：`Pirate c = b;` 用深拷贝构造出独立的海盗；改 `c` 的宝物（改名、改价）不会动 `b`。`d = b;` 用拷贝赋值把 `b` 的内容复制进已有的 `d`；`b = b` 自赋值被 `if (this == &other) return *this;` 拦住。
- **特性机制解说**：`Pirate` 的成员是裸指针 `Treasure*`，编译器默认的浅拷贝会让两个海盗**共享同一份宝物**——改一个另一个跟着变，而且析构时双重释放。深拷贝为每个副本 `new` 一块独立内存，使拷贝成为完全独立的个体。拷贝赋值比拷贝构造多两步：**先释放自己原有的资源**（否则旧宝物泄漏），**再检查自赋值**（否则 `b = b` 会先 delete 掉自己的宝物再读它，读到悬垂指针）。三个函数（析构 + 拷贝构造 + 拷贝赋值）必须成套出现，正是 Rule of Three。

示例 4：浅拷贝的双重释放与 `= delete`（C++11）

```
// C++11（危险示例，仅用于演示浅拷贝）
#include <iostream>

class Shallow {
    int* data;
public:
    explicit Shallow(int v) : data(new int(v)) {}
    ~Shallow() { delete data; }   // 自定义析构，但拷贝仍是默认的逐成员拷贝！
    int get() const { return *data; }
};

int main() {
    Shallow a(42);
    Shallow b = a;        // 默认拷贝构造：b.data 与 a.data 指向同一块内存
    std::cout << b.get() << '\n';   // 42（暂时正常）
}   // 析构顺序：b 先析构 → delete 共享内存；a 再析构 → 再次 delete → 双重释放！
```

```
// C++11：用 = delete 阻止拷贝
#include <iostream>

class PasswordManager {
public:
    PasswordManager() = default;
    PasswordManager(const PasswordManager&) = delete;            // 禁止拷贝构造
    PasswordManager& operator=(const PasswordManager&) = delete; // 禁止拷贝赋值
};

int main() {
    PasswordManager pm;
    // PasswordManager pm2 = pm;   // ❌ 编译错误：拷贝构造已删除
    // pm = PasswordManager();     // ❌ 编译错误：拷贝赋值已删除
    std::cout << "Copies are disabled.\n";
}
```

- **代码做什么**：第一个程序能编译、能打印 42，但程序结束时会**双重释放**（double free）——运行时崩溃或未定义行为。第二个程序把拷贝构造/拷贝赋值标为 `= delete`，任何拷贝尝试都变成编译错误。
- **特性机制解说**：`Shallow` 定义了析构却没有定义拷贝，编译器照样生成逐成员拷贝，于是两个对象共享堆内存。局部对象按"后声明先析构"的顺序销毁：`b` 先 `delete`，`a` 再 `delete` 同一块内存——第二遍 delete 是未定义行为。这演示了 Rule of Three 的反例：**只写析构、不写拷贝 = 双重释放**。`= delete` 与"不定义"不同：它是**显式地**让函数参与重载决议但一调用就报编译错误，比 C++98 的 private 技巧更清晰、更早暴露问题。`std::unique_ptr` 正是靠删除拷贝、保留移动来实现"唯一所有权"的。

示例 5：Rule of Zero——什么都不用写（C++11）

```
// C++11: Rule of Zero
#include <iostream>
#include <string>
#include <utility>

class a_string_with_an_id {
public:
    a_string_with_an_id(int id, std::string str)
        : id_(id), str_(std::move(str)) {}

    void print() const {
        std::cout << id_ << ": " << str_ << '\n';
    }

private:
    int id_;
    std::string str_;   // std::string 是自管理类型
};

int main() {
    a_string_with_an_id a(1, "hello");
    a_string_with_an_id b = a;                  // 自动生成的拷贝构造
    b.print();                                  // 1: hello
    a_string_with_an_id c = std::move(a);       // 自动生成的移动构造
    c.print();                                  // 1: hello
}
```

- **代码做什么**：类只含 `int` 和 `std::string` 两个自管理成员，因此一行 SMF 都不用写。拷贝构造、拷贝赋值、移动构造、移动赋值、析构全部由编译器自动生成且行为完全正确。
- **特性机制解说**：编译器生成的 SMF 会**递归调用成员自己的 SMF**：拷贝 `str_` 时调用 `std::string` 的拷贝构造（深拷贝其内部缓冲），析构时调用 `std::string` 的析构（释放其内部缓冲）。因为 `std::string` 自己已经正确实现了全套 SMF，外层类就无需重复实现——这就是 Rule of Zero 的底层原理：**把资源管理委托给自管理的成员，自己什么都不写**。

与旧标准（如C++98）的对比

- **`= delete` 是 C++11 才有的**：C++98 中要禁止拷贝，只能把拷贝构造/拷贝赋值声明在 `private` 区且不给定义；"意外调用"会得到晦涩的链接错误，且类内部/友元仍可能误用。C++11 的 `= delete` 把禁止意图写在声明处，任何调用（包括类内）都是清晰的编译错误。
- **移动构造/移动赋值是 C++11 才有的**：C++98 只有"默认构造 + 析构 + 拷贝构造 + 拷贝赋值"四个 SMF（即 Rule of Three），一切"转移"都只能靠拷贝模拟，性能浪费严重。C++11 新增移动语义（详见 Lecture 14）。
- **成员初始化列表 C++98 已有**：初始化列表不是新特性，但 C++98 中 const/引用成员同样只能靠它初始化；现代 C++ 只是更强调"初始化优于赋值"（统一初始化、`{}` 语法等）。

- **C++98 的替代手段**：想要"只允许一份实例"时，C++98 的做法是私有拷贝 + 友元或单例模式；现代 C++ 直接用 `= delete` 更简单直接。

关键要点

- 六大特殊成员函数（默认构造、析构、拷贝构造、拷贝赋值、移动构造、移动赋值）由编译器按需自动生成；**手写任何一个拷贝/析构相关函数时，就要想想是否该成套补齐**。
- 判断拷贝构造还是拷贝赋值：**新对象初始化 → 拷贝构造；两边都已存在 → 拷贝赋值**；`T b = a;` 是拷贝构造而不是赋值。
- 涉及裸指针/堆内存时，默认的浅拷贝会造成共享内存、双重释放；必须手写**深拷贝**（分配新内存 + 逐元素复制）。
- 用**成员初始化列表**初始化成员：更快，而且是 const/引用成员唯一合法的初始化方式。
- 用 `= delete` 在编译期禁止不需要的操作（如 `std::unique_ptr` 禁止拷贝）；遵循 Rule of Zero（默认能用就不写）→ Three（写析构就补拷贝）→ Five（再补移动）。

常见陷阱与注意事项

- **忘记深拷贝**：只写了析构没写拷贝构造/拷贝赋值（违反 Rule of Three），导致浅拷贝共享内存、双重释放。写"Big Three/Four"时要成套。
- **忘记自赋值检查**：拷贝赋值里若先 `delete` 自己的资源再复制 `other`，遇到 `a = a` 时会读悬垂指针。开头加 `if (this == &other) return *this;`。
- **`T b = a;` 误以为是"赋值"**：这是拷贝构造！真正触发拷贝赋值的是 `b = a;`（b 已存在）。
- **最令人头疼的解析（Most Vexing Parse）**：`Widget w();` 声明的是一个返回 `Widget` 的函数，而不是默认构造！想要默认构造请写 `Widget w;` 或 `Widget w{};`（幻灯片小测中的 `vector<int> vec4 ();` 即为此陷阱）。
- **在构造函数体内给成员赋值而非用初始化列表**：低效（先默认构造再赋值），且 const/引用成员直接编译失败。
- **拷贝赋值中忘记释放旧资源**：直接 `_data = new ...` 会让旧内存泄漏。

关联作业提示

本讲与 **A5: Treebook**（`assign5/`）直接相关：作业要求你在 `user.h` / `user.cpp` 中为 `User` 类（内含裸指针数组 `_friends`）实现/删除特殊成员函数：

1. **实现析构** `~User()`——释放 `_friends` 数组（本讲"析构释放堆内存"）。
2. **实现拷贝构造** `User(const User& user)`——在成员初始化列表中分配新的 `_friends` 数组，再循环复制元素，并设置 `_size`、`_capacity`、`_name`（本讲 Vector/Pirate 深拷贝的完全同款写法）。

3. **实现拷贝赋值** `User& operator=(const User& user)`——先释放旧数组、深拷贝新数组、返回 `*this`，记得自赋值检查。
4. **删除移动构造** `User(User&&) = delete;` **与移动赋值** `User& operator=(User&&) = delete;`——用本讲的 `= delete` 语法（与 `std::unique_ptr` 相反：unique_ptr 保留移动、删除拷贝；Treebook 要求删除移动、保留拷贝）。
5. Part 3 的 `operator<<`（friend 函数，访问私有 `_friends`）和 `operator<`、`operator+=` 则呼应了上一讲的重载内容。

编译命令 `g++ -std=c++20 main.cpp user.cpp -o main` 中的多文件编译会在 Lecture 16 详解。

Lecture 14 (Week 7 – Thursday): 移动语义 (Move Semantics)

概述

上一讲我们学会了拷贝（拷贝构造/拷贝赋值），但**拷贝数据是非常昂贵的**——想想从 `takePhoto()` 这样的函数返回一个包含 3840×2160 像素的照片对象：临时对象马上要被销毁，却还要先深拷贝一遍它的数据。本讲引入**左值（lvalue）与右值（rvalue）**的概念（区分"持久对象"与"临时对象"），并用**右值引用** `T&&` 重载特殊成员函数，实现**移动语义**：把临时对象的资源"偷"过来而不是复制，把 O(n) 的拷贝变成 O(1) 的指针窃取。最后介绍 `std::move`（它只是类型转换，不是移动本身）与 Rule of Zero / Three / Five。

核心特性与语法详解

1. 拷贝的代价与问题场景

- **定义与目的**：按值传递/返回、容器扩容等场景都会触发深拷贝，每次拷贝都要重新分配内存并逐元素复制（O(n)）。当源对象是**即将销毁的临时量**时，这份拷贝完全是浪费——我们真正需要的只是"转移资源所有权"。
- **核心语法**（问题场景）：
```
cpp Photo selfie = takePhoto(); // takePhoto() 返回临时对象，马上要销毁
// 却要先深拷贝它的像素数据，然后销毁原对象：纯浪费
```
- **设计意图与最佳实践**：区分两种情况——**对持久对象（可能以后还要用）做拷贝；对临时对象（用完即弃）做移动**。这就是移动语义要解决的问题。

2. 左值（lvalue）与右值（rvalue）

- **定义与目的**：C++ 需要一种机制来判断一个表达式是"持久的"还是"临时的"，从而决定走拷贝还是移动。左值/右值就是这种"临时性（temporariness）"的泛化。

- **核心语法**：`cpp void foo(Photo pic) { Photo* p1 = &pic; // ✅ pic 是左值：可以取地址 Photo* p2 = &takePhoto(); // ❌ 编译错误：takePhoto() 是右值，无地址 }`
- **设计意图与最佳实践**：直观判断标准——**能否取地址**：有确定地址的是左值，没有的是右值。另一个视角：**左值可以出现在 `=` 两侧，右值只能出现在右侧**（`5 = y;` 不合法）。生命周期上，左值活到作用域结束，右值活到语句结束。幻灯片还给出了一个更精细的分类图（glvalue/rvalue → lvalue/xvalue/prvalue），但课程明确说"别担心这个"，掌握左值/右值两层就够用。注意 `int& b = a;` 中的 `b` 是左值引用，它引用左值。

3. 右值引用 `T&&` 与左值/右值重载

- **定义与目的**：左值引用 `Type&` 只能绑定左值；右值引用 `Type&&` 只能绑定右值（临时量）。通过**重载** `&` 和 `&&` 两个版本的同名函数，编译器就能根据实参是左值还是右值自动选择版本。
- **核心语法**：`cpp void upload(Photo& pic); // 左值版本：pic 是持久的，必须保持有效 void upload(Photo&& pic); // 右值版本：pic 是临时的，可以偷走它的资源`
- **设计意图与最佳实践**：右值引用参数告诉我们"这个对象用完即弃"，因此我们可以随意窃取它的资源，把它置于任意状态（甚至无效）都没关系。这是移动构造/移动赋值的语法基础。

4. 移动构造与移动赋值

- **定义与目的**：让新对象/已有对象**窃取**临时对象的资源（通常是复制指针、接管堆内存），并把源对象置为"空"（如 `nullptr`），这样源对象析构时不会破坏被偷走的数据。复杂度 O(1)，与 O(n) 的深拷贝形成鲜明对比。
- **核心语法**：`cpp T(T&& other) noexcept; // 移动构造 T& operator=(T&& other) noexcept; // 移动赋值` `cpp // 移动构造：偷指针 + 源对象置空 Photo::Photo(Photo&& other) noexcept : width(other.width), height(other.height), data(other.data) { other.data = nullptr; // 关键！源对象析构时 delete nullptr 无害 }`
- **设计意图与最佳实践**：移动操作的实现套路固定——**（1）偷走源对象的资源指针；（2）把源对象置为安全空状态（valid-but-unspecified）**。务必加 `noexcept`：容器（如 `std::vector`）扩容时，若移动构造可能抛异常，它会**回退到拷贝**以保证强异常安全，导致性能白白损失。移动后源对象仍是一个"合法但状态未指定"的对象：可以析构、可以重新赋值，但不能假设它还有原来的内容。

5. `std::move`：只是类型转换

- **定义与目的**：编译器只在遇到**右值**时才自动选移动。但我们有时**明确知道某个左值以后不会再用了**（例如数组平移 `elems[i] = elems[i-1]` 后，旧位置不再需要），此时需要"手动把左值变成右值"来强制触发移动。
- **核心语法**：`cpp elems[i] = std::move(elems[i - 1]); // 把左值转成右值引用，触发移动赋值`
- **设计意图与最佳实践**：`std::move(x)` **不做任何移动**，它只是 `static_cast<T&&>(x)`——把左值 `x` 的类型转换成右值引用，从而让重载决议选中移动版本。它就像 `const_cast` 一样是"opt-in"：**你主动声明"这个对象我不要了"**。因此：除非有充分理由（性能关键、确定不再使用），否则不要到处乱用

`std::move`；移动后再使用源对象是未定义行为级别的错误（实际是 valid-but-unspecified，但语义上不应再用）。

6. Rule of Zero / Three / Five

- **定义与目的**：定义移动后，"要不要都写"的规则升级为五件套。
- **核心语法**：
- **Rule of Zero**：类不管理外部资源时，编译器生成的 SMF 全部够用，一个都不写。
- **Rule of Three**：需要自定义析构（管理外部资源）时，必须同时自定义拷贝构造和拷贝赋值；否则两个对象会共享底层资源。
- **Rule of Five**：定义了 Three 中任何一个时，还应（非强制但强烈建议）定义移动构造和移动赋值；否则移动会退化为拷贝，性能变差。
- **设计意图与最佳实践**：`struct Post { Photo photo; std::string caption; };` 是 Rule of Zero 的范例——编译器生成的 Post 的 SMF 会自动调用 `Photo` 和 `std::string` 各自的 SMF。反之，管理外部资源的类（如 Photo 的 `int* data`）必须遵守 Three/Five。

代码示例与逐步解说（核心）

示例 1：左值 vs 右值——取地址测试（C++11）

```
// C++11
#include <iostream>

int getFive() { return 5; }   // 返回临时值

int main() {
    int a = 4;
    int* p1 = &a;             // ✅ a 是左值，有地址
    std::cout << "&a = " << p1 << '\n';

    // int* p2 = &getFive();  // ❌ 编译错误：getFive() 的返回值是右值（临时量），没有地址

    int* p3 = &a;             // a 可以出现在 = 左侧：a = 5; ✅
    // getFive() = 5;         // ❌ 右值不能出现在 = 左侧
    (void)p3;
}
```

- **代码做什么**：演示判断左值/右值的两个标准：能否取地址、能否出现在 `=` 左侧。`a` 可以，函数返回值不行。
- **特性机制解说**：左值有确定的存储位置（地址），生命周期到作用域结束；右值（此处是 prvalue——纯右值，如函数返回值、字面量）没有地址，生命周期到语句结束。注意：虽然标准里还有 xvalue 等细分（`std::move(x)` 的结果就是 xvalue，属于 glvalue，理论上可取地址），但课程层面只需记住"临时对象是右值、持久对象是左值"。编译器正是靠这个分类来决定调用哪个重载。

示例 2：左值/右值重载——`&` 与 `&&`（C++11）

```
// C++11
#include <iostream>
#include <string>

void upload(const std::string& pic) {   // 左值重载：持久对象，只能引用
    std::cout << "upload(lvalue&): " << pic << '\n';
}

void upload(std::string&& pic) {        // 右值重载：临时对象，可以偷资源
    std::cout << "upload(rvalue&&): " << pic << '\n';
}

std::string takePhoto() { return "selfie.jpg"; }

int main() {
    std::string selfie = takePhoto();
    upload(selfie);         // 左值 → 调用第一个重载
    upload(takePhoto());    // 右值 → 调用第二个重载
    upload("direct");       // 字符串字面量隐式转为临时 std::string → 右值重载
}
```

- **代码做什么**：定义同名 `upload` 的两个重载，实参为左值时走 `&` 版本，为右值时走 `&&` 版本，编译器自动选择。
- **特性机制解说**：重载决议根据实参的"值类别（value category）"挑选版本：左值绑定 `&`，右值绑定 `&&`（const 左值引用也能绑定右值，但**非 const** 左值引用不能）。这正是移动语义的钥匙：函数拿到 `Type&&` 参数时，就知道"这个对象是临时的、可以破坏"，于是可以偷走它的资源而不必拷贝。幻灯片中 `uploadToInsta(takePhoto())` 只传左值引用会报 "candidate function not viable: expects lvalue" 的编译错误，正是因为没有 `&&` 版本可匹配。

示例 3：Photo 类——拷贝 vs 移动（改写自幻灯片）（C++11）

```
// C++11
#include <algorithm>
#include <iostream>

class Photo {
public:
    Photo(int w, int h) : width(w), height(h), data(new int[w * h]) {}
    ~Photo() { delete[] data; }

    // 拷贝构造：深拷贝 (O(n))
    Photo(const Photo& other)
        : width(other.width), height(other.height),
          data(new int[width * height]) {
        std::copy(other.data, other.data + width * height, data);
        std::cout << "copy ctor (O(n))\n";
    }

    // 移动构造：偷指针 (O(1))
    Photo(Photo&& other) noexcept
        : width(other.width), height(other.height), data(other.data) {
        other.data = nullptr;   // 源对象置空：析构 delete nullptr 无害
        std::cout << "move ctor (O(1))\n";
    }

    // 拷贝赋值：先释放旧数据，再深拷贝
    Photo& operator=(const Photo& other) {
        if (this == &other) return *this;
        delete[] data;
        width = other.width;
        height = other.height;
        data = new int[width * height];
        std::copy(other.data, other.data + width * height, data);
        return *this;
    }

    // 移动赋值：释放自己的旧数据，偷走对方的
    Photo& operator=(Photo&& other) noexcept {
        if (this == &other) return *this;
        delete[] data;          // 先清掉自己原来的像素
        width = other.width;
        height = other.height;
        data = other.data;      // 偷走对方的数据
        other.data = nullptr;   // 源对象置空
        return *this;
    }

    int get(int i) const { return data[i]; }

private:
    int width;
    int height;
    int* data;
};
```

```
Photo takePhoto() { return Photo(10, 10); }

int main() {
    Photo selfie = takePhoto();   // 移动构造（临时对象）——编译器可能进一步做 RVO 省略
    Photo pic = selfie;           // 拷贝构造（左值）
    pic = takePhoto();            // 移动赋值（临时对象）
    pic = selfie;                 // 拷贝赋值（左值）
}
```

- **代码做什么**：`Photo selfie = takePhoto();` 源是临时对象 → **移动构造**，把 `data` 指针直接拿过来（O(1)）；`Photo pic = selfie;` 源是左值 `selfie` → **拷贝构造**（O(n)）；`pic = takePhoto();` → 移动赋值；`pic = selfie;` → 拷贝赋值。每个操作打印自己的名字。
- **特性机制解说**：移动构造只做三件事——复制宽高、**复制指针**、**把源对象指针置 `nullptr`**。置空至关重要：`takePhoto()` 的临时对象在语句结束时析构，`delete[] data` 对 `nullptr` 是空操作，于是被偷走的内存安全存活。若不置空，临时对象的析构会把刚偷来的数据 delete 掉（幻灯片中的 "Oh no… the destructor deletes our stolen data" 场景）。**移动赋值**额外多一步：先 `delete[]` 自己的旧数据，否则旧像素泄漏。移动后源对象处于 **valid-but-unspecified** 状态——可以安全析构/重新赋值，但内容未指定。另外注意：`Photo selfie = takePhoto();` 在开启优化的编译器上可能被 **RVO（Return Value Optimization）** 完全省略（直接在 `selfie` 处构造），连移动构造都不调用——这是允许的优化，本讲先按"会调用移动构造"来理解，RVO 细节下节再谈。

示例 4：完整 String 类——移动构造与移动赋值（含 noexcept、源对象置空）（C++11）

```cpp
// C++11：完整 String 类（深拷贝 + 移动）
#include <cstring>
#include <iostream>
#include <utility>

class String {
public:
    String() : buf_(nullptr), size_(0) {}

    String(const char* s) : size_(std::strlen(s)) {
        buf_ = new char[size_ + 1];
        std::strcpy(buf_, s);
    }

    // 拷贝构造：深拷贝（O(n)）
    String(const String& other) : size_(other.size_) {
        buf_ = new char[size_ + 1];
        std::strcpy(buf_, other.buf_);
        std::cout << "copy ctor\n";
    }

    // 拷贝赋值：自赋值检查 → 释放旧 → 深拷贝
    String& operator=(const String& other) {
        if (this == &other) return *this;
        delete[] buf_;
        size_ = other.size_;
        buf_ = new char[size_ + 1];
        std::strcpy(buf_, other.buf_);
        return *this;
    }

    // 移动构造：偷指针 + 源对象置空（O(1)）
    String(String&& other) noexcept
        : buf_(other.buf_), size_(other.size_) {
        other.buf_ = nullptr;
        other.size_ = 0;
        std::cout << "move ctor\n";
    }

    // 移动赋值：释放自己的旧资源 → 偷对方的 → 源对象置空
    String& operator=(String&& other) noexcept {
        if (this == &other) return *this;
        delete[] buf_;
        buf_ = other.buf_;
        size_ = other.size_;
        other.buf_ = nullptr;
        other.size_ = 0;
        return *this;
    }

    ~String() { delete[] buf_; }

    void print() const { std::cout << (buf_ ? buf_ : "(empty)") << '\n'; }
```

```
    std::size_t size() const { return size_; }

private:
    char* buf_;
    std::size_t size_;
};

String makeString() { return String("hello"); }

int main() {
    String s1("CS106L");
    String s2 = s1;               // 拷贝构造（左值）
    String s3 = std::move(s1);    // 移动构造（std::move 把左值转成右值）
    s1.print();                   // (empty)：s1 已被掏空，处于 valid-but-unspecified 状态

    String s4;
    s4 = makeString();            // 移动赋值（函数返回值是临时对象）
    s4.print();                   // hello

    s4 = s2;                      // 拷贝赋值（左值）
    s4.print();                   // CS106L
}
```

- **代码做什么**：完整实现六大 SMF 的 String 类。`s2 = s1` 拷贝（深拷贝）；`s3 = std::move(s1)` 强制移动（s1 被掏空）；`s4 = makeString()` 移动赋值（源是临时对象）；`s4 = s2` 拷贝赋值。每次构造/赋值都打印自己的类别。
- **特性机制解说**：**移动构造**的机制：`String(String&& other)` 的参数是右值引用，`other.buf_` 指向堆上的字符数组；移动构造直接把这个指针复制给自己（O(1)），然后把 `other.buf_` 置 `nullptr`、`other.size_` 置 0。于是源对象析构时 `delete[] nullptr` 空操作，数据安全转移。**移动赋值**额外先 `delete[] buf_` 释放自己原有的旧缓冲（否则旧字符串泄漏），再偷指针、置空源对象。**`noexcept` 的意义**：`std::vector<String>` 扩容需要把元素搬到新数组，若移动构造可能抛异常，vector 会**退化为拷贝**以保证异常安全（旧元素不能处于半搬状态）；标了 `noexcept` 后 vector 才敢用移动，性能才不会白丢。**valid-but-unspecified**：被移动的 `s1` 仍可析构、可重新赋值、可调用不依赖内容的成员（如 `print()`），但内容未指定——所以移动后**不要假设源对象还有原来的值**。

示例 5：`std::move` 只是类型转换 + 移动后慎用源对象（C++11）

```
// C++11
#include <iostream>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::vector<std::string> elems = {"a", "b", "c"};

    // 场景：数组平移，旧位置的值不再需要 → 强制移动而不是拷贝
    elems[2] = std::move(elems[1]);   // std::move 只是把 elems[1] 转成右值
    std::cout << elems[2] << '\n';    // b

    // 反例：移动后继续使用源对象
    std::string a = "treasure";
    std::string b = std::move(a);
    std::cout << "b = " << b << '\n';
    std::cout << "a = " << a << '\n'; // 合法，但内容是"未指定"的（libstdc++ 下通常是空串）
    // 千万不要在移动后调用依赖 a 内容的逻辑，例如 a.size() > 0 的假设！
}
```

- **代码做什么**：第一个场景模拟幻灯片中 `PhotoCollection::insert` 的数组平移——`elems[i] = std::move(elems[i-1])` 把"不再使用的旧位置"的内容移走而不是拷贝。第二个场景展示移动后源对象 `a` 仍可读取但内容未指定。
- **特性机制解说**：`std::move(x)` 的实现等价于 `static_cast<T&&>(x)`——**它不移动任何东西，只是把左值 x 的类型标注为右值引用**，使后续的重载决议选中移动版本。移动真正发生的地方是移动构造/移动赋值函数体里。这解释了为什么"std::move 一个没有移动构造的类型"会退化为拷贝。**Be wary of std::move**：一旦你对左值调用了 `std::move`，就等于向编译器承诺"这个对象我不再需要了"；此后继续使用它（如幻灯片中的 `whoAmI.get_pixel(21, 24)`）可能解引用空指针。课程建议：除非性能关键且你确定对象不再被使用，否则不要显式使用 `std::move`。

与旧标准（如C++98）的对比

- **移动语义、右值引用、`std::move` 全部是 C++11 的新特性**：C++98 里没有 `T&&`，没有移动构造/移动赋值。按值返回大对象时只能深拷贝，函数返回临时对象= 拷贝 + 析构，性能损失无法避免。
- **C++98 的替代方案**：
- **`std::auto_ptr` (C++98)**：标准库曾提供 `auto_ptr` 来模拟"转移所有权"，但它的拷贝构造语义是"转移"而非"复制"，实现上把拷贝构造当移动用，行为反直觉且**不能放进 `std::vector` 等标准容器**（容器要求可正常拷贝），因此 C++11 用 `unique_ptr` 取代它（详见 Lecture 16）。这也是为什么移动语义必须是语言特性而非库技巧——拷贝/移动的区分需要语言层面的值类别支持。
- **裸指针手工管理**：想"转移"就手动 `p = q; q = nullptr;`，容易漏、容易错，全靠自觉。
- **现代优势**：语言自动区分左值/右值并选择拷贝或移动；`noexcept` 让容器放心使用移动；规则清晰（Rule of Five），性能与安全兼得。

关键要点

- **拷贝给持久对象，移动给临时对象**：编译器依据左值（能取地址、活到作用域结束）与右值（临时、活到语句结束）自动选择。
- **右值引用 `T&&` 绑定临时量**；重载 `&` / `&&` 两个版本即可让编译器按值类别选择。
- **移动操作 = 偷指针 + 源对象置空（valid-but-unspecified）**，O(1)；拷贝是深拷贝，O(n)。移动构造 `T(T&&) noexcept`、移动赋值 `T& operator=(T&&) noexcept`。
- **`std::move(x)` 只是 `static_cast<T&&>(x)`**，它不移动任何东西；使用它等于承诺"x 我不要了"，移动后不要再依赖源对象的内容。
- **务必给移动操作加 `noexcept`**，否则容器扩容会回退到拷贝；遵守 Rule of Zero/Three/Five。

常见陷阱与注意事项

- **移动后使用源对象**：移动后源对象处于 valid-but-unspecified 状态，继续读它的内容（如 `whoAmI.get_pixel(...)`）可能解引用 `nullptr`。移动后要么立刻析构/重新赋值，要么不再碰它。
- **忘记把源对象置空**：移动构造/移动赋值里若只偷指针不置 `nullptr`，源对象析构时会 delete 掉你刚偷来的数据（幻灯片中的经典错误）。
- **忘记 `noexcept`**：移动操作没标 `noexcept` 时，`std::vector` 扩容、`std::sort` 等会**回退到拷贝**，性能白白损失。
- **滥用 `std::move`**：对还会继续使用的左值乱用 `std::move`，等于主动制造 use-after-move；课程建议只在性能关键且确定对象不再使用时显式使用。
- **移动赋值忘记释放自己的旧资源**：`operator=(T&& other)` 里不先 `delete[]`/释放自己的数据就偷指针，会造成旧资源泄漏。
- **误以为 `std::move` 会"移动"**：`std::move` 只是类型转换；若类型没有移动构造，代码静默退化为拷贝，行为正确但性能没有提升。

关联作业提示

本讲与 A7: Unique Pointer（`assign7/`，实现你自己的 `cs106l::unique_ptr`）直接相关：

1. **用移动语义实现"唯一所有权"**：作业要求实现 `unique_ptr(unique_ptr&& other)` 与 `operator=(unique_ptr&& other)`——这正是本讲的"偷指针 + 源对象置空"套路：把 `other.ptr` 复制给自己，然后把 `other.ptr = nullptr`（否则两个指针指向同一内存，双双析构造成双重释放；short answer Q2 问的正是这个问题）。
2. **用 `= delete` 禁止拷贝**：作业要求 `unique_ptr(const unique_ptr&) = delete;` 与 `operator=(const unique_ptr&) = delete;`——呼应 Lecture 13 的 `= delete` 与 `std::unique_ptr` 的设计。

3. **`std::move` 的典型应用**：作业 Part 2 的 `create_list` 中，`node->next = std::move(head);` 就是本讲的核心用法——`head` 是左值且不再需要，用 `std::move` 强制触发移动赋值以转移所有权（short answer Q4）。为什么安全？因为移动后 `head` 不再被使用，且移动保持了"同一时刻只有一个 owner"的不变量。
4. **为什么 unique_ptr 不可拷贝**：若有拷贝，两个指针指向同一内存，先析构者 delete 后另一个变成悬垂指针（作业中的示例代码）——这正是本讲"拷贝 vs 移动"的核心动机。
5. **RAII 自动释放**：`unique_ptr` 的析构自动 `delete`（A7 还问递归释放链表时的栈深度问题），详见 Lecture 16。

Lecture 15 (Week 8 – Tuesday): std::optional 与类型安全 (std::optional & Type Safety)

概述

本讲从**类型安全（Type Safety）**的定义出发，探讨"函数签名应如何诚实地表达可能失败的结果"。经典反例是 `vector::back()`：当容器为空时返回"最后一个元素"是未定义行为，因为函数签名 `valueType&` 向调用者做出了虚假的承诺。`std::pair<bool, T>` 方案有种种缺陷（要求 T 可默认构造、需要构造一个无意义的值、语义仍不可靠），而 **`std::optional<T>`** 优雅地解决了问题：它要么包含一个 T，要么什么都不包含（`std::nullopt`）。本讲还介绍 `has_value()` / `value()` / `value_or()` 接口与 C++23 的 **monadic 操作**（`and_then` / `transform` / `or_else`），并对比 Rust/Swift 中 Option 类型的应用。

核心特性与语法详解

1. 类型安全（Type Safety）

- **定义与目的**：类型安全是"语言在多大程度上防止类型错误"（幻灯片第一版定义：The extent to which a language prevents typing errors；后修正为：The extent to which a **function signature** guarantees the behavior of a function）。核心思想：**让函数签名自己"说清楚"行为，把错误挡在编译期**。Python 中 `div_3("hello")` 要到运行时才崩溃；C++ 中 `int div_3(int x)` 传字符串是编译错误——代码永远不会运行。
- **核心语法**：`cpp int div_3(int x) { return x / 3; } // 签名承诺：参数是 int // div_3("hello"); // 编译错误：类型不匹配`
- **设计意图与最佳实践**：类型系统是"编译期强制安全"的体现（呼应 C++ 设计哲学：Enforce safety at compile time whenever possible）。判断一个签名是否类型安全，就看它是否**诚实**：如果函数可能没有返回值，签名就不该承诺"必然返回一个 T"。

2. `vector::back()` 的问题：虚假承诺与未定义行为

- **定义与目的**：`valueType& vector<valueType>::back()` 返回末元素的引用；当容器为空时，它返回 `*(begin() + size() - 1)`，即解引用一个指向"过去末尾"的指针——**未定义行为（UB）**：可能崩溃、可能返回垃圾值、甚至可能"碰巧"给出正确值，无法做出任何保证。
- **核心语法**：`cpp // 有 UB 的版本 valueType& vector<valueType>::back() { return *(begin() + size() - 1); // 空容器时解引用野指针 } // 可靠报错的版本 valueType& vector<valueType>::back() { if (empty()) throw std::out_of_range("back on empty"); return *(begin() + size() - 1); }`
- **设计意图与最佳实践**：幻灯片强调"保证 precondition 是程序员的责任"，但更好的做法是让**函数签名自己传达"可能没有值"**。抛异常是"至少可靠地出错"，而 `std::optional` 是"把'可能没有'写进类型里"。

3. `std::pair<bool, T>` 方案的缺陷

- **定义与目的**：一个直观的"第一版方案"是用 `std::pair<bool, valueType>` 同时返回"是否存在"和"值"，但幻灯片指出三大缺陷。
- **核心语法**：`cpp std::pair<bool, valueType> vector<valueType>::back() { if (empty()) return {false, valueType()}; // 需要一个"假值" return {true, *(begin() + size() - 1)}; }`
- **设计意图与最佳实践**：缺陷一：`valueType` 可能没有默认构造函数，无法构造"假值"；缺陷二：即便能构造，白白调用构造器也是浪费；缺陷三（最要命）：语义仍然不可靠——`while (vec.back().second % 2 == 1)` 在容器为空时会用"假值"继续运算，如果这个假值恰好是奇数，程序行为依旧错误。pair 把"是否存在"和"值"分离成两个字段，调用者必须记得检查 bool，而检查可以漏掉。

4. `std::optional<T>`：值或空

- **定义与目的**：`std::optional<T>` 是模板类，要么包含一个 T 类型的值，要么什么都不包含（用 `std::nullopt` 表示）。C++17 引入。
- **核心语法**：`cpp #include <optional> std::optional<int> o = std::nullopt; // 空 optional o = 42; // 有值 o = std::nullopt; // 又变空 std::optional<int> num1 = {}; // 空 optional`
- **设计意图与最佳实践**：**`nullopt` 不是 `nullptr`**：`nullptr` 可转换为任何指针类型，`nullopt` 可转换为任何 optional 类型——它是一个独立的"空标记"。与 pair 方案不同，optional 把"有没有值"内建为类型的一部分，**不需要单独构造一个假值**，也**强制**调用者在取值前处理"空"的情况。

5. `std::optional` 的接口：`has_value` / `value` / `value_or`

- **定义与目的**：读取 optional 内容的三个基本方法。
- **核心语法**：`cpp std::optional<T> opt; opt.has_value(); // bool：是否有值 opt.value(); // 返回 T&；空时抛出 std::bad_optional_access opt.value_or(default); // 有值返回值，空返回 default // 另外：operator bool / 直接 if(opt) 判断是否有值；operator* 无检查地取值（同 value() 但不抛异常，UB！）`
- **设计意图与最佳实践**：`value()` 在空时抛 `std::bad_optional_access`（比 UB 好——行为确定），`value_or(default)` 适合"有就用，没有用兜底"，`if (opt)` 是惯用的存在性检查（nullopt 是 falsy）。

6. Monadic 操作：`and_then` / `transform` / `or_else`（C++23）

- **定义与目的**：monadic（单子式）是一种把"可选值上的函数应用"串成链的设计模式——每一步要么继续计算，要么短路为 `nullopt`，避免层层手写 `if` 判空。
- **核心语法**：`cpp // 有值时对内部值调用 f 并返回 f 的结果（f 必须返回 optional） opt.and_then(f); // f: T → std::optional<U> // 有值时对内部值调用 f，把结果包进 optional（f 返回普通值） opt.transform(f); // f: T → U // 有值就返回自身，空时调用 f 产生兜底值（f 返回 optional） opt.or_else(f); // f: () → std::optional<U>`
- **设计意图与最佳实践**：三者都**短路**：一旦链上某一步是 `nullopt`，后续不再调用函数、整条链保持 `nullopt`。`and_then` 用于"后续步骤可能失败"（函数返回 optional），`transform` 用于"纯映射不会失败"（函数返回普通值），`or_else` 用于"空时兜底"。这让你写"如果 A 存在则做 B 否则给默认"时完全不需要 if 语句。注意：`std::optional<T&>` 不存在（引用必须绑定有效对象，而 optional 不保证这一点），需要引用语义时只能返回指针或用 `.at()` 抛异常。

代码示例与逐步解说（核心）

示例 1：UB 版本 vs 修复版本（C++11）

```
// C++11（示例来源：Jonathan Müller / foonathan.net，幻灯片引用）
#include <iostream>
#include <vector>

// 有 UB 的版本：空容器时 vec.back() 是未定义行为
void removeOddsFromEndBad(std::vector<int>& vec) {
    while (vec.back() % 2 == 1) {   // 空容器时：解引用野指针 → UB
        vec.pop_back();
    }
}

// 修复：先检查 empty()，保证 precondition
void removeOddsFromEnd(std::vector<int>& vec) {
    while (!vec.empty() && vec.back() % 2 == 1) {
        vec.pop_back();
    }
}

int main() {
    std::vector<int> v = {2, 3, 5, 8};
    removeOddsFromEnd(v);           // 去掉末尾的奇数？不——back=8 是偶数，直接结束
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';              // 2 3 5 8

    std::vector<int> w = {2, 3, 5};
    removeOddsFromEnd(w);           // 5→pop, 3→pop, 2 停
    for (int x : w) std::cout << x << ' ';
    std::cout << '\n';              // 2

    // removeOddsFromEndBad(std::vector<int>{});  // 空容器 → 未定义行为，切勿运行
}
```

- **代码做什么**：`removeOddsFromEnd` 从末尾删除连续的奇数。修复版先 `!vec.empty()` 短路，空容器时 `back()` 根本不会执行。
- **特性机制解说**：`vec.back()` 内部是 `*(begin() + size() - 1)`。容器为空时 `size() - 1` 下溢，`begin() + 巨大值` 指向野内存，解引用就是 UB。UB 意味着**程序行为无任何保证**（崩溃/垃圾值/碰巧正确都有可能），这正是类型安全要消灭的东西。修复方案把"vec 非空"这个 precondition 的检查责任留在调用处——可行，但幻灯片追问：能不能让**函数签名自己**告诉调用者"可能没有末元素"？这引出 optional。

示例 2：`std::optional` 基础——除法防除零（改写自课堂代码）（C++17）

```
// C++17（课堂 main.cpp 的 divide 示例）
#include <iostream>
#include <optional>

std::optional<int> divide(int numerator, int denominator) {
    if (denominator != 0) {
        return numerator / denominator;   // 有值
    } else {
        return std::nullopt;              // 空!
    }
}

int main() {
    std::optional<int> result = divide(10, 2);
    if (result) {                                  // optional 可隐式转 bool
        std::cout << "Result: " << result.value() << '\n';   // Result: 5
    } else {
        std::cout << "Division by zero occurred.\n";
    }

    result = divide(10, 0);
    if (result) {
        std::cout << "Result: " << result.value() << '\n';
    } else {
        std::cout << "Division by zero occurred.\n";   // 走这里
    }
}
```

- **代码做什么**：`divide` 返回 `std::optional<int>`：除数为 0 时返回 `std::nullopt`，否则返回商。main 里用 `if (result)` 判断，有值用 `.value()` 取。
- **特性机制解说**：函数签名 `std::optional<int>` **在类型层面宣告**"可能没有结果"——调用者无法假装一定有值（想用值必须先过存在性检查）。`result` 的 truthiness：`operator bool` 等价于 `has_value()`，`std::nullopt` 是 falsy。若空 optional 上调 `.value()`，会抛 `std::bad_optional_access`（确定的行为，可被 catch），而 `*result`（`operator*`）不做检查，空时是 UB——课程提醒：**用 `*` 需自己保证有值**。

示例 3：查找函数——`has_value` / `value` / `value_or`（C++17）

```cpp
// C++17
#include <iostream>
#include <optional>
#include <vector>

// 返回 value 在 vec 中的下标；找不到返回 nullopt（"可能不存在"写进签名）
std::optional<size_t> findIndex(const std::vector<int>& vec, int value) {
    for (size_t i = 0; i < vec.size(); ++i) {
        if (vec[i] == value) return i;
    }
    return std::nullopt;
}

int main() {
    std::vector<int> v = {1, 3, 7};

    auto idx = findIndex(v, 3);
    std::cout << idx.value_or(999) << '\n';    // 1（有值 → 返回值）
    std::cout << idx.has_value() << '\n';      // 1（true）

    auto missing = findIndex(v, 42);
    std::cout << missing.value_or(999) << '\n'; // 999（空 → 返回兜底）
    std::cout << missing.has_value() << '\n';   // 0（false）

    try {
        missing.value();                        // 空 optional 调 value() → 抛异常
    } catch (const std::bad_optional_access& e) {
        std::cout << "Caught bad_optional_access\n";
    }
}
```

- **代码做什么**：`findIndex` 用 optional 表达"下标可能不存在"。`value_or(999)` 一句代码同时处理"有值"和"空"两种情况；`.value()` 在空时抛 `std::bad_optional_access`。
- **特性机制解说**：对比 `pair<bool, T>` 方案：`findIndex` 的 optional 版本**不需要**为"找不到"构造任何假下标（pair 方案要 `valueType()` 默认构造，且可能没默认构造）；返回 `std::nullopt` 就是"空"。`value_or` 按值返回兜底，适合"取默认值"；`value()` 抛异常，适合"逻辑上必须有值、没有就是 bug"的场景。这套接口让"可能失败"从程序员的自律变成类型的强制。

示例 4：模拟 `optional` 版 `back()` 与 monadic 链（C++17 / C++23）

```
// C++23（back 返回 optional 是教学假设；真实 std::vector::back() 并不返回 optional）
#include <iostream>
#include <optional>
#include <vector>

// 教学用的"安全 back"：空容器返回 nullopt
std::optional<int> safeBack(const std::vector<int>& vec) {
    if (vec.empty()) return std::nullopt;
    return vec.back();
}

int main() {
    std::vector<int> w = {2, 3, 5};

    // transform：把内部值 int 映射成 bool（奇数？），空时短路为 nullopt
    // value_or(false)：空 → false，循环结束
    while (safeBack(w)
               .transform([](int x) { return x % 2 == 1; })
               .value_or(false)) {
        w.pop_back();
    }
    for (int x : w) std::cout << x << ' ';   // 2
    std::cout << '\n';

    // and_then：f 接收内部值、返回 optional，可继续接链
    std::optional<int> o = 10;
    auto chained = o
        .and_then([](int n) -> std::optional<int> {   // 10 → 100
            return n * 10;
        })
        .transform([](int n) { return n + 1; });      // 100 → 101
    std::cout << chained.value_or(-1) << '\n';        // 101

    // or_else：空时调用兜底函数
    std::optional<int> empty = std::nullopt;
    auto rescued = empty.or_else([]() -> std::optional<int> {
        return std::optional<int>(7);
    });
    std::cout << rescued.value() << '\n';             // 7
}
```

- **代码做什么**：第一段把 `removeOddsFromEnd` 改写成 monadic 风格——`transform` 把"末元素是否奇数"算出来，`value_or(false)` 在空容器时给出 false，循环自然终止，**无需手写判空**。后两段演示 `and_then` 与 `or_else` 的链式语义。
- **特性机制解说**：`transform(f)`：有值 → 返回 `std::optional<f(值)>`；空 → 返回 `nullopt`（f 不会被调用）。`and_then(f)`：有值 → 返回 `f(值)`（f 必须返回 optional，因此可用于"下一步可能失败"）；空 → `nullopt`。`or_else(f)`：有值 → 返回自身；空 → 调用 f 得到兜底 optional。三者都**短路**：链上任一步得到 `nullopt`，后面全部跳过。monadic（单子）是一种软件设计模式——把函数组合包装进带额外计算（这里是"可能为空"）的类型里；课程提到我们早就在用 monadic 思想：**链式 view 管道**

（`views::filter | views::transform`）**同样是一步步组合函数并短路**。幻灯片还提醒：真实的 `std::vector::back()` **并不**返回 optional（也不大可能改），optional 主要用于你自己的接口设计。

与旧标准（如C++98）的对比

- `std::optional` **是 C++17 才加入标准库**（`<optional>`）；monadic 操作 `and_then` / `transform` / `or_else` 更是 C++23 才加入。C++98 中表达"可能没有值"只能靠：
- **哨兵值（sentinel）**：如返回 `-1`、`nullptr` 或 `""` 表示"无"。缺点：必须与合法值域不冲突、调用者要记得比对、错误容易漏检（例如 `-1` 恰好是合法下标）。幻灯片中 `pair<bool,T>` 方案的"假值"问题正是哨兵值缺陷的体现。
- `std::pair<bool, T>`：需要 T 可默认构造、浪费构造开销、检查可被忽略（语义不可靠）。
- **指针**：返回 `T*`，用 `nullptr` 表示无。缺点：T 必须可寻址（值语义对象被迫堆分配）、所有权语义模糊、还要判空。
- **现代优势**：optional 把"空"作为**类型系统的一部分**——编译器强制你在取值前处理空；`value_or`、monadic 操作把判空逻辑压缩成一行；`nullopt` 是独立的空标记，不会与指针的 `nullptr` 混淆。课程结论：C++ 的标准容器出于性能（optional 有额外状态开销、调用链繁琐）大多不使用 optional，但**在你自己的接口设计中强烈鼓励使用**。

关键要点

- **类型安全 = 函数签名诚实地保证行为**；"可能没有结果"就不该承诺"必然返回 T"。`vector::back()` 在空容器上是未定义行为，因为签名 `valueType&` 是虚假承诺。
- `std::optional<T>`**（C++17）**：要么含一个 T，要么为空（`std::nullopt`）；`nullopt` 不是 `nullptr`，它可转换为任何 optional 类型。
- **三件套接口**：`has_value()`（或 `if (opt)`）判断、`value()`（空时抛 `std::bad_optional_access`）、`value_or(default)` 兜底取值。
- **Monadic 操作（C++23）**：`and_then`（f 返回 optional，下一步可能失败）、`transform`（f 返回普通值，纯映射）、`or_else`（空时兜底），全部短路，可无限链式组合、免写 if。
- **"Well typed programs cannot go wrong"**：把"可能失败"写进类型，把错误从运行时搬到编译期——这正是 C++ "在编译期尽可能强制安全"设计哲学的体现。

常见陷阱与注意事项

- **对空 optional 调** `.value()`：抛 `std::bad_optional_access`（比 UB 好，但仍是运行时错误）；更危险的是用 `*opt`（`operator*`）——它**不做任何检查**，空时是未定义行为。
- **误把** `nullopt` **当** `nullptr`：`std::optional<int*> o = nullptr;` 是"有值的 optional，值是空指针"；`o = std::nullopt;` 才是"optional 本身为空"。两者语义完全不同。

- **忘记判空就取值**：optional 的接口设计是"强迫"你判空，但如果用 `opt.value()` 直接取值且不 catch，程序照样崩——monadic 链和 `value_or` 才是优雅解。
- **想要 `std::optional<T&>`**：C++ 标准库**没有** `optional<T&>`（引用必须绑定有效对象，optional 无法保证）。需要"可选引用"时改用指针或 `std::reference_wrapper`；下标越界这类场景用 `.at()` 抛 `std::out_of_range`。
- **误以为 monadic 操作会吞掉异常**：`transform` / `and_then` 里 f 自己抛的异常照常传播，短路只针对 `nullopt`，不针对异常。
- **对"检查一下更稳妥"的代码偷懒**：例如 `removeOddsFromEnd` 的 UB 版——即使"看起来能跑"，未定义行为也随时可能翻车，别赌。

关联作业提示

本讲与 **A6: ExploreCourses**（`assignment6/`）直接相关：作业要求用 `std::optional` 表达"课程可能不存在"。

1. **Part 1 写 `find_course`**：把返回类型从占位的 `FillMeIn` 改为 **`std::optional<Course>`**——在 `CourseDatabase` 的私有 `courses` 里按 `course_title` 查找，找到返回该 Course，找不到返回 `std::nullopt`。这正是本讲"签名要诚实表达可能没有值"的核心用法。
2. **Part 2 用 monadic 操作替换 if 语句**：`main` 中 `auto course = db.find_course(argv[1]);` 之后，要用**恰好两个** monadic 操作把 `course` 变成 `std::string output`，且**不能使用任何 if**：
3. `transform`：把 `Course` 映射成格式化字符串（如 `"Found course: " + title + "," + units + "," + quarter`）——f 返回普通 `std::string`，正适合 transform。
4. `or_else`：当 course 为空时提供兜底字符串 `"Course not found."`——空时调 f 返回 `std::optional<std::string>`。
5. 最后 `.value()`（或 `.value_or(...)`）取出字符串。注意链上类型流：`optional<Course>` → `transform` → `optional<string>` → `or_else` → `optional<string>` → `.value()` → `string`。作业提示"从 output 的类型倒推"正是这个思路。
6. 编译命令 `g++ -std=c++23 main.cpp -o main` 说明 monadic 操作需要 **C++23** 标准（`std::optional` 本身 C++17 即可）。

Lecture 16 (Week 8 – Thursday): RAII、智能指针与构建项目 (RAII, Smart Pointers & Building C++ Projects)

概述

本讲分三部分。首先从**异常（try/catch/throw）**入手：异常会带来大量"代码路径"，一旦在 `new` 与 `delete` 之间抛出异常，内存就会泄漏。答案是 **RAII（Resource Acquisition Is Initialization，资源获取即初始化）**——资源在构造函数中获取、在析构函数中释放，无论是否抛异常，析构函数都保证执行。其次介绍三大**智能指针** `std::unique_ptr`（唯一所有权、不可拷贝）、`std::shared_ptr`（引用计数、可共享）、`std::weak_ptr`（不增加引用计数、打破循环引用），以及工厂函数 `std::make_unique` / `std::make_shared`。最后是构建工具链：编译命令、Makefile 与 CMake/CMakeLists.txt，让多文件项目不再靠手敲命令。

核心特性与语法详解

1. 异常：try / catch / throw

- **定义与目的**：异常是"错误发生时"的处理机制——错误被 **throw** 抛出，代码用 **try/catch** 捕获并继续运行，而不是直接终止。结构类似 if/else if/else 链。
- **核心语法**：`cpp try { // 可能抛异常的代码 } catch (const std::runtime_error& e1) { // 类似 "if" // 处理第一种错误 } catch (const std::exception& e2) { // 类似 "else if" // 处理第二种错误 } catch (...) { // 类似 "else" // catch-all：兜底捕获一切 }`
- **设计意图与最佳实践**：异常类型应尽量具体（`catch (const std::exception&)` 比 `catch (...)` 信息更多）；捕获后要么处理、要么重新抛出。异常的代价是**控制流不再线性**——幻灯片统计一行 `return Pet(...)` 的代码有至少 23 条可能路径（拷贝构造、临时 string 构造、重载运算符、返回字符串拷贝都可能抛）。这意味着"new 之后、delete 之前"任何一步抛异常，delete 就可能被跳过 → 内存泄漏。

2. RAII：Resource Acquisition Is Initialization

- **定义与目的**：Bjarne Stroustrup 提出的思想：**所有资源应在构造函数中获取，在析构函数中释放**。这样资源生命周期与对象作用域绑定：对象创建即可用，对象销毁必然释放——无论中途是否抛异常（栈展开时析构函数必被调用）。
- **核心语法**（反例 vs 正例）：`cpp // 反例：ifstream 在代码中打开/关闭，异常时可能漏关 void bad() { std::ifstream file; file.open("data.txt"); // 资源在"代码中"获取 // ... 若这里抛异常，file 永远不会 close file.close(); // 手工释放 } // 正例：RAII 类在构造时打开、析构时关闭 void good() { std::ifstream file("data.txt"); // 构造 = 获取资源 // ... 无论发生什么，file 离开作用域时析构自动 close }`
- **设计意图与最佳实践**：RAII 消除"半有效状态"：对象要么不存在，要么完全可用；资源要么没获取，要么保证释放。`std::lock_guard` 是 RAII 的典范——构造时获取锁，析构（出作用域）时自动释放，锁永远不会因为异常而"忘记解锁"。智能指针就是把 RAII 应用到内存管理的产物。

3. `std::unique_ptr<T>`：唯一所有权

- **定义与目的**：独占管理一块堆内存的"包装指针"：构造（或 `make_unique`）时拥有资源，析构时自动 `delete`。**不可拷贝**（拷贝构造/拷贝赋值被 `= delete`），只能**移动**（所有权转移）。这正是 Lecture 13/14 的 SMF 与移动语义的实际应用。
- **核心语法**：`cpp #include <memory> std::unique_ptr<T> p = std::make_unique<T>(args...); // 首选！ auto q = std::move(p); // 移动转移所有权，p 变空 T* raw = p.get(); // 取裸指针（不转移所有权） p->member; (*p).member; // 像裸指针一样用`
- **设计意图与最佳实践**：为什么不可拷贝？如果两个 unique_ptr 指向同一内存，先析构者 delete 后，另一个就成了悬垂指针（幻灯片场景）。移动则始终保证"同一时刻只有唯一 owner"。**用 `std::make_unique<T>` 而不是 `new`**（后面详解）。

4. `std::shared_ptr<T>`：引用计数共享所有权

- **定义与目的**：多个 shared_ptr 可共享同一块内存，内部维护**引用计数（reference count）**：每拷贝一次计数 +1，每析构一个 −1；计数归零时才真正 `delete` 底层资源。解决"unique_ptr 不能拷贝"的共享需求。
- **核心语法**：`cpp std::shared_ptr<T> a = std::make_shared<T>(args...); std::shared_ptr<T> b = a; // 拷贝：计数 +1（现在 2） b.reset(); // b 释放：计数 −1（现在 1） a.reset(); // 计数归零 → 真正 delete long n = a.use_count(); // 当前引用计数`
- **设计意图与最佳实践**：计数增减是原子的（线程安全）；但**循环引用**是致命陷阱：A 持有指向 B 的 shared_ptr、B 持有指向 A 的 shared_ptr 时，两者计数永远到不了 0，内存永不释放——这就是 weak_ptr 要解决的。

5. `std::weak_ptr<T>`：打破循环引用

- **定义与目的**：观察 shared_ptr 管理的对象但**不增加引用计数**（不拥有所有权）。典型用途：打破 A↔B 循环引用。因为不计数，它指向的对象可能随时被销毁，使用时需 `lock()` 升级为 shared_ptr 再访问。
- **核心语法**：`cpp std::weak_ptr<T> w = a; // 从 shared_ptr 构造，计数不变 if (auto s = w.lock()) { // lock() 返回 shared_ptr；对象已死则返回空 s->use(); // 安全访问 } else { // 对象已被销毁 }`
- **设计意图与最佳实践**：`weak_ptr` 永远不能直接解引用（没有 `operator*` / `operator->`）——必须先 `lock()`，因为对象可能已不存在。它不是"第二所有权"，而是"不拥有所有权的观察者"。

6. `std::make_unique` / `std::make_shared`

- **定义与目的**：智能指针的推荐工厂函数，替代裸 `new`。
- **核心语法**：`cpp auto p = std::make_unique<T>(ctorArgs...); // C++14 起 auto s = std::make_shared<T>(ctorArgs...); // 反面教材（避免）： std::shared_ptr<T> s2(new T(...)); // 两步：先 new 再包装`

- **设计意图与最佳实践**：两个理由——（1）**异常安全**：`f(std::shared_ptr<A>(new A), g())` 中若 `g()` 先于包装抛异常，`new A` 出来的裸指针就泄漏了；`make_shared` 一步到位无此窗口。（2）**效率**：`make_shared` 把对象与计数块放进同一次分配；`make_unique` 让代码统一（"用了 make_unique 就也用 make_shared"），避免 `new` 与智能指针混用。

7. 构建项目：编译命令、Makefile、CMake

- **定义与目的**：C++ 源码要先翻译成机器码（编译器）才能运行；多文件项目需要把多个 `.cpp` 一起编译。手工敲命令不现实，于是有了 **make（构建系统，读 Makefile）** 和 **CMake（构建系统生成器，读 CMakeLists.txt，生成 Makefile）** 两层工具。
- **核心语法**：`sh # 编译命令：g++ -std=<标准> <源文件...> -o <可执行文件名> g++ -std=c++20 main.cpp user.cpp -o main ./main` `makefile # Makefile CXX = g++ # 编译器 CXXFLAGS = -std=c++20 # 编译选项 SRCS = $(wildcard *.cpp) # 自动收集所有 .cpp TARGET = main all: $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET) clean: rm -f $(TARGET)` `cmake # CMakeLists.txt cmake_minimum_required(VERSION 3.10) project(cs106l_classes) set(CMAKE_CXX_STANDARD 20) # 指定 C++20 file(GLOB SRC_FILES "*.cpp") # 通配收集源文件 add_executable(main ${SRC_FILES}) # 生成可执行文件 main`
- **设计意图与最佳实践**：make 的优点是**增量编译**（只重编改动过的文件）和集中管理编译参数；CMake 在 Makefile 之上再加一层抽象（跨平台、可生成不同构建系统）。CMake 标准流程：`sh mkdir build && cd build cmake .. # 用根目录的 CMakeLists.txt 生成 Makefile make # 编译 ./main # 运行`

代码示例与逐步解说（核心）

示例 1：异常基础——throw 与 catch（C++11）

```
// C++11
#include <iostream>
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) throw std::runtime_error("division by zero");  // 抛出异常
    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 2) << '\n';    // 5（正常）
        std::cout << divide(10, 0) << '\n';    // 抛出异常，跳到 catch
        std::cout << "never printed\n";
    } catch (const std::runtime_error& e) {    // 按类型捕获
        std::cout << "Caught: " << e.what() << '\n';
    }
    std::cout << "Program continues...\n";     // 捕获后继续执行
}
```

- **代码做什么**：`divide(10, 0)` 抛出 `std::runtime_error`；`try` 块立即中断，控制流跳到匹配的 `catch`，打印错误信息后程序继续运行。
- **特性机制解说**：`throw` 会**展开栈（stack unwinding）**——从抛出点向上，逐层销毁已构造的局部对象并调用其析构函数，直到找到匹配的 `catch`。这正是 RAII 能保证异常安全的机制基础：**即使抛出异常，局部对象的析构函数也一定会执行**。异常匹配是按类型（含继承）进行的，`catch (const std::exception&)` 能捕获所有派生异常，`catch (...)` 兜底一切。若没有 catch，程序调用 `std::terminate` 终止。

示例 2：异常导致内存泄漏（C++11）

```
// C++11：演示为什么"裸 new + 手工 delete"在异常面前如此脆弱
#include <iostream>
#include <stdexcept>

void process(bool fail) {
    int* data = new int[1000];          // 手工分配堆内存
    if (fail) {
        throw std::runtime_error("boom");   // 异常在这里抛出！
        // delete[] data;                   // ❌ 永远不会执行 → 泄漏 1000 个 int
    }
    delete[] data;                          // 只有不抛异常才执行
}

int main() {
    try {
        process(true);
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << '\n';   // 捕获了，但内存已泄漏
    }
}
```

- **代码做什么**：`process(true)` 在 `new` 之后抛异常，`delete[]` 被跳过，程序"捕获成功"却泄漏了 4KB 内存。
- **特性机制解说**：裸指针不是 RAII 对象——它没有析构函数，栈展开时**没有任何机制**帮它释放内存。幻灯片强调"这不只是指针的问题"：文件句柄、锁、数据库连接……一切"需要获取后释放"的资源在异常下都可能泄漏。解决思路就是给资源套一层 RAII 包装：**资源在构造时获取、在析构时释放**，析构由栈展开保证执行。`std::lock_guard`（构造加锁、析构解锁）和智能指针（构造持有、析构释放）都是这个思路的实例。

示例 3：自定义 RAII 类（C++11）

```
// C++11：手写一个 RAII 文件类，展示"析构必执行"
#include <iostream>
#include <stdexcept>

class File {
public:
    explicit File(const char* name) {
        std::cout << "Opening " << name << '\n';
        // 真实代码：fopen / std::ifstream 打开资源
    }
    ~File() {
        std::cout << "Closing file (always runs!)\n";
        // 真实代码：fclose / 关闭资源
    }
};

void useFile(bool fail) {
    File f("data.txt");         // 构造 = 获取资源
    if (fail) {
        throw std::runtime_error("error while using file");
    }
    // 资源使用完毕
}   // ← 无论是否抛异常，f 的析构函数都会在这里执行

int main() {
    try {
        useFile(true);          // 抛异常 → 栈展开 → f 析构 → catch
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << '\n';
    }
    useFile(false);             // 正常路径同样析构
}
```

- **代码做什么**：`File` 在构造时"打开"、析构时"关闭"。`useFile(true)` 抛异常，但 `f` 的析构仍被执行（栈展开），输出 "Closing file (always runs!)"；正常路径也一样。
- **特性机制解说**：这就是 RAII 的全部奥义——**资源生命周期 = 对象生命周期**。构造函数里获取资源，对象诞生即"完全可用"（无半有效状态）；析构函数里释放资源，对象死亡即"资源已还"。异常安全来自语言保证：**栈展开时所有已构造的局部对象的析构函数必然被调用**。对比示例 2：裸 `new` 的指针在栈展开时"没人管"，而 RAII 对象永远有人管。`std::lock_guard` 正是这个模式在锁上的应用——critical section 里抛异常，锁也会被自动释放，绝不死锁。

示例 4：`std::unique_ptr` 链表——RAII 自动释放（改写自 A7 的 ListNode 示例）（C++14，`make_unique` 需 C++14）

```
// C++14（std::make_unique 自 C++14 起可用）
#include <iostream>
#include <memory>

struct Node {
    int value;
    std::unique_ptr<Node> next;              // 递归持有下一个节点
    explicit Node(int v) : value(v) {}
    ~Node() { std::cout << "Destroying node " << value << '\n'; }
};

int main() {
    auto head = std::make_unique<Node>(1);
    head->next = std::make_unique<Node>(2);
    head->next->next = std::make_unique<Node>(3);

    // std::unique_ptr<Node> copy = head;   // ❌ 编译错误：unique_ptr 不可拷贝
    std::unique_ptr<Node> moved = std::move(head);   // ✅ 移动转移所有权
    std::cout << "head is " << (head ? "non-null" : "null") << '\n';  // null

    if (moved) std::cout << "moved->value = " << moved->value << '\n'; // 1
}   // moved 析构 → 递归销毁 1 → 2 → 3，全程无手工 delete!
```

- **代码做什么**：用 `make_unique` 建一条三节点链表，演示移动转移所有权；main 结束时 `moved` 析构，整条链表递归释放，打印三次 "Destroying node"。
- **特性机制解说**：`unique_ptr` 的析构调用 `delete`，而 `Node` 的析构会销毁成员 `next`（另一个 unique_ptr），后者又触发下一个 `Node` 的析构——**递归释放**整条链。不可拷贝是设计核心：若允许拷贝，两个指针指向同一节点，先析构者 delete 后另一个悬垂（幻灯片"original destructor is called after the copy happens"场景）。移动保持唯一性：只是把所有权从 `head` 转给 `moved`，`head` 变空指针。A7 提醒：这种递归析构对**很长的链表**会消耗调用栈（每次析构嵌套一层），可能栈溢出。

示例 5：`shared_ptr` 引用计数与 `weak_ptr` 打破循环（C++11）

```
// C++11：循环引用 = 泄漏；weak_ptr 打破循环
#include <iostream>
#include <memory>

struct A;
struct B;
struct A { std::shared_ptr<B> b; ~A() { std::cout << "~A\n"; } };
struct B { std::shared_ptr<A> a; ~B() { std::cout << "~B\n"; } };  // 循环！

struct C;
struct D;
struct C { std::shared_ptr<D> d; ~C() { std::cout << "~C\n"; } };
struct D { std::weak_ptr<C> c; ~D() { std::cout << "~D\n"; } };   // weak 打破循环

void badCycle() {
    auto pa = std::make_shared<A>();
    auto pb = std::make_shared<B>();
    pa->b = pb;
    pb->a = pa;   // A 与 B 互相持有 shared_ptr → 计数都 ≥1，永不释放
}                 // 函数结束：什么析构都不打印 → 内存泄漏！

void goodCycle() {
    auto pc = std::make_shared<C>();
    auto pd = std::make_shared<D>();
    pc->d = pd;
    pd->c = pc;   // weak_ptr 不增加引用计数
}                 // 正常释放：打印 ~C 与 ~D

int main() {
    std::cout << "bad:\n";
    badCycle();
    std::cout << "good:\n";
    goodCycle();
}
```

- **代码做什么**：`badCycle` 中 A↔B 互相持有 shared_ptr，函数结束时两者引用计数都降不到 0，析构函数永远不被调用（泄漏）；`goodCycle` 中 D 用 `weak_ptr` 持有 C，不计数，全部正常释放。
- **特性机制解说**：shared_ptr 内部有一个引用计数（常与对象一起分配在"控制块"里）：每份拷贝 +1，每个析构 −1，归零才 `delete` 底层对象。循环引用时：`pa` 与 `pb->a` 各持一份指向 A 的计数（2），`pb` 与 `pa->b` 各持一份指向 B 的计数（2）。函数结束时 `pa`、`pb` 各 −1，计数仍为 1——**谁也到不了 0，谁也不释放**。`weak_ptr` 旁观不计数：`pd->c` 是 weak_ptr，指向 C 的计数只有 `pc` 一份；`pc` 析构 → C 计数归零 → `~C` → 销毁成员 `d`（shared_ptr）→ D 计数归零 → `~D`。使用 weak_ptr 时要注意：它没有 `operator->`，必须 `lock()` 得到 shared_ptr 才能访问（对象可能已被销毁，lock 返回空）。

示例 6：`make_shared` 的异常安全（C++11）

```
// C++11
#include <iostream>
#include <memory>
#include <stdexcept>

struct Widget { Widget(int) {} };

void helper() { throw std::runtime_error("boom"); }

int main() {
    // 危险写法：先 new 再包装，若 helper() 先抛异常，new 的内存没人接管 → 泄漏
    // std::shared_ptr<Widget> w(new Widget(1), helper());   // 求值顺序不定，可能泄漏

    // 安全写法：make_shared 一步完成，要么都成功要么都失败
    try {
        auto w = std::make_shared<Widget>(1);
        helper();                  // 若这里抛异常，w 的析构会正常释放 Widget
        std::cout << "OK\n";
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << '\n';
    }
}
```

- **代码做什么**：演示 `make_shared` 与 RAII 组合的异常安全性：即使 `helper()` 抛异常，`w` 作为局部对象仍会在栈展开时释放其管理的 Widget。
- **特性机制解说**：`std::shared_ptr<Widget>(new Widget(1), helper())` 有两个独立步骤（先 `new`、再构造 shared_ptr 接管），C++ 允许求值顺序导致"helper() 先抛、new 出的裸指针无人接管"的泄漏窗口。`make_shared` 把分配与包装合成一个原子操作，堵死这个窗口；同时它把对象与控制块放进**同一次分配**，比"new + shared_ptr 构造"少一次堆分配。这就是"永远用 make_unique / make_shared，不要用裸 new"的完整理由。

示例 7：构建工具链——编译命令、Makefile、CMake

```
# ① 直接编译（多文件）
g++ -std=c++20 main.cpp user.cpp -o main
./main
```

```
# ② CMake 标准工作流（Makefile 与 CMakeLists.txt 的完整写法见上文"核心特性与语法详解"第 7 节）
mkdir build && cd build     # 在项目内建 build 目录（生成物集中存放）
cmake ..                    # 用根目录的 CMakeLists.txt 生成 Makefile
make                        # 编译
./main                      # 运行
```

- **代码做什么**：两段命令演示"直接编译"与"CMake 标准工作流"（Makefile/CMakeLists.txt 内容见上文，此处不再重复）。

- **特性机制解说**：`g++` 是编译器，`-std=c++20` 指定语言标准，`-o main` 指定输出名。make 是**构建系统**：读 Makefile，按目标（target）执行命令，并能**增量编译**——只重新编译自上次构建以来修改过的文件（幻灯片强调这是 make 的核心优势）。CMake 是**构建系统生成器**：读 CMakeLists.txt（更高层的抽象），生成 Makefile（或其他构建系统的文件）。CMake 的 `file(GLOB SRC_FILES "*.cpp")` 等价于 Makefile 的 `$(wildcard *.cpp)`——自动收集源文件，新增 .cpp 无需改构建脚本。真实世界如 TensorFlow Core 有 2000+ 源文件，显然需要这样的工具链而不是手敲命令。

与旧标准（如C++98）的对比

- **智能指针在 C++98 的处境**：C++98 标准库只有 `std::auto_ptr`（`<memory>`）。它的"拷贝"语义其实是转移所有权（拷贝构造会把源指针置空），行为反直觉、无法放进标准容器（`vector<auto_ptr<T>>` 会出问题），也没有移动语义支撑。C++11 用 `std::unique_ptr` 取代它，`auto_ptr` 在 C++17 被正式移除。
- **C++98 的资源管理**：只有裸指针 + 手工 `new`/`delete`，异常安全完全靠程序员自律（示例 2 的泄漏在 C++98 同样存在且无解）；锁、文件等资源同样手工管理，异常路径极易泄漏。RAII 思想在 C++98 就存在（`std::string`、`std::vector` 就是 RAII 类），但直到 C++11 才把它系统化应用到指针上。
- **`std::make_shared` 是 C++11 的，`std::make_unique` 是 C++14 的**（`make_unique` 是 C++14 才进入标准库，此前常用自定义版本）。C++98 只能 `new` + 手工包装。
- **构建工具**：make 和 CMake 都是老工具（Make 自 1976 年，CMake 自 2000 年），不是 C++ 标准的一部分；区别在于 CMake 的 `set(CMAKE_CXX_STANDARD 20)` 让"指定 C++ 版本"变得显式、可跨平台。C++20 起还有官方的模块（modules）与包管理器（如 CMake 的 FetchContent）在演进，但课程仍以 Makefile/CMake 为教学主线。

关键要点

- **异常**：`try/catch/throw` 让错误可捕获、程序可继续；但异常带来大量隐式代码路径，任何"获取后未释放"的资源都可能泄漏。
- **RAII**：资源在构造函数获取、析构函数释放；栈展开保证析构必执行，因此 RAII 是 C++ 异常安全的基石（`lock_guard`、智能指针、`string`/`vector` 都是 RAII）。
- **`unique_ptr`**：唯一所有权、不可拷贝、可移动；析构自动 `delete`；**永远用 `std::make_unique`**。
- **`shared_ptr`**：引用计数共享所有权，计数归零才释放；`weak_ptr` 旁观不计数，用来**打破循环引用**；**永远用 `std::make_shared`**（异常安全 + 一次分配）。
- **构建**：`g++ -std=c++XX files -o exe` 直接编译；多文件项目用 Makefile（make，增量编译）；再往上用 CMake（CMakeLists.txt → 生成 Makefile → `cmake .. && make`）。

常见陷阱与注意事项

- **循环引用导致内存泄漏**：A↔B 互相持有 `shared_ptr`，引用计数永远到不了 0，析构永不执行。打破方法：让其中一方（通常是被依赖/生命周期更短的一方）持有 `weak_ptr`。
- **`weak_ptr` 直接解引用**：`weak_ptr` 没有 `operator*` / `operator->`，必须先 `lock()` 检查对象是否还活着；对象可能已被销毁，`lock()` 返回空 shared_ptr。
- **用裸 `new` 而不是 `make_unique` / `make_shared`**：既有异常安全窗口（先 new 后包装，中间抛异常就泄漏），又多一次分配；记住"new 与 delete 每出现一次，就是一次手动管理的机会"。
- **忘记 `noexcept`（呼应 L14）**：unique_ptr 的移动操作若可能抛异常，容器会退化为拷贝。
- **手工 `delete` 智能指针管理的内存**：`delete p.get();` 是双重释放；`p.release()` 后不接管则是泄漏——尽量不要触碰 `.get()` / `.release()` 返回的裸指针的所有权。
- **Makefile 的 tab 陷阱**：`all:` 下的命令必须以**制表符（Tab）**开头，空格会导致 `make` 报错 "missing separator"；CMake 记得在 build 目录里执行 `cmake ..`，不要把生成物散落在源码目录。

关联作业提示

本讲与 **A7: Unique Pointer**（`assign7/`）直接相关——你要亲手实现一个简化版 `cs106l::unique_ptr`，正是本讲三个主题的集大成：

1. **实现 RAII**：`~unique_ptr()` 里 `delete ptr;`（本讲"析构释放资源"）；short answer Q1 问的正是"用 RAII 管理内存相比手工 new/delete 的好处"（自动释放、异常安全、无泄漏）。
2. **实现不可拷贝 + 可移动**：`unique_ptr(const unique_ptr&) = delete;` 与 `operator=(const unique_ptr&) = delete;`（L13 的 `= delete`），`unique_ptr(unique_ptr&& other)` 与 `operator=(unique_ptr&& other)` 用 L14 的"偷指针 + 置空源对象"套路实现（short answer Q2 问不置空会怎样——双重释放）。
3. **Part 2 的链表**：`head->next = std::make_unique<ListNode<int>>(2)` 的递归结构正是本讲示例 4；`create_list` 里用 `node->next = std::move(head)` 转移所有权（L14 的 `std::move`）。short answer Q3 问"递归析构对长链表的隐患"——本讲示例 4 的机制解说里提到了栈深度问题。
4. **构建命令**：`g++ -std=c++20 main.cpp -o main`（A7 单文件）与 A5 的 `g++ -std=c++20 main.cpp user.cpp -o main`（多文件）就是本讲编译命令一节的内容；若项目变大，可以用 Makefile 或 CMake 来管理。

另外，L16 与本讲之外的两讲也有关联：`std::optional`（L15）在 A6 中使用；A5 的 `User` 类本质上是"手动管理裸指针数组的 RAII 类"（析构释放、拷贝深拷贝），理解 RAII 能帮你写对它的析构函数。

Lecture 17 (Week 9 – Tuesday): 可选讲座：C++ 冰山 (Optional Lecture: C++ Iceberg)

概述

本讲是学期末的可选趣味讲座（参加可补一次缺勤）。它像一座"冰山"一样，展示 C++ 中那些藏在表面之下、令人惊讶甚至"反直觉"的语言细节与历史轶事：从 `else if` 的真相、range-for 循环的临时对象生命周期陷阱，到 `iostream` 的错误检查缺陷与 ABI（应用二进制接口）如何"冻结"了标准库的改进。本讲不引入新的大语法特性，而是**深化你对已有知识的理解**，并提醒你 C++ 中"看似理所当然"之处往往暗藏机制。

核心特性与语法详解

- **`else if` 的真相**：C++ 里其实**没有** `else if` 关键字。
  - *定义与目的*：`else if` 是"`else` 分支里嵌套了一个 `if` 语句"的语法糖——`if` 本身是一条语句，`else` 后面可以跟任意一条语句，包括另一个 `if`。
  - *核心语法*：`cpp if (cond1) { /* ... */ } else if (cond2) { /* ... */ } // 实际是 else { if (cond2) {...} } else { /* ... */ }`
  - *设计意图*：理解这一点能解释为什么 `else if` 可以无限级联，以及悬挂 `else`（dangling else）这类经典陷阱。
- **range-for 的生命周期陷阱**：`for (auto e : expr)` 中，范围表达式 `expr` 生成的**临时对象会存活整个循环**——但仅限于"直接"的临时对象。
  - *定义与目的*：C++ 保证绑定到范围表达式的临时对象活到循环结束，避免悬垂引用。
  - *核心语法*：`cpp for (auto e : getCollection()) // ✅ 临时集合活到循环结束 for (auto e : getCollection().getRef()) // ❌ 危险!`
  - *设计意图*：当临时对象**内部**返回的引用（`getRef()` 指向临时集合内部元素）被用作范围时，外层临时集合在循环开始前就被销毁了，`getRef()` 返回的引用指向已销毁的内存——这是经典的**悬垂引用/悬垂迭代器**陷阱。通常"碰巧能跑"只是因为内存尚未被复用（见"常见陷阱"）。
- **`iostream` 的错误检查缺陷（"hello world has a bug"）**：
  - *定义与目的*：`std::cout << "hello"` 这类输出语句**不会**在写失败时立刻抛异常或返回错误；失败被记录在流的状态位里，除非你显式检查，否则程序会"假装成功"。
  - *核心语法*：`std::cout << "hello";` 后检查 `std::cout.good()` / `std::cout.fail()`，或调用 `std::cout.exceptions(std::ios::failbit)` 让失败抛异常。
  - *设计意图*：写入缓冲区后立即返回，若目标设备（如 `/dev/full`）拒绝写入，错误发生在之后的 flush 阶段——错误"确实发生了，但程序忽略了它"。
- **ABI（Application Binary Interface）**：代码在二进制层面的契约，包括调用约定（calling conventions）、数据表示（data representation）、系统调用、**名字修饰（name mangling）**、异常处理等。
  - *定义与目的*：ABI 决定了编译出的二进制文件能否与别的编译器/版本链接。标准库的很多"理想改进"因为会破坏 ABI 而无法实现。

    - *例子*：让关联容器（`map` / `set`）更快、让 `std::regex` 更快（目前"启动 PHP 跑一次正则都比 `std::regex` 快"）、给 `unique_ptr` 加语言级支持使其零开销装入寄存器、给 `regex` 加 UTF-8 支持、大幅降低异常成本——**全部因为 ABI break 而被搁置**。

- **`iostream` 的设计史**：Stroustrup 在《The C++ Programming Language》第 8.3.1 节记载：C 的 `printf` 家族"有效但不类型安全、不可扩展"，于是他寻找"类型安全、简洁、可扩展且高效"的替代品；Douglas McIlroy 建议仿照 Unix 流的 `>>`、`>`、`|`（还考虑过 `=`、`<`、`>`）；Andrew Koenig 提出了操纵符（manipulator）的概念。Ada 的官方论证（Ichbiah, 1979）曾认为"没有特殊语言特性就不可能做出简洁且类型安全的 I/O"——C++ 的流做到了。
- **`->` 运算符的真相**：`obj->member` 本质是 `(*obj).member` 的语法糖。本讲吐槽：它的名字甚至不叫"arrow operator"，标准里叫 **member access operator**。

代码示例与逐步解说

示例 1：range-for 的临时对象生命周期陷阱（C++11 起）

```cpp
#include <iostream>
#include <string>
#include <vector>

std::vector<int> getCollection() {
    return {1, 2, 3};  // 返回一个临时 vector
}

class Wrapper {
    std::vector<int> data_ = {4, 5, 6};
public:
    const std::vector<int>& getRef() const { return data_; }  // 返回内部引用
};

int main() {
    // ✅ 安全：临时 vector 活到循环结束
    for (auto e : getCollection()) {
        std::cout << e << ' ';
    }
    std::cout << '\n';

    // ❌ 危险：getCollection() 临时对象在循环开始前销毁，
    //    .getRef() 返回的引用指向已销毁的 vector
    // for (auto e : getCollection().getRef()) { ... }   // 不要这样写！

    // ✅ 正确写法：先把临时对象存进具名变量
    auto coll = getCollection();
    for (auto e : coll) { std::cout << e << ' '; }
    std::cout << '\n';

    // ✅ Wrapper 的 getRef() 安全：data_ 是 Wrapper 的成员，
    //    只要 Wrapper 对象本身存活
    Wrapper w;
    for (auto e : w.getRef()) { std::cout << e << ' '; }
    std::cout << '\n';
    return 0;
}
```

【代码做什么？】 1. `getCollection()` 返回临时 `vector<int>{1,2,3}`；`for (auto e : getCollection())` 中该临时对象被延长生命周期至循环结束，安全打印 `1 2 3`。 2. 被注释掉的 `getCollection().getRef()` 会先构造临时 vector，再在临时 vector 上调用 `getRef()`——但 `getCollection()` 返回的是 `vector`（按值），它没有 `getRef()` 成员，所以这一行其实是概念示意（更真实的危险版本是：一个类对象的临时实例返回其成员引用，随后临时对象销毁）。 3. 正确姿势：把临时对象存入具名变量 `coll`，再对其 range-for。 4. `Wrapper w;` 中 `w.getRef()` 返回 `w` 内部成员的引用，`w` 存活期间引用始终有效。

【特性机制解说】 * range-for 等价于：`cpp auto&& __range = 范围表达式; // 临时对象绑定到引用，生命周期延长 for (auto __begin = begin(__range), __end = end(__range); __begin != __end; ++__begin) { ... }` * 生命周期的延长只发生在"范围表达式是纯临时对象"且直接绑定到 `auto&&` 时；一旦临时对象在

表达式内部被"用完"（如调用成员函数后返回内部引用），延长的对象是那个内部引用所指向的对象（若它本身是临时对象的成员，则临时对象已死）。这就是"range-for 是 broken 的"说法的来源：**它承诺整个循环期间范围表达式都活着，但"范围表达式求值结果的成员"不一定活着**。

示例 2：`iostream` 忽略写错误（C++11 起）

```
#include <iostream>
#include <fstream>

int main() {
    std::ofstream out("/dev/full");  // 一个总是写失败的设备（Linux）
    if (!out.is_open()) { std::cerr << "open failed\n"; return 1; }

    out << "hello";                  // 写入缓冲区
    out << std::flush;               // 强制刷新 -> 真正写盘 -> 失败!
    if (out.fail()) {
        std::cerr << "write failed (failbit set)\n";   // 现在才报告失败
    }

    // 想让失败直接抛异常：
    out.exceptions(std::ios::failbit | std::ios::badbit);
    out << "boom";                   // 若失败，这里抛出 std::ios_base::failure
    return 0;
}
```

【代码做什么？】 向 `/dev/full`（一个写入必失败的伪设备）写数据：`<<` 只写进缓冲区立即返回；`flush` 才触发真正写入并失败；随后 `out.fail()` 才能检测到。若设置了 `exceptions()`，失败会以异常形式立即抛出。

【特性机制解说】 输出流默认**缓冲**，`<<` 的返回值是流本身（`std::ostream&`），失败只置位 `failbit/badbit` 状态位而不报错——这是"hello world 也有 bug"的由来：绝大多数教程代码从不检查 `cout` 是否真的写成功了。高性能与简单接口的取舍是 C++ 的一贯哲学：**把控制权交给程序员，代价是你必须主动检查**。

示例 3：`else if` 只是嵌套 if（C++98 起）

```
#include <iostream>

int main() {
    int x = 5;
    if (x > 10) {
        std::cout << "big\n";
    } else if (x > 3) {          // 等价于 else { if (x > 3) {...} }
        std::cout << "medium\n";
    } else {
        std::cout << "small\n";
    }
    return 0;
}
```

【代码做什么？】 打印 `medium`。 **【特性机制解说】** `else` 后跟一条语句（这里是另一个 `if` 语句），因此 `else if` 链本质是嵌套 `if`。这也解释了悬挂 `else`：`else` 与**最近**的未配对 `if` 结合。

与旧标准（如C++98）的对比

- 本讲的多数内容是 C++ 的"历史包袱与设计决策"，自 C++98 起就存在（`else if` 语法、`iostream` 缓冲语义、ABI 约束）。
- **变化点**：`std::cout.exceptions()`（状态位→异常）自 C++98 就有；range-for 是 C++11 新增，其生命周期语义（临时对象延长至循环结束）正是 C++11 标准明确规定的；名字修饰（name mangling）与异常表（unwind tables）在 C++98 起就属于 ABI 的一部分，因此 C++11 引入移动语义等新特性时也刻意避免破坏 ABI。

关键要点

1. **没有 `else if`**：它是"`else` + 嵌套 `if`"的语法糖；理解它有助于理解悬挂 `else`。
2. **range-for 只保证"范围表达式这个临时对象"活到循环结束**，不保证临时对象内部返回的引用仍然有效——把临时对象先存进具名变量再遍历。
3. **`iostream` 的写失败默认被静默忽略**：关键输出后检查 `fail()`，或设置 `exceptions()`；`<<` 只是"写入缓冲区"。
4. **ABI 是 C++ 标准库改进的最大掣肘**：很多"本应更好"的标准库设计（更快的 `regex`、零开销 `unique_ptr`）因 ABI 兼容而无法落地。
5. **C++ 的"怪异"背后往往是设计权衡与历史**：流 I/O 的诞生（对 `printf` 的回应）与 `->`、`else if` 等语法糖都是如此。

常见陷阱与注意事项

- **在 range-for 里对临时对象成员遍历**：`for (auto e : makeObj().getRef())` 会产生悬垂引用；程序可能"碰巧正常"（内存未被覆盖），这是未定义行为，随时可能崩溃。
- **把 `std::endl` 当普通换行**：`endl` 会 flush，在循环里大量使用会显著拖慢程序；无刷新需求时用 `'\n'`。
- **从不检查输出流状态**：写日志/写文件时，失败可能在你完全不知情的情况下发生。
- **尝试"改进"标准库结构**：不要指望 `std::regex` 很快变快，也不要自己定义与 ABI 相关的类型布局去碰运气；优先用成熟方案（如对性能敏感的正则改用其他引擎）。

关联作业提示

本讲为可选趣味讲座，**不关联任何作业**。但它是对全课程的"压力测试"：理解 range-for 生命周期（L6 迭代器）、`iostream` 状态位（L4 流）、ABI 与编译/链接（L16 构建项目）都是对前序知识的巩固。若你正在做 **A7: Unique Pointer**，可顺带体会：如果标准库为 `unique_ptr` 打破 ABI 做了语言改动，你手写的简化版会有什么不同。

现代 C++ 核心特性速查表（Modern C++ Cheat Sheet）

> 按类别汇总 CS106L 全课程涉及的关键特性与语法。标准标注：C++11 / C++14 / C++17 / C++20 / C++23 / C++26。

1. 类型推导（Type Deduction）

特性	语法	标准	说明
`auto` 变量	`auto x = expr;`	C++11	编译器推导类型；仍是**静态类型**，`auto i = 1; i = "hi";` 编译错误
`auto` 返回类型	`auto f() { return 42; }`	C++14	返回类型由 `return` 推导
尾置返回类型	`auto f() -> int;`	C++11	显式指定返回类型
泛型 lambda / 模板参数	`auto` 形参：`[](auto x){...}`	C++14	`auto` 形参等价于模板参数
`decltype`	`decltype(expr) y = x;`	C++11	不求值地取表达式的声明类型
`decltype(auto)`	`decltype(auto) y = expr;`	C++14	保留引用/值语义地推导
结构化绑定	`auto [k, v] = map_pair;`	C++17	解构 pair/tuple/struct
`using` 类型别名	`using Zeros = std::pair<double,double>;`	C++11	替代 C++98 的 `typedef`，可带模板参数

2. 初始化（Initialization）

特性	语法	标准	说明
统一/列表初始化	`T obj {a, b, c};`	C++11	最通用、最安全的初始化方式；禁止窄化转换（narrowing）

特性	语法	标准	说明
成员初始化列表	`Foo(int x) : x_{x}, y_{} {}`	C++98（`{}` 版 C++11）	唯一能初始化 `const`/引用成员的方式；避免"先默认构造再赋值"的双重开销
默认成员初始化器	`int x_ = 0;`	C++11	类内给成员默认值
默认构造 `= default`	`Foo() = default;`	C++11	显式保留编译器生成的版本
删除函数	`Foo(const Foo&) = delete;`	C++11	禁止拷贝等操作（如 `unique_ptr`）

3. 引用与移动语义（References & Move Semantics）

特性	语法	标准	说明
左值引用	`int& r = x;`	C++98	别名；`x = y;` 两侧都可出现
`const` 左值引用	`const T& r = x;`	C++98	只读别名，可绑定临时值
右值引用	`T&& r = temporary;`	C++11	只能绑定临时对象（右值），用于"窃取"资源
移动构造/赋值	`T(T&& other) noexcept;` / `T& operator=(T&& other) noexcept;`	C++11	转移资源所有权，通常 O(1)
`std::move`	`std::move(x)`	C++11	仅把左值**转换**为右值引用（`static_cast<T&&>`），本身不移动任何东西
`std::forward`	`std::forward<T>(x)`	C++11	完美转发，保留实参的左右值类别
移动后状态	源对象"有效但未指定"	C++11	通常应把源指针置 `nullptr`；只允许对源对象销毁或重新赋值

4. 特殊成员函数（Special Member Functions）与规则

规则	内容	说明
六大 SMF	默认构造 `T()`、析构 `~T()`、拷贝构造 `T(const T&)`、拷贝赋值 `T& operator=(const T&)`、移动构造 `T(T&&)`、移动赋值 `T& operator=(T&&)`	需要时才由编译器隐式生成
Rule of Zero	成员都是自管理类型（`string`、`vector`、智能指针）时，什么都不用写	最推荐

规则	内容	说明
Rule of Three	需要自定义析构 ⇒ 通常也要自定义拷贝构造 + 拷贝赋值	手工管理资源（如 `new` / `delete`）时
Rule of Five	Rule of Three 成立时，通常还应定义移动构造 + 移动赋值	否则会退化为拷贝，性能受损

5. 智能指针与 RAII（Smart Pointers & RAII）

特性	语法	标准	说明
`std::unique_ptr`	`auto p = std::make_unique<T>(args);`	C++11	独占所有权；不可拷贝、可移动；离开作用域自动 `delete`
`std::shared_ptr`	`auto p = std::make_shared<T>(args);`	C++11	共享所有权，引用计数归零时释放；注意循环引用
`std::weak_ptr`	`std::weak_ptr<T> w = sp;` `w.lock()`	C++11	观察者，不增加引用计数，打破循环依赖
RAII 思想	资源在**构造**时获取、**析构**时释放	—	保证异常安全：析构函数必然被调用
其他 RAII 例子	`std::lock_guard`、`std::ifstream/ofstream`	—	锁、文件等在析构时自动释放

6. 模板（Templates）

特性	语法	标准	说明
模板类	`template <typename T> class Vector { ... };`	C++98	按需实例化，为每种 `T` 生成代码
模板函数	`template <typename T> T min(T a, T b);`	C++98	显式 `min<int>(...)` 或隐式推导
非类型模板参数	`template <size_t N> struct Array;` `std::array<T, N>`	C++11	编译期常量作参数；`array` 栈上分配
变参模板	`template <typename T, typename... Args>` + 包展开 `args...`	C++11	接受任意数量/类型参数，递归实例化
模板特化	`template <> struct Foo<int> {...};`	C++98	为特定类型提供专门实现
模板实现位置	`.h` 底部 `#include "Foo.cpp"`，定义用 `Foo<T>::`	—	实例化需要完整定义可见

7. Concepts 与编译期计算（C++20）

特性	语法	标准	说明
自定义 concept	`template <typename T> concept Comparable = requires(T a, T b) { {a < b} -> std::convertible_to<bool>; };`	C++20	在实例化前约束模板参数，显著改善错误信息
约束模板	`template <Comparable T>` 或 `template <typename T> requires Comparable<T>`	C++20	简写与完整两种写法等价
内置 concepts	`std::input_iterator`、`std::range`、`std::convertible_to`、`std::same_as` 等	C++20	标准库自带
`constexpr`	`constexpr size_t fact(size_t n);`	C++11（放宽于 C++14）	"请尽量在编译期求值"
`consteval`	`consteval size_t f(size_t n);`	C++20	"必须在编译期求值"
模板元编程（TMP）	`Factorial<N-1>::value` 递归结构	C++98	传统 TMP；现代用 `constexpr` 更可读

8. Lambda 与函数对象（Lambdas & Functors）

特性	语法	标准	说明
Lambda	`auto f = [captures](params) { body };`	C++11	编译器展开为匿名 functor 类
捕获列表	`[x]`（值）、`[&x]`（引用）、`[=]`、`[&]`、`[&, x]`、`[this]`、`[x = expr]`（初始化捕获 C++14）	C++11/14	值捕获复制；引用捕获注意生命周期
泛型 lambda	`[](auto x) { return x * 2; }`	C++14	等价于模板
函数指针	`bool(*pred)(char) = isVowel;`	C++98	只能指向无捕获的普通函数
仿函数（functor）	`struct G { bool operator()(int a, int b) const {...} };`	C++98	重载 `operator()` 的对象，可有状态
`std::function`	`std::function<bool(int)> f = lambda;`	C++11	统一容器类型（可存任何可调用对象），略慢
`std::bind` / `std::ref`	—	C++11	部分应用与引用包装（较少用了，lambda 更清晰）

9. STL 容器（Containers）

容器	头文件	特点	迭代器类别
`std::vector<T>`	`<vector>`	动态数组，随机访问 O(1)，尾部增删摊还 O(1)	随机访问
`std::deque<T>`	`<deque>`	双端队列，头尾增删均 O(1)	随机访问
`std::array<T, N>`	`<array>`	定长数组，栈上分配，大小编译期已知（C++11）	随机访问
`std::list<T>`	`<list>`	双向链表，任意位置插入 O(1)（需迭代器）	双向
`std::map<K,V>` / `std::set<K>`	`<map>` / `<set>`	有序（红黑树），需 `K` 有 `operator<`，查找 O(log n)	双向
`std::unordered_map<K,V>` / `std::unordered_set<K>`	`<unordered_map>` / `<unordered_set>`	哈希表，需 `std::hash<K>`，平均查找 O(1)	前向
`std::pair<T1,T2>`	`<utility>`	两个字段的泛型 struct	—
`std::tuple`	`<tuple>`	任意多个字段（C++11）	—

10. 算法与 Ranges（Algorithms & Ranges）

特性	语法	标准	说明
迭代器对算法	`std::sort(b, e)`、`std::find(b, e, v)`、`std::count_if(b, e, p)`、`std::copy_if(b, e, o, p)`、`std::transform(b, e, o, op)`、`std::unique_copy(b, e, o, p)`	C++98	`<algorithm>` 通用算法，作用于迭代器区间 `[first, last)`
范围（range）算法	`std::ranges::find(v, c)`、`std::ranges::sort(v)`	C++20	直接传容器；受 concepts 约束，错误信息更好
视图（view）	`auto v = c | std::views::filter(p) | std::views::transform(f);`	C++20	**惰性**组合，逐元素按需计算；类似 Python 生成器
物化视图	`std::ranges::to<std::vector<T>>(view)`	C++23	把惰性视图收集成容器
插入迭代器	`std::back_inserter(v)`	C++98	让算法向容器"推入"输出

11. 运算符重载（Operator Overloading）

特性	语法	标准	说明
成员重载	`bool operator<(const T& other) const;`	C++98	左操作数是 `this`
非成员重载	`bool operator<(const T& a, const T& b);`	C++98	更对称、更惯用；左操作数可为非类类型
`friend`	`friend bool operator<(const T&, const T&);`	C++98	非成员函数访问私有成员
流插入/提取	`std::ostream& operator<<(std::ostream&, const T&);`	C++98	让 `std::cout << obj` 可用；返回流以支持链式
规则	语义应显然（Principle of Least Astonishment）；`==` 与 `!=` 成对（rule of contrariety：`!=` 用 `!(a==b)` 实现）；不可重载 `::` `?:` `.` `.*` `sizeof` `typeid`	—	运算符的意义必须符合直觉，否则用命名函数

12. 类型安全与 `std::optional`（C++17）

特性	语法	标准	说明
`std::optional<T>`	`std::optional<int> o;` / `o = 42;` / `o = std::nullopt;`	C++17	可能包含值，也可能为空
判空与取值	`o.has_value()`、`o.value()`（空则抛 `std::bad_optional_access`）、`o.value_or(0)`、`bool(o)` / `if (o)`	C++17	空 optional 隐式转换为 `false`
Monadic 操作	`o.and_then(f)`（f 返回 optional）、`o.transform(f)`（f 返回值）、`o.or_else(f)`	C++23	链式处理"可能失败"的计算
`std::nullopt` vs `nullptr`	`nullopt` 用于 optional；`nullptr` 用于指针	C++17	不要混淆
类型安全理念	"Well typed programs cannot go wrong."	—	用类型系统把"可能失败"写进签名

13. 并发与杂项（C++11 起）

特性	语法	标准	说明
线程	`std::thread t(f, args); t.join();`	C++11	`<thread>`
互斥锁	`std::mutex m;` `std::lock_guard<std::mutex> lk(m);`	C++11	RAII 管理锁
原子操作	`std::atomic<int> counter;`	C++11	`<atomic>`，无锁编程基础
`nullptr`	`T* p = nullptr;`	C++11	类型安全的空指针（替代 `NULL` / `0`）
`enum class`	`enum class Color { Red, Green };`	C++11	强类型、有作用域的枚举
`override` / `final`	`void update() override;` / `class D final {};`	C++11	显式覆写/终结虚函数，防拼写错误
`noexcept`	`void f() noexcept;`	C++11	声明不抛异常；移动构造标 `noexcept` 让容器优先移动
range-for	`for (const auto& x : c) {...}`	C++11	基于迭代器的语法糖
字符串字面量	`u8"..."`、`R"(raw)"`、`"..."s`（`std::string` 字面量）	C++11/14	编码与原始字符串

笔记完。祝学习愉快！ 🌽